

Dear VLDB Meta-Reviewer and Reviewers,

We sincerely thank the Meta-Reviewer and Reviewers for their thorough reading and insightful feedback. In response, we have made substantial revisions to the paper, including the introduction of real-world datasets, rigorous scalability analyses, clearer articulation of novelty, and additional baseline comparisons. The updated contents in the paper are highlighted in blue. Figures and tables in this response letter are numbered with Roman numerals (e.g., Table I and Figure II), while those in the main paper use Arabic numerals (e.g., Table 1 and Figure 2).

Below, we first summarize the **main changes**, and then present **detailed responses** to each reviewer and the meta-reviewer.

Our Main Changes

M1 [Seven Groups of New Experiments]: We have significantly revised the experimental sections in response to reviewer feedback, incorporating seven new groups of experiments.

- (1) We have added real-world evaluation on the Buildings [36] dataset, a benchmark comprising 105 ADP tasks derived from data processing pipelines collected from 21 energy companies. We compare DEEPPREP with strong baselines and report accuracy, completion rate, and inference cost. The experimental results are reported in Table II.
- (2) We have evaluated the scalability of DEEPPREP in terms of data sizes, table numbers and pipeline lengths. We reported the accuracy, complete rate and resource usage in cases with varying data sizes, table numbers and pipeline lengths. The experimental results are shown in Figure III, Figure IV, Figure VII, Figure VIII and Table III
- (3) We have conducted sensitivity analysis of exploration turns. We report the complete rate and accuracy of DEEPPREP when varying maximum exploration turns. The experimental results are reported in Figure I and Figure II.
- (4) We have compared DEEPPREP with two agentic coding baselines: Claude Code (CC), and the open-source code agent TRAE. The experimental results are reported in Table I and Figure 5.
- (5) We have added an overall ablation study to isolate how much of the performance gain comes specifically from tree-based reasoning framework and progressive agentic training. The experimental results are reported in Table IV, with detailed analysis provided in Exp-3 of Section 6.4.
- (6) We have conducted error analysis of DEEPPREP in real-world cases to determine its strength and limitations. The analysis is provided in Exp-6 of Section 6.4.
- (7) We have added a finer-grained analysis of operator coverage and compositional diversity between Synth-Spider and Parrot, measuring unique bigrams and trigrams, concentration ratios, and cross-category ratios. We also conduct a cross-training experiment comparing models trained on Synth-Spider vs. Parrot, both evaluated on the Parrot test set. We provide detailed analysis in Exp-5 of Section 6.4.

M2 [Technical Details]: We have revised Section 6 to provide more comprehensive technical details. In addition, we have included

Table I: Comparison between DEEPPREP and Agentic Coding Systems.

LLM	Method	Synth-Spider		Synth-Bird		Parrot	
		Acc.	Cost (\$)	Acc.	Cost (\$)	Acc.	Cost (\$)
GPT-5	TRAE	72.36	0.038	61.75	0.033	41.19	0.041
	DEEPPREP	71.76	0.014	60.46	0.015	41.33	0.009
Claude-4	CC	72.71	0.169	68.33	0.141	42.98	0.188
	TRAE	73.80	0.148	66.93	0.160	41.93	0.125
	DEEPPREP	73.46	0.054	67.43	0.062	46.07	0.044

detailed LLM prompts and hyper-parameters of all baselines implemented in this paper in our technical report [1] due to space limitations. We have also made the following clarifications: (i) We clarified that the environment is deterministic and replaced the probabilistic notation with a deterministic transition function. (ii) We revised Section 4 to explain how failure branches are preserved with error traces and reused to inform future exploration. (iii) We added detailed discussions of tree-search control, implicit pruning mechanisms, and complexity analysis ($O(T \times L)$) in Section 4.2.

M3 [Clarification on Technical Novelty]: We have clarified the technical innovations of DEEPPREP beyond existing agentic LLM frameworks from two aspects: (i) The agentic reasoning tree with materialized execution states and failure-aware branching, which differs from ToT that operates over text and MCTS that uses scalar reward signals. (ii) Progressive agentic training with environment token masking and hybrid reward design. And reversible noise injection with verification-in-the-loop for data synthesis.

Responses to Reviewer #1

R1W1: Recent agentic coding systems (e.g., Claude Code, Codex) already support planning-coding-testing loops with environment feedback. The paper does not discuss and compare against such strong general-purpose coding agents, leaving unclear whether a customized ADP-specific agent is necessary.

RESPONSE: We thank the reviewer for the insightful suggestion. Following this comment, we have **added two strong agentic coding systems as baselines: Claude Code (CC)¹ and TRAE²**. These systems represent state-of-the-art general-purpose coding agents that implement planning-coding-testing loops with environment feedback. Specifically, we evaluate CC using the Claude-4 backbone, as required by its framework. We also include TRAE, an open-source coding agent from ByteDance, which has shown strong performance on SWE-bench [19].

As shown in Table I, DEEPPREP achieves accuracy comparable to CC and TRAE across all benchmarks while *consistently reducing cost*. With GPT-5, DEEPPREP achieves accuracy comparable to TRAE while reducing cost by 2.7× on Synth-Spider, 2.2× on Synth-Bird, and 4.6× on Parrot. With Claude-4, DEEPPREP achieves accuracy comparable to both CC and TRAE, while achieving the best accuracy on Parrot and reducing cost by 2.7×–4.3×. These results indicate that DEEPPREP provides a more effective cost-accuracy trade-off than general-purpose coding agents, highlighting the value of a task-specific ADP agent, particularly in cost-sensitive settings.

The advantage of DEEPPREP is mainly attributed to its task-specific design. First, DEEPPREP produces intermediate tables as structured

¹Claude Code v2.1.119

²<https://github.com/bytedance/trae-agent>

feedback after each operator, enabling precise diagnosis of pipeline failures. In contrast, general-purpose coding agents must localize errors within long code blocks, often requiring additional debugging iterations. Second, DEEPPREP generates only operators and their arguments, which are automatically translated into executable data preparation programs. In contrast, coding agents repeatedly regenerate full code blocks, leading to higher token consumption.

We have revised **Section 6.1** and **Section 6.2** to include the additional baselines, experimental results, and analysis discussed above.

R1W2: Several key design aspects of the tree-based exploration, e.g., prompt construction, state representation, schema handling, and reuse of failed branches, are not sufficiently specified, limiting reproducibility and conceptual clarity.

R1D1: Several technical details are missing in the paper: (1) What information is included in prompts? Schema? Table content? Intermediate table? (2) Are schemas of intermediate tables included in prompt? If so, how are intermediate schemas derived? (3) Whether full table contents are provided to the LLM or summarized (e.g., via sample-based representations) to control token usage.

RESPONSE: We thank the reviewer for pointing out the missing technical details. Following this suggestion, we have clarified the implementation of DEEPPREP in five aspects.

Prompt Construction. The prompt of DEEPPREP consists of the following three components:

- **System-level instruction**, which specifies the Tree-based Agentic Reasoning mechanism described in Section 4.
- **ADP task description**, including the source tables and target schema. For each source table, we provide its schema together with sampled table content. For the target schema, we provide natural-language specifications as shown in Figure 2.
- **Intermediate tables.** During reasoning, DEEPPREP generates data preparation operators that are executed to produce intermediate tables. The resulting intermediate tables are serialized and included in subsequent prompts together with their schemas and sampled content.

Note that all source and intermediate tables are serialized in markdown format.

Schema Handling. Yes, schemas of intermediate tables are included in prompts. Intermediate schemas are derived from the source tables and the generated operators. For example, given a source table containing a column “birth”, DEEPPREP may generate an operator `AddNewColumn(table, name, func)` to create a new column “birth-year” using an extraction function `func`. The schema of the resulting intermediate table therefore consists of the original schema plus the newly generated column.

Table Content Representation. To control token usage, DEEPPREP provides sampled table content rather than full tables. Specifically, we randomly sample 20 columns and 20 rows for source tables, and 8 columns and 5 rows for intermediate tables.

State Representation. As defined in Section 4, a state is represented by the *current working set of tables* produced during data preparation. For example, suppose the initial state n_0 contains a source table A . After applying an operator that splits A into tables

Table II: Results on the Buildings dataset (real-world).

Method	Acc.	Comp.	Cost (\$)
ReAct	74.29	99.05%	0.0052
CodeGen	71.43	100%	0.0054
MontePrep	72.38	86.67%	0.043
MCTS-OP	69.52	99.05%	0.057
DEEPPREP (Ours)	82.85	100%	0.0082

B and C , the new state n_1 consists of $\{B, C\}$. Then, if a subsequent operator transforms B into D , the next state n_2 becomes $\{D, C\}$. In other words, newly generated tables replace their source tables in the state representation.

Reuse of Failed Branches. When expanding a failed branch, DEEPPREP preserves both the erroneous operator and its execution error log. These records are incorporated into subsequent exploration to avoid repeating previously observed failures.

We have revised the paper accordingly. Specifically, we added the above technical details to **Appendix A.1** (Tree-based Agentic Reasoning) and provided additional implementation examples in **Appendix B.1** (Implementation Details of Baselines). Please refer to our technical report [1].

R1W3: Most experiments rely on synthesized benchmarks derived from structured SQL datasets (e.g., Spider). Real-world data preparation often involves messy, underspecified, or ambiguous requirements. The evaluation provides limited evidence of robustness in such open-ended scenarios.

RESPONSE: We thank the reviewer for raising this important concern. To evaluate robustness in more realistic and open-ended settings, we have **added a new experiment (Exp-6) on the real-world Buildings dataset [36]**.

Specifically, the Buildings dataset contains 105 real-world ADP tasks and is particularly relevant to this concern for two reasons. First, it involves messy data. Column names are often abbreviated or domain-specific (e.g., `hdd`, `cdd`), and the same concept may be represented by different names across tables. Second, it contains ambiguous data preparation requirements. For example, target specifications are often described only at a high level, without explicitly defining important details such as the desired aggregation granularity (e.g., daily versus monthly).

Table II reports the results. Consistent with the observations on Synth-Spider, Synth-Bird, and Parrot, DEEPPREP achieves the best trade-off between accuracy and cost on this real-world benchmark. These results provide additional evidence that DEEPPREP remains effective under noisy schemas and underspecified requirements, beyond the settings considered in the original submission.

We have revised **Section 6.1** to include the new dataset and added experimental results and analysis to **Exp-6** of **Section 6.4**.

R1D2: The benefit of the tree structure for agentic inference is not clearly justified. According to the illustrations and descriptions, the branches appear to originate from failure attempts. Will these nodes be reused to inform future exploration? If not, why do not discard these failure attempts?

RESPONSE: We thank the reviewer for this insightful question. Failure nodes are not discarded in DEEPPREP. Instead, they are explicitly reused to guide subsequent exploration.

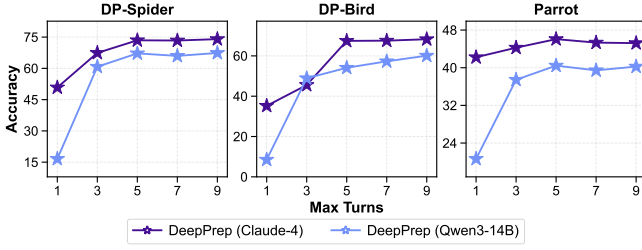


Figure I: Accuracy of DEEPPREP when changing the maximum number of exploration turns.

Specifically, a failure node is typically caused by a runtime error when executing a generated operator. For each failure node, DEEPPREP records both the generated operator and its execution error trace. This information is incorporated into subsequent prompts, enabling the agent to avoid repeating the same mistakes and generate corrected operators in future exploration steps. For example, in Example 4, Step 2 fails because the generated ExeCode operator references a non-existent column “year” in table ratings. DEEPPREP records the corresponding error trace and appends it to the prompt. In the next exploration step, the agent analyzes the error and generates a revised operator that preserves the required column when applying SelectColumn, thereby correcting the failure.

More importantly, the tree structure allows DEEPPREP to preserve exploration history rather than treating each attempt independently. Failure nodes provide explicit negative feedback about invalid operations, while sibling branches represent alternative transformation strategies. Together, they guide subsequent exploration and improve search efficiency.

We have revised **Section 4.1** to clarify the role of failure nodes and how they are reused during tree-based exploration.

R1D3: The paper would benefit from deeper sensitivity analyses. For instance, how does the system’s accuracy and completion rate vary with the maximum number of exploration turns? how does the agentic system scale when the underlying source tables grow in row or column size? Will the system collapse?

RESPONSE: As suggested, we have conducted additional **sensitivity analysis** on both **exploration turn** and **data size**. We also examine the **scalability** of DEEPPREP by reporting its time and memory usage under varying data characteristics.

Sensitivity Analysis by Varying Exploration Turns. Figure I and Figure II show the impact of the maximum number of exploration turns on accuracy and completion rate. Overall, the performance of DEEPPREP improves as the exploration turn increases and stabilizes at around five turns across all datasets and model backbones. For example, on Claude-4, increasing the budget from one to five turns improves accuracy by 22.63 points on Synth-Spider and 32.21 points on Synth-Bird. Beyond five turns, the additional gain is less than one point.

Sensitivity Analysis by Varying Data Sizes. To evaluate robustness under different data sizes, we analyze performance from two perspectives: column size and row size. Specifically, we sort all cases by the number of columns and rows, respectively, and divide them into five equal-width buckets ($b_1 - b_5$). Figure III and Figure IV report the corresponding results.

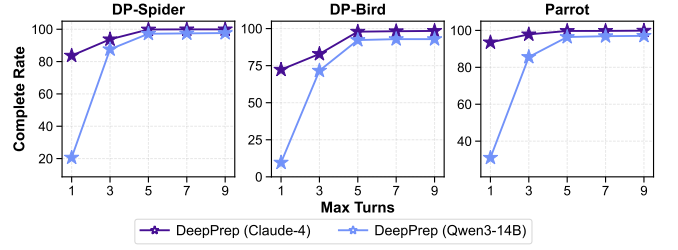


Figure II: Completion rate of DEEPPREP when changing the maximum number of exploration turns.

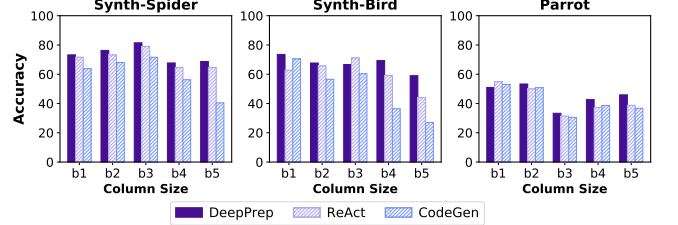


Figure III: Accuracy comparison on tables with varying number of columns. For Synth-Spider, b_1 (1-4), b_2 (5-7), b_3 (8-9), b_4 (10-14), b_5 (15-124,843). For Synth-Bird, b_1 (1-11), b_2 (12-27), b_3 (29-74), b_4 (75-1,003), b_5 (1,004-20,015). For Parrot, b_1 (2-4), b_2 (5-7), b_3 (8-11), b_4 (12-16), b_5 (18-587).

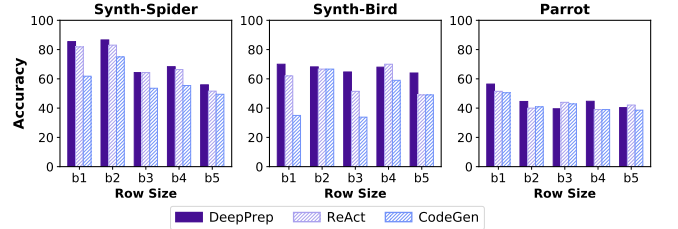


Figure IV: Accuracy comparison on tables with varying number of rows. For Synth-Spider, b_1 (1-9), b_2 (10-15), b_3 (16-21), b_4 (22-35), b_5 (36-124,842). For Synth-Bird, b_1 (1-110), b_2 (115-1,000), b_3 (1,001-1,022), b_4 (1,023-2,004), b_5 (2,005-360,006). For Parrot, b_1 (2-13), b_2 (14-30), b_3 (31-100), b_4 (107-2,000), b_5 (2,075-200,000).

The results show that DEEPPREP remains robust on small and medium-sized tables. For example, on Synth-Spider, accuracy remains stable when the number of columns is below nine. Performance degrades only for the largest buckets. For cases containing (36 – 124,842) rows (i.e., b_5), accuracy decreases by 29.54 points compared with cases containing (1 – 9) rows (i.e., b_1). This degradation is primarily caused by the table sampling strategy used to control token consumption. When tables become very large, relevant information may be omitted from the sampled content and therefore unavailable to DEEPPREP. We leave more sophisticated sampling strategies for large tables as future work.

Sensitivity Analysis on Time and Resource Usage. To address the concern of whether DEEPPREP collapses as table size grows, we further report average execution time and memory usage across different row-size and column-size buckets. As shown in Table III (in the response of R2W4 & R2D4 of Reviewer #2), both metrics remain stable as data size increases. For example, on Synth-Spider,

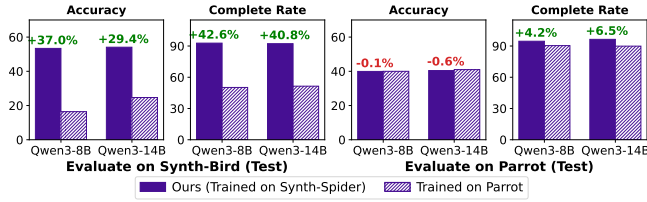


Figure V: Evaluation results of models trained on Synth-Spider (Train) and Parrot (Train).

the average execution time increases only from 8.91 to 11.30 seconds when moving from the smallest to the largest column-size bucket, while memory usage remains between 66 MB and 69 MB. Similar trends are observed on Synth-Bird and Parrot. These results suggest that DEEPPREP scales with increasing table sizes and does not suffer from system collapse in terms of execution time or memory consumption.

We have revised **Exp-7** of **Section 6.4** to include the above experiments and discussions. A complete version can be found in **Appendix D.2-D.5** of our technical report [1].

R1D4: The explanation attributing poor transfer to pipeline length distribution (e.g., Parrot vs. Synth-Spider) may be incomplete. In my opinion, operator-type distribution shift could be a more significant factor, especially in the case where two datasets (Synth-Spider and Synth-Bird) share the same synthesis strategy. A finer-grained analysis of operator coverage and compositional diversity would provide a stronger causal explanation.

RESPONSE: We thank the reviewer for this insightful suggestion. As suggested, we conducted a finer-grained analysis of operator coverage and compositional diversity across Synth-Spider, Synth-Bird, and Parrot. The results confirm that operator-type distribution shift is indeed the primary factor explaining transfer performance, rather than pipeline length distribution.

Operator Coverage. Our Synth-Spider dataset provides broader operator coverage than Parrot in three aspects. First, as shown in Table 1, Synth-Spider supports 14 additional types of operators compared with Parrot, including commonly used data cleaning operations such as Missing Value Imputation, as well as common schema editing operations such as Concatenation. Second, our operators support function arguments, enabling more customized data preparation operations. For example, AddNewColumn can take a lambda expression as an argument to define a customized mapping from source columns to the newly created column.

Operator Distribution Similarity. The unigram Jensen-Shannon Divergence (JSD) between Synth-Spider and Synth-Bird is only 0.025, with a cosine similarity of 0.952, indicating nearly identical distributions. In contrast, the JSD between Synth-Bird and Parrot is 0.264 (**10.6× larger**), and the bigram and trigram JSDs reach 0.660 and 0.886 respectively, indicating nearly disjoint compositional patterns.

Compositional Diversity. Synth-Spider and Synth-Bird produce 496 and 417 unique bigrams (2,875 and 1,947 unique trigrams), while Parrot yields only 189 and 758 respectively. Synth-Spider and Synth-Bird also exhibit higher cross-category bigram ratios (0.799/0.765

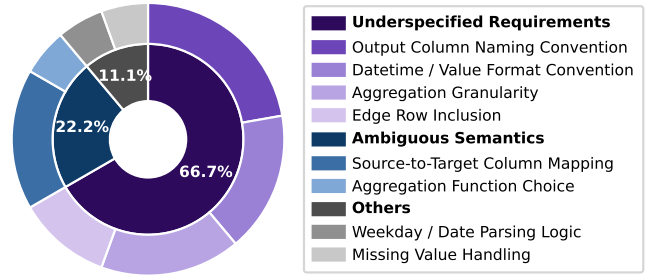


Figure VI: Error analysis on Buildings.

vs. Parrot’s 0.708), meaning their operator combinations span more diverse functional categories.

These results explain the strong cross-transfer between Synth-Spider and Synth-Bird: their shared synthesis strategy yields consistent, broad-coverage operator distributions with rich compositional diversity. However, one might argue that this transfer success is trivially expected given the distributional similarity. To provide a more rigorous test, we compare two models, one trained on Synth-Spider and the other trained on Parrot, both evaluated on the Parrot test set (**out-of-domain vs. in-domain**). As shown in the right part of Figure V, the model trained on Synth-Spider achieves comparable accuracy to the model trained on Parrot (40.44 vs. 40.99 on Qwen3-14B), while obtaining a notably higher completion rate (96.41% vs. 89.89%). This suggests that Synth-Spider’s broader operator coverage and more diverse operator distribution expose the model to a richer set of operator compositions during training, leading to stronger exploration capability and higher completion rates even on an out-of-domain test set, which further confirms that operator-type distribution, rather than pipeline length, is the primary factor governing transfer performance.

We have revised **Exp-5** of **Section 6.4** with this updated experiment.

R1D5: The paper would be strengthened by discussing the limitations with a dedicated analysis of real-world failure cases.

RESPONSE: To better understand the limitations of DEEPPREP in realistic settings, we have **conducted a failure-case analysis on the Buildings dataset**, which consists of real-world ADP tasks. We added a new experiment, **Exp-6**, in Section 6.4.

As summarized in Figure VI, most failures can be attributed to the following two key challenges commonly encountered in real-world data preparation.

Ambiguous Column Semantics. In synthesized benchmarks, column names are typically self-descriptive (e.g., “movie-title”, “release-year”), making source-to-target mappings relatively straightforward. In contrast, real-world datasets often contain abbreviated or domain-specific column names (e.g., “hdd”, “cdd”), and the same concept may be represented differently across tables. This ambiguity leads to *semantic column mapping errors*, which account for approximately 50% of all failures. For example, a source column “hdd” (heating degree days) may need to be transformed into a target column *heating-demand*, but the semantic relationship is difficult to infer from the column name alone.

Underspecified Data Preparation Requirements. Another major challenge is that real-world target specifications are often incomplete. Unlike synthesized benchmarks, which explicitly define target schemas and transformation objectives, real-world tasks frequently describe desired outputs only at a high level. Important details, such as aggregation granularity (e.g., daily versus monthly) or aggregation semantics (e.g., sum versus average), may be omitted. As a result, DEEPPREP may generate incorrect grouping strategies or aggregation functions, leading to *granularity mismatch errors* (22% of failures) and *domain logic errors* (11% of failures).

These findings suggest that future work should focus on improving schema-aware reasoning under ambiguous semantics and incorporating mechanisms for resolving underspecified user requirements. Such capabilities are likely to be important for deploying autonomous data preparation systems in real-world environments.

We have revised **Section 6.4** and added an experiment **Exp-6** to include the above failure-case analysis and discussion. A complete version of the failure-case analysis can be found in **Appendix D.6** of our technical report [1].

Responses to Reviewer #2

R2W1: *Most components of the proposed framework resemble existing agentic LLM frameworks.*

R2D1: *The overall architecture combines several elements that are now common in recent LLM-based agentic systems: iterative plan-execute loops, execution feedback, structured search over candidate solutions, reinforcement learning training, and synthetic task generation. While the integration of these components for autonomous data preparation is interesting, the paper would benefit from a clearer description of which aspects represent new technical contributions. In particular, the authors should help the reader to understand more precisely what algorithmic insight or system capability distinguishes the proposed approach from existing agentic solutions.*

RESPONSE: We thank the reviewer for this important comment. While DEEPPREP incorporates several techniques that have appeared in prior agentic systems, such as execution feedback, tree search, and reinforcement learning, its contribution is not a simple combination of existing components. Autonomous data preparation (ADP) differs fundamentally from tasks such as code generation and Text-to-SQL: it requires reasoning over evolving table states and multi-step data transformations. To address these challenges, DEEPPREP introduces a reasoning framework and training methods specifically designed for ADP.

Specifically, the novelty of DEEPPREP lies in two aspects.

(1) Tree-based Agentic Reasoning for ADP. We introduce Tree-based Agentic Reasoning, a structured reasoning mechanism for multi-step data preparation. We further compare the mechanism with existing solutions to highlight its novelty. Unlike ReAct [43], DEEPPREP maintains explicit table states and supports rollback to previously valid states. Unlike Tree-of-Thought [42], exploration is grounded in executable data transformations and execution feedback. Unlike MCTS-based approaches [24], the search process is represented as an explicit reasoning tree that can be serialized into prompts, enabling global reasoning over both successful and failed exploration trajectories.

(2) Progressive Agentic Training for ADP. Existing agentic systems are primarily designed for inference-time exploration and are not explicitly trained to follow a structured reasoning protocol. To address this gap, DEEPPREP introduces Progressive Agentic Training, which trains the model to perform Tree-based Agentic Reasoning through control-tag supervision, execution-aware optimization, and a hybrid reward that combines outcome, partial, and process rewards. This enables effective multi-turn reasoning and exploration for ADP tasks. Moreover, DEEPPREP introduces a verification-in-the-loop mechanism to construct training data tailored for ADP tasks. Unlike standard data augmentation that applies random perturbations without guaranteeing solvability, DEEPPREP injects noise using inverse operators of data-cleaning functions (e.g., the inverse of date-standardization converts normalized dates into heterogeneous formats), and verifies reversibility by applying the corresponding cleaning operator to ensure the previous table state is restored. This guarantees that every synthesized task has a valid pipeline solution, which random noise injection cannot ensure.

To address the reviewer’s concern, we have added a new paragraph, i.e., **Differences from existing agentic solutions**, in the **Introduction** (Section 1) to more clearly distinguish the technical contributions of DEEPPREP from existing agentic frameworks.

R2W2: *Much of the evaluation uses datasets synthesized from NL2SQL benchmarks, which may not capture realistic data preparation challenges.*

R2D2: *A substantial portion of the experimental evaluation relies on datasets synthesized from NL2SQL benchmarks. While this strategy enables the generation of large datasets with long transformation pipelines, such synthetic tasks may not fully capture the complexity of real-world data preparation scenarios. In practice, data preparation often involves data quality issues that are difficult to reproduce through synthetic generation. I would suggest adding experiments on real-world datasets or practical case studies to strengthen the empirical evidence and provide a clearer picture of the system’s applicability in realistic settings.*

RESPONSE: We thank the reviewer for this valuable suggestion. To better evaluate the applicability of DEEPPREP in realistic settings, we have **added a new real-world dataset, Buildings [36], and conducted a new experiment (Exp-6) on the dataset.**

The Buildings dataset contains 105 real-world ADP tasks collected from data processing pipelines in the smart building domain across 21 energy companies. Specifically, it contains practical data quality issues commonly encountered in real-world data preparation. For example, column names are often abbreviated or domain-specific (e.g., “hdd”, “cdd”), and the same concept may be represented using different names across tables. These characteristics require semantic reasoning beyond schema matching and therefore provide a more realistic evaluation of autonomous data preparation systems.

We compare DEEPPREP with all ADP baselines in terms of accuracy, completion rate, and API cost. As reported in Table II, DEEPPREP achieves the best overall performance. Specifically, it attains a 100% completion rate and improves accuracy by 8.56 points over

the second-best method. At the same time, its API cost remains relatively low compared with competing approaches. Therefore, consistent with the results on Synth-Spider, Synth-Bird, and Parrot, DEEPPREP achieves the best trade-off between accuracy and cost on this real-world dataset.

We have revised **Section 6.1** and **Section 6.4** to include the Buildings benchmark, experimental results, and analysis. Moreover, we have added a practical case study to provide a clearer picture of the system’s applicability in realistic settings, please refer to **Appendix E** in our technical report [1] for more details.

R2W3: The experimental setup provides limited information on prompt design and tuning for baseline methods.

R2D3: The experimental setup includes several baseline methods, but the description of prompt design, parameter tuning, and configuration choices for these baselines is relatively limited. Since the performance of LLM-based baselines can depend heavily on prompt engineering and inference parameters, it is difficult to assess whether the comparisons are fair. The authors should provide more detailed information on how these baselines were implemented, which would make it easier to assess the fairness and reproducibility of the comparisons.

RESPONSE: We thank the reviewer for this important comment. To ensure fair and reproducible comparisons, we apply a consistent configuration and tuning method across all baselines and provide the complete prompts, hyper-parameters, and implementation details in our technical report [1] (due to the space limit).

Specifically, for methods with publicly available implementations, including MontePrep [11], DeepAnalyze [45], AutoPrep [9], and MCTS-Op [24], we use their official codebases and default configurations (e.g., prompt design, parameters, etc.). When a method was originally designed for a different task and could not be directly applied to ADP, we adapted its prompt to the ADP setting and tuned its parameters using the same grid-search procedure adopted by DEEPPREP. For example, AutoPrep was originally designed for TableQA rather than autonomous data preparation, so we replaced its original prompt with an ADP-oriented prompt and tuned it accordingly.

For methods without publicly available implementations, including CodeGen, ReAct, and PaS, we reimplemented them following the descriptions in their papers and applied the same prompt-tuning and parameter-selection procedure used for DEEPPREP. Therefore, all methods were evaluated under a comparable tuning method.

To improve reproducibility, we have added the description of prompts, inference parameters, and configuration choices for all baselines in **Section 6.1**. Complete prompts, hyperparameters, and implementation details are also provided **Appendix B** in our technical report [1].

R2W4: The evaluation mainly reports inference cost but lacks a systematic analysis of scalability with respect to data size, schema complexity, or pipeline length.

R2D4: The paper evaluates efficiency primarily through an inference cost metric based on runtime and GPU pricing. While this metric provides a useful indication of practical deployment cost, the evaluation does not include a systematic analysis of scalability. In particular, it is not clear how the system behaves as input data size increases, as

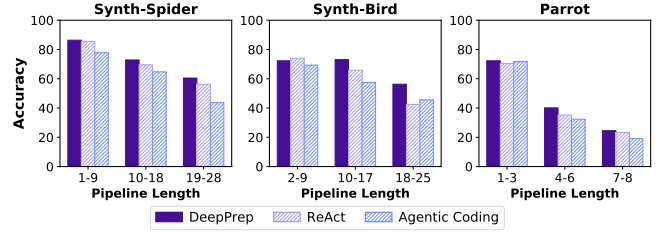


Figure VII: Comparison on Accuracy with varying Pipeline Length.

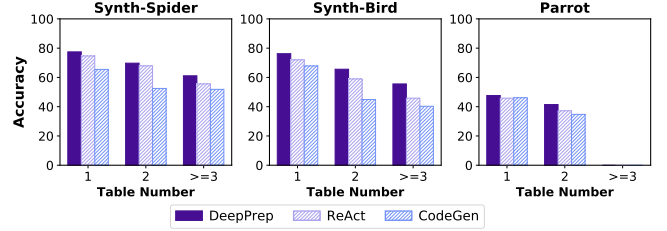


Figure VIII: Comparison on Accuracy with varying number of source tables.

Table III: Average Execution Time (s) and Memory Usage (MB) under Varying Data and Task Complexity.

Data Type	Group	Synth-Spider		Synth-Bird		Parrot	
		Time	Mem.	Time	Mem.	Time	Mem.
Column Size	b1	8.91	66.31	11.46	77.34	7.09	114.42
	b2	8.21	66.54	12.13	74.07	9.76	116.51
	b3	8.48	67.07	12.12	85.35	11.49	125.12
	b4	10.19	69.32	12.88	83.31	10.36	137.34
	b5	11.30	68.37	13.78	90.70	11.36	143.35
Row Size	b1	7.87	65.69	11.48	88.66	9.54	124.43
	b2	8.13	65.99	11.36	74.92	10.57	124.72
	b3	9.30	66.78	13.57	79.24	9.07	120.89
	b4	10.63	68.33	12.64	80.00	10.13	127.47
	b5	11.72	71.30	13.96	88.31	10.77	135.00
Operator Length	g1	8.54	66.74	11.26	80.08	6.85	114.01
	g2	12.75	70.30	14.58	85.02	11.06	130.69
	g3	14.62	68.53	14.01	81.92	12.22	132.31
Table Number	1	8.06	66.51	9.94	79.00	9.58	124.41
	2	10.54	68.82	13.11	83.38	10.81	128.74
	≥3	13.49	68.54	14.92	81.08	–	–

the number of tables grows, or as pipeline length and reasoning-tree exploration become more complex. Additional experiments analyzing runtime and resource usage under varying workload sizes would provide a clearer understanding of the scalability of the proposed approach and would strengthen the evaluation of the system expected for a VLDB paper.

RESPONSE: We thank the reviewer for this valuable suggestion. We have added a new scalability study that evaluates DEEPPREP under varying row sizes, column sizes, pipeline lengths, and table numbers. We further report runtime and memory usage across different complexity groups to provide a systematic understanding of the scalability of DEEPPREP.

Time and Resource Usage. We group cases according to row size, column size, pipeline length, and table number, and report the average execution time and memory usage in Table III. In particular,

Algorithm 1 DEEPPREP: Tree-Based Agentic Inference

Require: Source tables S and target schema Σ^*

```
1: Initialize  $G_0 = (N, E)$  with  $N = \{n_0\}$ ,  $n_0 \leftarrow S$ 
2: while the agent does not use  $\langle \text{answer} \rangle$  do
3:    $\psi_t \sim \text{Pr}_{\text{LLM}}(\cdot \mid G_{t-1}, \Sigma^*, \psi_{<t})$   $\triangleright \langle \text{plan} \rangle$ 
4:    $\mathcal{P}_t \sim \text{Pr}_{\text{LLM}}(\cdot \mid G_{t-1}, \psi_t)$   $\triangleright \langle \text{expand} \rangle$  from chosen parent
5:    $n_{\text{curr}} \leftarrow$  parent node identified by  $\psi_t$ 
6:   for each operator  $o$  in  $\mathcal{P}_t$  do
7:      $\mathcal{T}_{\text{out}} \leftarrow o(n_{\text{curr}})$   $\triangleright \langle \text{execute} \rangle$  operator on current state
8:     if execution succeeds then
9:        $n_{\text{new}} \leftarrow$  new node with state  $\mathcal{T}_{\text{out}}$ 
10:       $N \leftarrow N \cup \{n_{\text{new}}\}$ ;  $E \leftarrow E \cup \{(n_{\text{curr}} \xrightarrow{o} n_{\text{new}})\}$ 
11:       $n_{\text{curr}} \leftarrow n_{\text{new}}$ 
12:    else
13:      Annotate  $n_{\text{curr}}$  with failure trace
14:    break
15:  end if
16: end for
17:   $G_t \leftarrow (N, E)$ 
18: end while
19:  $n_{\text{leaf}} \leftarrow \text{select}(\cdot \mid S, \Sigma^*, G_t)$ 
20:  $D^* \leftarrow n_{\text{leaf}}$ 
21: return  $D^*$ 
```

for row-size and column-size analyses, we sort all cases by the corresponding metric and divide them into five equal-width buckets $b_1 - b_5$ (detailed bucket ranges for each dataset will be reported later). As shown in the table, DEEPPREP exhibits stable growth in both execution time and memory usage as workload complexity increases. For example, on Synth-Spider, the average execution time increases only from 8.91 to 11.30 seconds when moving from the smallest to the largest column-size bucket, while memory usage remains between 66 MB and 69 MB. Similar trends are observed on Synth-Bird and Parrot. These results indicate that DEEPPREP scales gracefully with increasing data size and task complexity.

Why DEEPPREP Scales Stably. The stable scaling behavior can be explained by the generative search framework formalized in Algorithm 4. At each step, the agent generates a $\langle \text{plan} \rangle \psi_t$ (Line 3) and an $\langle \text{expand} \rangle$ action (Line 4), followed by the execution of the proposed operators (Line 7). The process is bounded by the maximum exploration turn T and the maximum operator sequence length L per expansion, yielding a worst-case complexity of $O(T \times L)$. Unlike classical tree-search methods such as MCTS, which explicitly expand and evaluate many candidate branches, DEEPPREP generates only one operator pipeline conditioned on the current tree state. The “pruning” is implicit in the LLM’s decision-making: the agent observes failure traces and avoids revisiting failed branches, rather than enumerating all possible expansions. Consequently, the effective branching factor depends on the LLM’s learned policy and remains bounded regardless of input size, which explains the stable growth in time and memory usage.

We have revised **Exp-7 (Scalability Analysis)** of **Section 6.4** to include the above experiments and discussion. A complete version can be found in **Appendix D.2-D.5** in our technical report [1]. *Additional Experimental Details.* For row-size and column-size analyses, we sort all cases by the corresponding metric and divide them into five equal-width buckets $b_1 - b_5$, as described below.

Algorithm 2 Direct ToT Adaptation for Data Preparation

Require: Source tables S and target schema Σ^*

```
1: Initialize a textual reasoning tree  $G_0 = (N, E)$  with  $N = \{n_0\}$ 
2:  $n_0 \leftarrow \text{TextSerialize}(S, \Sigma^*)$ 
3: while the search budget is not exhausted do
4:    $n_i \leftarrow \text{Select}(G_{t-1})$ 
5:    $\{z_t^{(k)}\}_{k=1}^K \sim \text{Pr}_{\text{LLM}}(\cdot \mid n_i, G_{t-1}, S, \Sigma^*)$ 
6:    $\{v_t^{(k)}\}_{k=1}^K \sim \text{Pr}_{\text{LLM}}(\cdot \mid z_t^{(k)}, S, \Sigma^*)$ 
7:    $G_t \leftarrow \text{Update}(G_{t-1}, \{z_t^{(k)}, v_t^{(k)}\}_{k=1}^K)$ 
8: end while
9:  $z^* \leftarrow \text{SelectBest}(G_T)$ 
10:  $\mathcal{P}^* \sim \text{Pr}_{\text{LLM}}(\cdot \mid z^*, S, \Sigma^*)$   $\triangleright$  translate final thought into executable pipeline
11:  $D^* \leftarrow \text{Env}(S, \mathcal{P}^*)$ 
12: return  $D^*$ 
```

• **The Synth-Spider dataset.**

- Column buckets: b_1 (1-4), b_2 (5-7), b_3 (8-9), b_4 (10-14), b_5 (15-124,843).
- Row buckets: b_1 (1-9), b_2 (10-15), b_3 (16-21), b_4 (22-35), b_5 (36-124,842).

• **The Synth-Bird dataset.**

- Column buckets: b_1 (1-11), b_2 (12-27), b_3 (29-74), b_4 (75-1,003), b_5 (1,004-20,015).
- Row buckets: b_1 (1-110), b_2 (115-1,000), b_3 (1,001-1,022), b_4 (1,023-2,004), b_5 (2,005-360,006).

• **The Parrot dataset.**

- Column buckets: b_1 (2-4), b_2 (5-7), b_3 (8-11), b_4 (12-16), b_5 (18-587).
- Row buckets: b_1 (2-13), b_2 (14-30), b_3 (31-100), b_4 (107-2,000), b_5 (2,075-200,000).

Responses to Reviewer #3

R3W1: *The difference between this method and other tree-search LLM agents (such as Tree-of-Thoughts or MCTS-based methods) is not explained clearly enough. The paper should more clearly show what is new in the search strategy and how it is different in practice.*

RESPONSE: We thank the reviewer for this insightful question. To clarify the differences from existing tree-search LLM agents, we first summarize the search strategies of Tree-of-Thought (ToT) and MCTS-based methods and then present the search strategy and key novelty of DEEPPREP for autonomous data preparation (ADP).

Search Strategy of Tree-of-Thought (ToT) and its Limitation.

Tree-of-Thought (ToT, Algorithm 5) performs search over a tree of textual reasoning states. At each step, the LLM generates multiple candidate thoughts and evaluates them using an LLM-based value function (Lines 5–7). The search process operates entirely on textual representations, and the execution environment is invoked only once after a final solution is selected (Line 11).

The key limitation of ToT is the lack of execution grounding. Since intermediate reasoning steps are not validated against real data, errors, such as invalid operator parameters and non-existent columns, may remain undetected throughout the search process. Figure IX(a) shows a typical failure example. ToT decides to apply a Pivot operator using the column year, although this column has

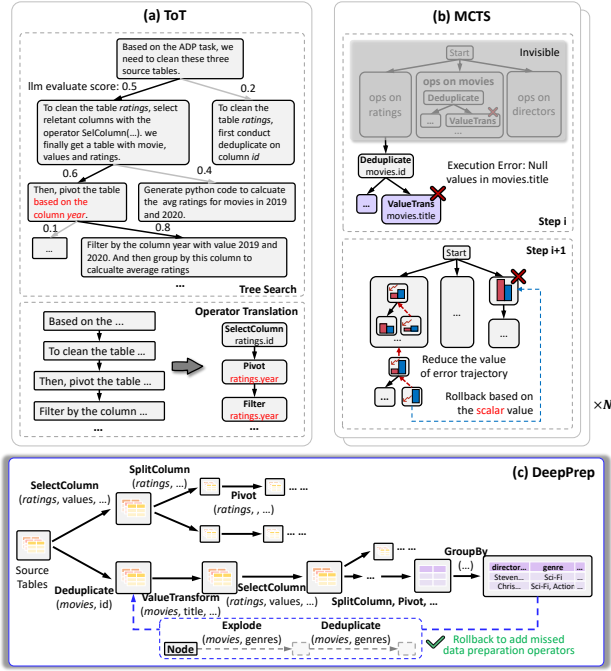


Figure IX: Comparison of ToT, MCTS and DEEPREP.

Algorithm 3 Direct MCTS Adaptation for Data Preparation

Require: Source tables $\mathcal{S}$ and target schema Σ^*

- 1: Initialize a search tree $G_0 = (N, E)$ with $N = \{n_0\}$, $n_0 \leftarrow \mathcal{S}$
- 2: Initialize node scores $Q(n_0) \leftarrow 0$
- 3: **while** the search budget is not exhausted **do**
- 4: $n_i \leftarrow \text{Select}(G_{t-1}, Q)$
- 5: $\{z_t^{(k)}\}_{k=1}^K \sim \text{Pr}_{\text{LLM}}(\cdot \mid n_i, G_{t-1}, \mathcal{S}, \Sigma^*)$
- 6: $n_j \leftarrow \text{Expand}(G_{t-1}, n_i, z_t^{(1)})$
- 7: $Q(n_j) \leftarrow 0$
- 8: $\{z_t^{(l)}\}_{l=1}^L \sim \text{Pr}_{\text{LLM}}(\cdot \mid n_j, \mathcal{S}, \Sigma^*)$
- 9: $\{\mathcal{T}^{(l)}\}_{l=1}^L \leftarrow \text{Env}(n_j, \{z_t^{(l)}\}_{l=1}^L)$
- 10: $r \leftarrow \text{Evaluate}(\{\mathcal{T}^{(l)}\}_{l=1}^L, \Sigma^*)$
- 11: **for** each ancestor n_a of n_j in G_t **do**
- 12: $Q(n_a) \leftarrow Q(n_a) + r$
- 13: **end for**
- 14: $G_t \leftarrow G_{t-1}$
- 15: **end while**
- 16: $z^* \leftarrow \arg \max_{n \in N} Q(n)$
- 17: $\mathcal{P}^* \sim \text{Pr}_{\text{LLM}}(\cdot \mid z^*, \mathcal{S}, \Sigma^*)$
- 18: $D^* \leftarrow \text{Env}(\mathcal{S}, \mathcal{P}^*)$
- 19: **return** D^*

not been generated yet. Because no execution feedback is available during search, the error is discovered only when the final pipeline is executed, causing the subsequent reasoning steps to fail.

Search Strategy of MCTS-based Methods and its Limitation. Monte Carlo Tree Search (MCTS, Algorithm 6) performs search through statistical value estimation. At each iteration, it selects a promising node according to accumulated node scores (Line 4), expands the search tree (Line 6), and estimates the quality of the expanded node through rollout simulations (Lines 8–9). The resulting

rewards are then propagated through the tree to guide future exploration (Lines 10–12). The final solution is selected based on the accumulated node scores (Line 15).

The key limitation of MCTS for ADP is that exploration is guided primarily by scalar rewards. While execution outcomes affect node scores, the search process retains only numerical estimates of branch quality rather than detailed failure information. As a result, the agent learns that a branch is undesirable, but not why it failed, making targeted correction difficult. Figure IX(b) illustrates a typical example. When an execution error occurs due to null values in movies.title during ValueTrans, the failure is propagated as a scalar penalty through the search tree. Consequently, MCTS discourages the branch but does not preserve the cause of the error, often leading to repeated invalid explorations.

Search Strategy of DEEPREP. DEEPREP (Algorithm 4) performs *execution-grounded search* over materialized data states. Each node in the search tree stores intermediate tables produced by executing data preparation operators (Line 1). At each step, the LLM reasons over the complete tree state, including intermediate tables, execution feedback, and recorded failure traces, to generate a logical plan and an operator pipeline (Lines 3–4). The generated operators are then physically executed on real data (Line 7), and the resulting intermediate tables are stored as new tree nodes (Lines 9–11). If execution fails, the failure trace is attached to the corresponding node and incorporated into future reasoning (Line 13).

Figure IX(c) illustrates a typical example. DEEPREP executes generated operators and observes the resulting intermediate tables. When execution reveals missing data preparation steps, the agent does not merely penalize the corresponding branch. Instead, it analyzes the failure trace and operator branches, rolls back to a valid state, and inserts the missing operators (e.g., Explode and Deduplicate on movies.genres) before continuing exploration. This enables precise error correction while avoiding redundant search.

Novelty of DEEPREP. The novelty of DEEPREP does not lie in the use of a tree structure itself, which is shared by ToT and MCTS. Instead, it arises from three key properties:

- *Execution-grounded search.* Unlike ToT, every tree expansion is validated through physical execution on real data, grounding search decisions in materialized intermediate states rather than textual reasoning alone.
- *Failure-aware exploration.* Unlike ToT and MCTS, execution failures are preserved as structured error traces and reused in subsequent exploration, enabling the agent to reason about why a branch failed rather than simply retrying alternative branches.
- *Generative tree reasoning.* Unlike MCTS, which guides exploration through rollout-based value estimation, DEEPREP reasons over the entire search tree, including intermediate tables, execution results, and failure traces, to generate subsequent actions.

We have revised the **Introduction** to clarify the key differences and novelty of DEEPREP, and included the detailed discussions in our technical report [1]

R3W3: The paper does not discuss scalability in enough detail. With many operator types and parameters, the search tree could grow large. A short discussion about complexity or pruning would improve the paper.

R3D1: *Limited algorithmic detail on tree search control. Although the system adopts a tree structure, the paper does not provide enough detail on how node selection, pruning, or branch limits are handled in practice. It is unclear how the search behaves when pipelines become longer or schemas become larger. A clearer discussion of search control strategy, complexity, or pruning mechanisms would improve the technical depth and scalability analysis.*

RESPONSE: We thank the reviewer for this valuable suggestion. Following this comment, we have **added detailed pseudo-code and discussions of tree-search control, pruning mechanisms, and complexity analysis** in both the paper and our technical report [1]. We summarize the key points below.

Node Selection and Search Control. Unlike classical tree-search algorithms that rely on score-based node selection and backpropagation, DEEPPREP adopts a generative search paradigm. As formalized in Algorithm 4, at each step the agent generates a `<plan>` (Line 3) based on the complete tree, including intermediate tables and failure traces. The plan determines whether to continue exploration, backtrack to a previous node, or terminate. The agent then generates an `<expand>` action (Line 4), which selects a parent node and produces the next operator sequence. Thus, node selection and branch expansion are controlled by the LLM’s reasoning over the current tree state rather than a predefined scoring function.

Pruning Mechanism. DEEPPREP does not employ an explicit pruning algorithm. Instead, pruning is performed implicitly through execution feedback. When an operator sequence fails during execution (Line 13), the corresponding failure trace is recorded and attached to the search tree. These failure traces are incorporated into subsequent prompts, enabling the agent to avoid revisiting previously failed branches. Therefore, search control is guided by implicit execution experience rather than explicit pruning rules.

Complexity Analysis. The search process is bounded by two configurable parameters: the maximum exploration turn T and the maximum operator sequence length L per expansion. Therefore, the worst-case search complexity is $O(T \times L)$. Unlike MCTS-style methods that explicitly expand and evaluate many candidate branches, DEEPPREP generates only one operator sequence at each exploration step. As a result, the effective branching factor remains bounded by the agent’s exploration turns rather than growing combinatorially with the number of operators or parameters.

Scalability Evaluation. To empirically validate scalability, we further evaluate DEEPPREP under varying row sizes, column sizes, and pipeline lengths. We additionally report execution time and memory consumption across different complexity groups. The results show stable growth in both runtime and memory usage, which indicates that DEEPPREP scales well with increasing data size and pipeline complexity. Please refer to the response to **R2W4 & R2D4 of Reviewer #2** for more details.

We have revised **Section 4.2** to provide search-control discussions and complexity analysis. We have also added the above scalability analysis to **Exp-7 (Scalability Analysis)** of **Section 6.4**.

R3W4: *Most experiments use synthesized datasets created by the authors. Although well designed, more evaluation on fully independent real-world data preparation tasks would make the results stronger.*

R3D2: *Heavy reliance on synthetic datasets. Most experiments are conducted on synthesized datasets created by the authors. While the synthesis process is carefully designed, this setup may favor the system’s assumptions and design choices. Additional evaluation on fully independent, real-world ADP tasks would make the empirical claims more convincing.*

RESPONSE: We thank the reviewer for the suggestion. We have **added evaluation on an independent real-world dataset and conducted a dedicated failure-case analysis**.

Specifically, we evaluate DEEPPREP on the Buildings dataset [36], which contains 105 real-world ADP tasks derived from data processing pipelines collected from 21 energy companies. Importantly, this dataset is entirely independent of our synthesis process and was not constructed from Text-to-SQL datasets. The tasks involve practical data preparation challenges, including ambiguous column semantics, domain-specific abbreviations, heterogeneous schemas, and underspecified transformation requirements. As reported in Table II, DEEPPREP achieves the highest accuracy among all baselines while maintaining low inference cost. These results demonstrate that the advantages of DEEPPREP are not limited to synthesized datasets.

Moreover, we conduct a failure-case analysis on Buildings. The analysis reveals two major sources of errors: ambiguous column semantics (50% of failures) and underspecified data preparation requirements (33% of failures). These findings not only provide additional evidence that Buildings captures realistic ADP challenges, but also highlight important directions for future work. Please refer to our responses to **R1W3** and **R1D5** for detailed analysis.

We have added the Buildings dataset and corresponding experimental results and included the failure-case analysis in **Exp-6** of **Section 6.4**.

R3W2: *The paper uses a probabilistic formula for environment transitions, but execution seems deterministic once an operator is fixed. The authors should clarify whether the environment is actually stochastic or not.*

R3D3: *Formalization of environment transitions. The paper models environment transitions as probabilistic, but execution appears deterministic once an operator sequence is fixed. This is not a serious issue, but it may cause confusion. Clarifying whether any real stochasticity exists in the environment would improve theoretical clarity.*

RESPONSE: We thank the reviewer for this suggestion. We agree that the environment in DEEPPREP is deterministic: given a state and an operator sequence with fixed parameters, the resulting state is uniquely determined by operator execution.

We have revised the manuscript to clarify this point and replaced the probabilistic transition function with a deterministic state-transition function in **Section 4.2**.

Table IV: Overall Ablation Study on Qwen3-8B. PAT represents Progressive Agentic Training and TAR represents Tree-based Agentic Reasoning.

Method	Synth-Spider		Synth-Bird		Parrot	
	Acc.	Comp.	Acc.	Comp.	Acc.	Comp.
DeepPrep	65.99	97.46%	53.39	92.78%	39.93	98.46%
w/o PAT	22.21	59.76%	8.87	41.90%	25.27	70.62%
w/o PAT, TAR	4.68	7.42%	0.65	4.33%	3.09	8.32%

R3D4: *Some results reflect expected trends. Performance improvements with larger backbone models are expected in LLM-based systems and are not unique to this approach. The paper could better isolate how much of the performance gain comes specifically from the tree-based reasoning framework and progressive training design.*

RESPONSE: As suggested, to isolate the contributions of the tree-based reasoning framework (TAR) and progressive agentic training (PAT), we have **added a comprehensive ablation study in Exp 4**. The results are reported in Table IV.

The results show that the performance gains cannot be explained solely by stronger backbone models. Removing PAT while retaining TAR causes substantial degradation, reducing accuracy by 43.78 on Synth-Spider, 44.52 on Synth-Bird, and 14.66 on Parrot. This indicates that the model must first learn the operator syntax, reasoning protocol, and multi-turn search behavior before tree-based reasoning can be effectively utilized.

Further removing TAR causes the method to degenerate into single-turn operator generation (i.e., PaS), leading to an even larger performance drop of 61.31 on Synth-Spider, 52.74 on Synth-Bird, and 36.84 on Parrot, while completeness falls to near-zero levels. These results demonstrate that TAR is critical for maintaining historical states, leveraging execution feedback, and recovering from erroneous decisions through backtracking.

We have revised the **Section 6.3** and added a new experiment **Exp-3** to demonstrate the contributions of TAR and PAT module in DEEPREP.

Responses to Meta-Reviewer

We sincerely thank the Meta-Reviewer for the constructive feedback and the opportunity to revise our work. We have carefully addressed the revision items by conducting new experiments on real-world datasets, clarifying the novel contributions, and performing rigorous scalability analyses.

MC1: *Identify a real scenario and run the system on real data, rigorously, to determine its strengths and limitations.*

RESPONSE: We have significantly expanded our evaluation to include real-world scenarios. Specifically, we have evaluated DEEPREP on the Buildings dataset [36], a real-world benchmark comprising 105 tasks derived from data processing pipelines collected from 21 energy companies. DEEPREP achieves the highest accuracy among all methods while maintaining relatively low inference cost, confirming that the advantages observed on synthetic benchmarks transfer to real-world scenarios. We have also added a dedicated failure analysis on the Buildings benchmark, identifying two key challenges largely absent in synthesized benchmarks: ambiguous column semantics (50% of failures) and underspecified data

preparation requirements (33% of failures). Please refer to the detailed responses in **R1W3**, **R1D5**, **R2W2/R2D2**, and **R3W4/R3D2**.

MC2: *Identify the generalizable contributions which make the approach followed by the work novel.*

RESPONSE: We have revised the paper to explicitly clarify the technical novelty of DEEPREP, distinguishing it from general purpose agentic frameworks. We highlight our contributions in three specific dimensions: (1) The agentic reasoning tree with materialized execution states and failure-aware branching, which differs from ToT/MCTS that operate over text/latent space with scalar reward signals. (2) Progressive agentic training with environment token masking and hybrid reward design. and (3) Reversible noise injection with verification-in-the-loop for data synthesis. Please see the responses to **R2W1/R2D1** and **R3W1**.

MC3: *Address individual, actionable, criticisms raised in the reviews.*

RESPONSE: We have addressed all specific technical concerns raised by the reviewers. Most notably:

- (1) Real-World Evaluation & Failure Analysis. We have evaluated on the Buildings dataset and conducted a dedicated error analysis. Please refer to **R1W3**, **R1D5**, **R2W2/R2D2**, **R3W4/R3D2**.
- (2) Scalability & Sensitivity. We have conducted new experiments analyzing how DEEPREP behaves (accuracy, runtime and memory usage) with increasing table sizes (rows/columns), varying exploration turns, source table numbers, and pipeline lengths. Please refer to **R1D3**, **R2W4/R2D4**, **R3W3/R3D1**.
- (3) Implementation Details & Fairness. We have provided comprehensive details regarding prompt construction, state representation, and baseline configurations to ensure reproducibility. Please refer to **R1W2/R1D1**, **R2W3/R2D3**.
- (4) Baselines. We have clarified the comparison with Code Agents and added Claude Code (CC) and TRAE as new agentic coding baselines. Please refer to **R1W1**.
- (5) Ablation Study. We have added an overall ablation study isolating the contributions of the tree-based reasoning framework and progressive agentic training. Please refer to **R3D4**.
- (6) Operator Diversity Analysis. We have conducted a finer-grained analysis of operator coverage and compositional diversity between Synth-Spider and Parrot, along with a cross-training experiment. Please refer to **R1D4**.
- (7) Deterministic Environment. We have clarified that the environment is deterministic and replaced the probabilistic notation with a deterministic transition function. Please refer to **R3W2/R3D3**.
- (8) Failure Branch Reuse. We have revised the manuscript to explain how failure branches are preserved with error traces and reused to inform future exploration. Please refer to **R1D2**.

DEEPPREP: An LLM-Powered Agentic System for Autonomous Data Preparation

Meihao Fan[†], Ju Fan[†], Yuxin Zhang[†], Shaolei Zhang[†], Xiaoyong Du[†],
Jie Song^{*}, Peng Li^{*}, Fuxin Jiang^{*}, Tieying Zhang^{*}, Jianjun Chen^{*}

[†] Renmin University of China; ^{*} ByteDance;

{fmh1art,fanj,yuxin.zhang,zhangshaolei98,duyong}@ruc.edu.cn

{jie.song,peng.li01,jiangfuxin,tieying.zhang,jianjun.chen}@bytedance.com

ABSTRACT

Data preparation, which aims to transform heterogeneous and noisy raw tables into analysis-ready data, remains a major bottleneck in data science. Recent approaches leverage large language models (LLMs) to automate data preparation from natural language specifications. However, existing LLM-powered methods either make decisions without grounding in intermediate execution results, or rely on linear interaction processes that offer limited support for revising earlier decisions. To address these limitations, we propose DEEPPREP, an LLM-powered agentic system for autonomous data preparation. DEEPPREP constructs data preparation pipelines through iterative, execution-grounded interaction with an environment that materializes intermediate table states and returns runtime feedback. To overcome the limitations of linear interaction, DEEPPREP organizes pipeline construction with tree-based agentic reasoning, enabling structured exploration and non-local revision based on execution feedback. To enable effective learning of such behaviors, we propose a progressive agentic training framework, together with data synthesis that supplies diverse and complex ADP tasks. Extensive experiments show that DEEPPREP achieves data preparation accuracy comparable to strong closed-source models (e.g., GPT-5) while incurring $15\times$ lower inference cost, while establishing state-of-the-art performance among open-source baselines and generalizing effectively across diverse datasets.

PVLDB Reference Format:

Authors. DEEPPREP: An LLM-Powered Agentic System for Autonomous Data Preparation. PVLDB, 19(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://anonymous.4open.science/r/DeepPrep-0432>.

1 INTRODUCTION

Data preparation remains a major bottleneck in data science, often consuming 60%–80% of end-to-end analysis time [6]. This is due to the heterogeneity, noise, and misalignment across sources, as well as the iterative and interdependent nature of data preparation

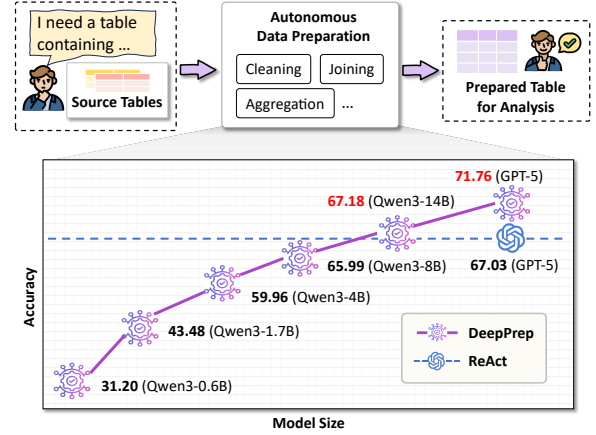


Figure 1: Overview of Autonomous Data Preparation (Top) and Performance of DEEPPREP (Bottom).

operations such as cleaning, joining, and aggregation. To address these challenges, autonomous data preparation (ADP) aims to automatically transform multiple source tables into analysis-ready data based on a high-level target specification, by generating an executable transformation pipeline. For example, as shown in Figure 1, given several source tables and a target description such as “I need a table containing ...”, the system automatically generates and executes the required transformation steps.

State-of-the-Art Approaches. Existing approaches to autonomous data preparation broadly fall into two categories.

- Traditional methods rely on rigid specifications such as input-output examples, transformation templates, or rule-based program synthesis [3, 16, 17, 21, 37]. While effective in constrained settings, these approaches are not designed to flexibly accommodate data preparation intent expressed in natural language.
- Recent approaches leverage large language models (LLMs) to automate data preparation from natural language specifications [7, 30, 32]. Prompting-based methods generate data preparation code directly from task descriptions and examples [25, 29, 33], but make decisions without systematic grounding in intermediate execution results, often leading to error-prone pipelines. ReAct-style approaches incorporate execution feedback by interleaving reasoning with task-specific actions [2, 43]. However, this interaction follows a linear trajectory without explicit support for revising earlier decisions, which may be inadequate for autonomous data preparation, where early mistakes often become evident only after several dependent operations.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 19, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

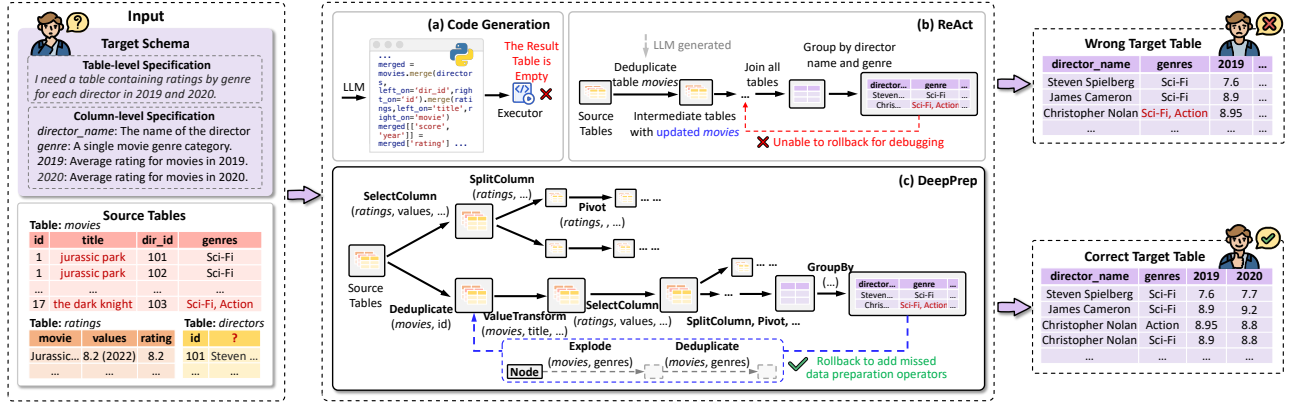


Figure 2: An illustrative example highlighting the core idea of DEEPPREP. (a) Code generation makes decisions without grounding in intermediate execution results. (b) ReAct-style approaches follow linear reasoning with limited ability to revise earlier decisions. (c) DEEPPREP applies a tree-based structured exploration, which enables backtracking to resolve non-local errors.

Our Proposal. To address these limitations, we propose **DEEPPREP**, an LLM-powered agentic system for autonomous data preparation. DEEPPREP constructs data preparation pipelines through iterative, execution-grounded interaction with an environment that materializes intermediate table states and returns runtime feedback. Unlike one-shot code generation and linear ReAct-style interaction, DEEPPREP organizes pipeline construction with *tree-based agentic reasoning*. This design enables structured exploration and *non-local* revision when execution feedback indicates that earlier decisions are suboptimal. Specifically, given source tables and a target schema, DEEPPREP operates in an iterative loop as below:

- **Planning:** observing the current execution state and synthesizing a high-level plan for transforming existing tables.
- **Orchestration:** instantiating the plan as an executable operator pipeline with specified parameters.
- **Execution:** executing the pipeline to materialize new table states and returning runtime feedback for subsequent decisions.

Through iterative interaction with execution feedback, DEEPPREP incrementally refines its decisions and converges to a data preparation pipeline that produces a table conforming to the target schema.

EXAMPLE 1. Consider the example in Figure 2. The goal is to construct a table for analyzing director performance across genres using audience ratings. A linear agent following a ReAct-style interaction may prematurely join and aggregate the data, resulting in multi-valued entries such as “Sci-Fi, Action” in the genre column, which violates the target schema constraint requiring single-valued categories. Correcting this error requires revisiting an earlier cleaning decision on the movies table, i.e., inserting an Explode operation before reapplying downstream joins and aggregations.

DEEPPREP supports this naturally. It can backtrack to the execution state where the incorrect decision was made, expand an alternative branch with the corrected operation, and reuse valid operator prefixes from existing partial pipelines to produce a correct result.

Challenges and Solutions. We study the research challenges posed by the above agentic framework and outline our solutions.

The first challenge is how to support structured exploration and non-local backtracking under execution feedback. To address this,

DEEPPREP maintains an explicit tree of materialized execution states during pipeline construction, where nodes represent intermediate tables produced by executed operators and edges record the applied operator pipelines. This representation enables execution feedback to be precisely attributed to earlier decision points, allowing the agent to backtrack and revise upstream choices when downstream failures occur. Building on this structure, DEEPPREP introduces action-structured LLM interactions that operate directly over the tree, enabling the agent to incrementally expand, revise, or backtrack branches based on execution feedback.

The second challenge is how to effectively train LLMs for the tree-based agentic reasoning. A straightforward solution is to apply reinforcement learning with outcome-level rewards [14, 23]. However, this setting introduces two difficulties. First, *reward sparsity*: long and interdependent pipelines make outcome insufficient for learning structured exploration and non-local backtracking. Second, *data scarcity*: existing ADP datasets focus on simple, short pipelines [22, 28], limiting supervision for multi-step, execution-aware reasoning. To address these challenges, we propose a Progressive Agentic Training framework that (i) initializes the model with basic operator usage and tree-based interaction patterns, (ii) refines the policy through multi-turn reinforcement learning with a hybrid reward that considers both outcome-level and process-level signals, and (iii) leverages data synthesis to construct diverse and complex ADP tasks for training.

Differences from existing agentic solutions. DEEPPREP differs from existing systems in two key aspects. First, DEEPPREP performs *tree-based agentic reasoning* over explicit, materialized execution states. Unlike ReAct [43], which follows a linear trajectory, DEEPPREP can roll back to previously valid states and revise upstream decisions. Unlike Tree-of-Thought [42] and MCTS methods [24], its search is grounded in executable data preparation operators and runtime feedback, and the resulting reasoning tree can be serialized into prompts for the subsequent reasoning. Second, DEEPPREP introduces progressive agentic training tailored to tree-based agentic reasoning in ADP. Specifically, our training method enables the model to gradually transition from learning the basic operator syntax to complex tree search, which cannot be addressed by existing agentic systems [20, 47]. Moreover, we are the first to propose a

verification-in-the-loop method for constructing diverse and complex ADP tasks for training.

Contributions. We summarize our contributions as follows.

- (1) We study the problem of autonomous data preparation with support for a wide range of data preparation operators (Section 2), and introduce an LLM-powered agentic framework (Section 3).
- (2) We propose two techniques to tackle the challenges of this problem: tree-based agentic reasoning for inference (Section 4) and progressive agentic training for agent optimization (Section 5).
- (3) We conduct extensive experiments on real-world ADP benchmarks. Results show that DEEPPREP achieves accuracy comparable to closed-source models (e.g., GPT-5) at 15× lower inference cost, while establishing state-of-the-art performance among open-source baselines and generalizing well across diverse datasets.
- (4) We release the code, data, and a comprehensive set of trained model weights (from 0.5B to 14B parameters), enabling users to deploy ADP agents under different computational budgets¹.

2 PRELIMINARIES

2.1 Problem Formulation

Source and Target Data. We consider a set of *source tables* $\mathcal{S} = \{S_1, \dots, S_n\}$. Each source table is denoted as $S_i = (\Sigma_i, D_i)$, where Σ_i denotes the schema of S_i and D_i denotes the set of tuples conforming to Σ_i . The schema Σ_i captures both table-level and column-level specifications and is defined as $\Sigma_i = (\tau_i, C_i)$. Here, τ_i represents a textual description that summarizes the semantic meaning of the table, and $C_i = \{c_i^1, \dots, c_i^{m_i}\}$ denotes the set of column specifications. Each column specification c_i^j includes column-level meta-data such as column name, data type, and semantic description. In practice, the schema Σ_i of a source table may be partially specified or entirely unavailable. In such cases, table- and column-level specifications can be inferred from the raw data in D_i . For example, each source table S_i in Figure 2 consists of a schema Σ_i which may be specified by table- and column-level information, and a set D_i of data tuples, e.g., (1, “jurassic park”, 101, “Sci-Fi”).

Similarly, the *target table* is denoted as $T = (\Sigma^*, D^*)$, where $\Sigma^* = (\tau^*, C^*)$ specifies the target schema and D^* denotes the target tuples to be produced. Note that, in autonomous data preparation, only the target schema Σ^* is given as input, while the target tuples D^* are unknown and must be generated by transforming and cleaning the source data. For example, in Figure 2, the target schema Σ^* is specified by a table-level description (e.g., “I need a table containing ...”) and a set of column specifications, each including a column name (e.g., “director_name”) and a corresponding semantic description (e.g., “The name of the director.”).

Data Preparation Pipeline. We model the process of data preparation as a composition of *operators* applied to source tables. Specifically, an operator represents a primitive data transformation. Formally, an operator o is characterized by its type κ and parameters θ , and defines a function $o_\theta : \mathcal{T} \rightarrow \mathcal{T}$, where $\mathcal{T}$ denotes a set of tables, and each table is of the form (Σ, D) as defined previously. Given an input table set $\mathcal{T}_{\text{in}}$, the operator produces an output table set, i.e., $\mathcal{T}_{\text{out}} = o_\theta(\mathcal{T}_{\text{in}})$. Note that different operator types capture different classes of data transformations, such as filtering, joining,

aggregation, and cleaning. The current operator types supported by DEEPPREP will be introduced in Section 2.2.

We then define a data preparation pipeline (or pipeline for short) as an ordered sequence of operators applied iteratively to transform a set of source tables $\mathcal{S}$ into a target table $\hat{T}$. We use the symbol “ $\rightarrow$ ” to denote the left-to-right application order between adjacent operators, i.e., the output produced by an operator is fed as the input to the operator to its right. Formally, a pipeline $\mathcal{P}$ is defined as an ordered composition of k operators over $\mathcal{S}$, i.e.,

$$\mathcal{P} = (\mathcal{S}, o_{\theta_1}^{(1)} \rightarrow o_{\theta_2}^{(2)} \rightarrow \dots \rightarrow o_{\theta_k}^{(k)}). \quad (1)$$

Given a set of source tables $\mathcal{S}$, the execution of pipeline $\mathcal{P}$ produces a sequence of intermediate table sets, i.e., $\mathcal{T}_0 = \mathcal{S}$, $\mathcal{T}_i = o_{\theta_i}^{(i)}(\mathcal{T}_{i-1})$, $i = 1, \dots, k$, where $\mathcal{T}_k$ denotes the final output of the pipeline, which contains only the target table $\hat{T}$, i.e., $\mathcal{T}_k = \{\hat{T}\}$. Consider operator Deduplicate (movies, [id], first) as an example. Its operator type κ is Deduplicate and its parameters θ refer to (movies, [id], first). Executing this operator on source tables produces intermediate tables consisting of a cleaned *movie*, while tables *ratings* and *directors* remain unchanged.

Problem Statement. Given a set of source tables $\mathcal{S} = \{S_1, \dots, S_n\}$, a target schema Σ^* , and a set of available operator types $\mathcal{O}$, the autonomous data preparation (ADP) problem is to find an optimal data preparation pipeline $\mathcal{P}$ that transforms $\mathcal{S}$ into a target table $\hat{T} = (\Sigma^*, \hat{D})$, where $\hat{D}$ denotes the tuples produced by $\mathcal{P}$ that satisfy the target schema Σ^* . We assume a ground-truth target table $T^* = (\Sigma^*, D^*)$, which is used *only* for evaluation. Thus, the optimality of a pipeline is defined by the quality of $\hat{D}$ with respect to D^* under task-specific evaluation metrics, which will be introduced in the experimental setup (Section 6.1).

EXAMPLE 2. Figure 2 provides an example of an ADP task with three source tables: *movies*, *ratings*, and *directors*. The goal is to generate a target table that conforms to a given target schema. The target schema describes both the overall semantics of the table and its desired structure. Specifically, it provides a table-level summary of the intended content and specifies column-level requirements, including column names (e.g., *genre*), semantic meanings (e.g., *movie genre category*), and data constraints (e.g., *single-valued values*). To achieve this goal, we construct a data preparation pipeline composed of multiple operators, such as Deduplicate and Join. Each operator is applied to the current set of tables and produces updated intermediate tables. For example, applying Join merges the *movies* and *directors* tables on movie titles, yielding a table named *movies_directors_join* that integrates movie data with director names, while keeping the *ratings* table unchanged. After executing all operators in the pipeline, the process produces the target table, shown on the right of Figure 2.

2.2 Supported Data Preparation Operators

DEEPPREP supports a broad set of data preparation operators and is designed to be extensible, allowing new operator types to be incorporated as needed. In total, we consider 31 operators, organized into eight categories that cover the major transformation patterns in data preparation pipelines. Due to space constraints, we provide descriptions of **typical operators** in each category, with complete specifications available in [1].

¹Accessible via: <https://anonymous.4open.science/r/DeepPrep-0432>

2.2.1 Data Cleaning. Data cleaning operators improve data quality by detecting, fixing, or removing erroneous, missing, or duplicate records, and are commonly applied in data preparation pipelines.

- `MissingValueImputation(table, column, mode)` imputes missing values in a specified column using a statistical strategy, with mode selecting the method (e.g., mean, median, or mode).
- `Deduplicate(table, subset, keep)` removes duplicate rows based on specified columns, retaining either the first or last occurrence as determined by keep.

2.2.2 Value Normalization. Value normalization operators standardize field values and data types to ensure semantic consistency.

- `ValueTransform(table, column, func)` applies a user-defined transformation function to values in a specified column to standardize their format.
- `StandardizeDatetime(table, column, format)` converts values in a specified column to a user-defined datetime format.

2.2.3 Schema Editing. Schema editing operators modify table structure and column semantics, enabling column selection, renaming, derivation, and restructuring to align raw data with the target schema.

- `RenameColumn(table, rename_map)` renames columns according to a specified mapping from original to target names.
- `AddNewColumn(table, name, func)` adds a new column computed from existing data using a row-wise function.

2.2.4 Row Selection. Row selection operators manipulate tables by filtering, ordering, or selecting subsets of rows.

- `Filter(table, func)` selects rows satisfying a given predicate.
- `Sort(table, by, ascending)` orders rows in a table based on specified columns, with ascending controlling the direction.
- `TopK(table, k)` retains the top- k rows of the table.

2.2.5 Aggregation. Aggregation operators summarize data by grouping records and computing statistics, transforming row-level data into compact, analysis-ready representations.

- `GroupBy(table, by, agg)` groups rows by specified columns and applies aggregation functions defined by agg.
- `Count(table)` returns the total number of rows in the table. We treat Count as a primitive aggregation operator for convenience, though it can be viewed as a special case of GroupBy without grouping keys.

2.2.6 Table Combination. Table combination operators integrate data from multiple tables into a unified representation, enabling multi-source data preparation pipelines.

- `Join(left, right, on, how)` combines two tables based on specified key columns using a join type defined by how.
- `Union(tables, how)` vertically combines multiple tables with compatible schemas, with how determining whether duplicate rows are retained or removed.

2.2.7 Table Reshaping. Table reshaping operators reorganize the structural layout of tables, such as converting between wide and long formats or expanding nested fields.

- `Pivot(table, index, columns, values, aggfunc)` reshapes a table from long to wide format by aggregating values at the intersection of specified index and column identifiers.

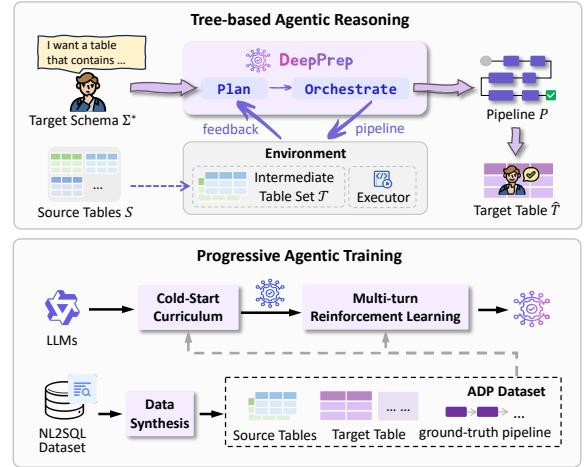


Figure 3: An overview of DEEPPREP.

- `Stack(table, id_vars, value_vars)` reshapes a table to long format by collapsing multiple columns into key-value pairs.

2.2.8 Program Synthesis. The program synthesis operator provides a general mechanism for handling transformations that cannot be expressed using the above predefined operators.

- `ExeCode(tables, target, func)` generates a target table from input tables by executing Python code synthesized by an LLM, enabling transformations that cannot be expressed using the above predefined operators.

3 AN OVERVIEW OF DEEPPREP

DEEPPREP employs an agentic framework to autonomously orchestrate data preparation pipelines, as illustrated in Figure 3. Given a set of source tables S and a target schema Σ^* , DEEPPREP iteratively interacts with an *environment* that executes data transformations and provides feedback. The environment maintains an execution state represented by a set of intermediate tables, initialized with the source tables S . Operator instances are executed within the environment through concrete execution backends (e.g., a Python interpreter), which apply transformations to the current state and produce updated tables along with execution feedback. Given the target schema Σ^* , DEEPPREP interacts with the environment through an iterative process that alternates among the *planning*, *orchestration*, and *execution* steps, as described below.

- In the *planning* step, the agent observes the current execution state and synthesizes a high-level textual plan that outlines how to transform the existing tables toward the target schema.
- In the *orchestration* step, DEEPPREP translates this plan into an executable pipeline by selecting appropriate operator types and instantiating them with specific parameters.
- In the *execution* step, the orchestrated pipeline is submitted to the environment, which executes the operators, updates its internal state with newly generated intermediate tables, and returns execution feedback, such as sample outputs or error traces.

Through iterative interaction with execution feedback, DEEPPREP progressively refines planning and orchestration decisions and converges to a pipeline that produces a desired target table.

EXAMPLE 3. Consider the ADP task shown in Figure 2. The environment is initialized with the source tables. At each iteration, DEEPREP observes the current execution state, including serialized table contents and available schema information.

- To align the data with the target schema, the agent first generates a high-level textual plan, such as “Clean table movies and then integrate it with table ratings”.
- Based on this plan, the orchestration step instantiates a pipeline, i.e., a sequence of operators, e.g., Deduplicate(...) followed by ValueTransform(...). The orchestrated operators are then submitted to the environment for execution.
- Upon execution, the environment applies the operators to the current state and returns execution feedback. Specifically, if execution is successful, the environment updates its internal state to include the cleaned movies table while retaining the original ratings and directors tables. Otherwise, execution errors are captured and propagated to the agent, which uses this feedback to revise subsequent planning and orchestration decisions.

The iterative process continues until the generated tables satisfy the target schema and quality requirements, at which point DEEPREP terminates and outputs the final data preparation pipeline.

To support the framework, we design two key components: *Tree-Based Agentic Reasoning* and *Progressive Agentic Training*.

Tree-based Agentic Reasoning. The key challenge in agentic pipeline orchestration is supporting *structured* exploration and backtracking under execution feedback. Specifically, pipeline construction involves interdependent operator decisions, where early choices may later prove suboptimal or infeasible. Linear agentic reasoning mechanisms such as ReAct [2] and CoT [39] follow a single decision trajectory and do not preserve alternative paths. Thus, once a candidate pipeline is executed, early errors can propagate to subsequent steps, and without a structured representation that retains historical states, recovery becomes difficult.

To address this challenge, we introduce a *tree-based agentic reasoning* mechanism. Instead of committing to a single linear execution trajectory, DEEPREP represents pipeline construction as a search tree, where each node corresponds to a planning state associated with a set of intermediate tables, and each edge represents applying a candidate operator pipeline. This tree structure preserves historical states and alternative construction paths, enabling systematic exploration and backtracking based on execution feedback. Building on this tree representation, we further define LLM interaction patterns and decision logic over the tree, allowing the agent to reason about which branch to expand, revise, or abandon during pipeline construction. We detail this tree-based reasoning and interaction mechanism in Section 4.

Progressive Agentic Training. The key challenge is how to effectively train LLMs for tree-based agentic reasoning. A straightforward approach is to directly apply existing reinforcement learning methods to optimize the model using outcome-level rewards. However, this setting poses two major difficulties. First, *reward sparsity*: data preparation pipelines often consist of long and interdependent operator sequences, and relying solely on final correctness makes it difficult for the model to learn structured exploration and non-local backtracking behaviors. Second, *data scarcity*: existing

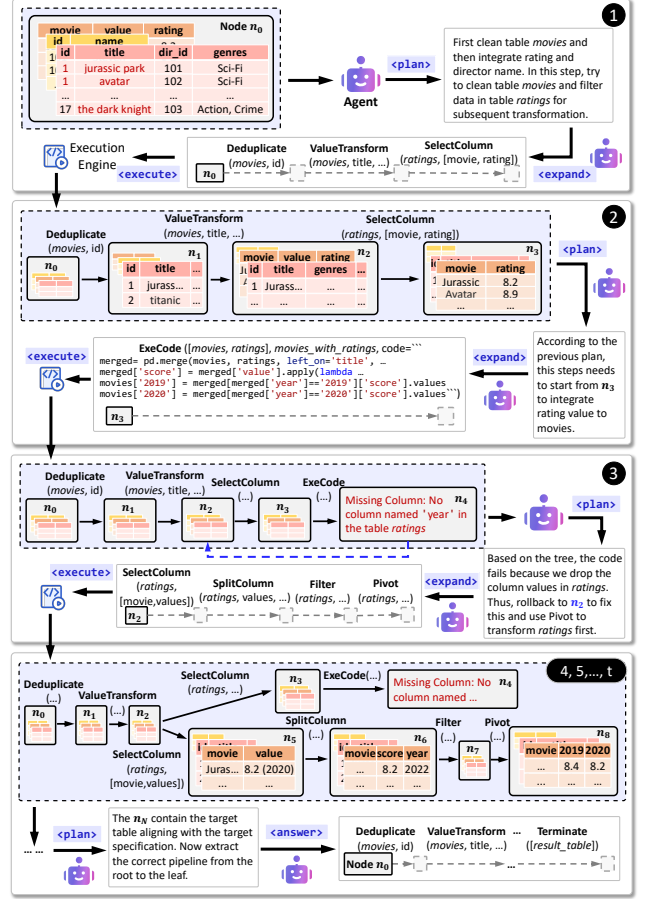


Figure 4: An example of tree-based agentic reasoning.

ADP datasets mainly involve simple transformations with short operator sequences [22, 28], providing insufficient supervision for learning multi-step reasoning that depends on execution feedback.

To address reward sparsity, we introduce a Cold-Start Curriculum that initializes the model with basic knowledge of data preparation operators and the tree-based agentic reasoning procedure. Building on this initialization, we further apply multi-turn reinforcement learning with a Hybrid Reward mechanism that combines outcome-level signals with process-level feedback. To alleviate data scarcity, we develop a data synthesis approach that generates complex ADP tasks grounded in meaningful analytical transformations, while injecting diverse data quality issues without breaking the intended transformation logic. We detail our agentic training strategy in Section 5.

4 TREE-BASED AGENTIC REASONING

To construct data preparation pipelines under complex dependencies and execution feedback, DEEPREP employs a *tree-based agentic reasoning* mechanism that supports structured exploration and non-local backtracking. Specifically, we introduce an *agentic reasoning tree* (Section 4.1) that explicitly preserves historical states and alternative data preparation pipelines. Building on this tree structure, we define a tree-based agentic inference procedure (Section 4.2) through a set of explicit agentic interaction actions. These

actions specify how the LLM observes the current tree state, proposes data preparation plans, and generates promising pipelines.

4.1 Agentic Reasoning Tree

We define the *agentic reasoning tree* as a directed tree $G = (N, E)$, which represents the structured search space of pipeline construction starting from the initial source tables $\mathcal{S}$.

- **Node (N).** Each node $n \in N$ corresponds to an *environment state*, represented by the current set of intermediate tables maintained by the environment. In particular, the root node n_0 represents the initial state initialized with the source tables $\mathcal{S}$.
- **Edge (E).** A directed edge $e = (n_i \xrightarrow{o} n_j) \in E$ represents a state transition induced by executing an *operator instance* o (i.e., an operator type with specific parameters) orchestrated by the agent at state n_i . Specifically, executing o updates the environment state from n_i to n_j , yielding updated intermediate tables.

In the agentic reasoning tree, each edge corresponds to a single operator instance, while a path from a node to another node represents an operator sequence, i.e., a data preparation pipeline.

4.2 Tree-Based Agentic Inference

The basic idea of our tree-based agentic inference is to develop an iterative decision-making process through a structured LLM interaction pattern. Specifically, the LLM autonomously generates agentic *interaction actions* to operate over a materialized agentic reasoning tree, incrementally expanding, revising, or backtracking tree branches based on execution feedback.

Agentic Inference Framework. Given source tables $\mathcal{S}$ and a target schema Σ^* , the framework produces a reasoning trajectory R and finally generates an answer A based on R . More formally, we model this process by a probabilistic model $\Pr(\cdot)$ over the reasoning process and the answer generation conditioned on the inputs:

$$\Pr(A, R \mid \mathcal{S}, \Sigma^*) = \underbrace{\prod_{t=1}^T \Pr(R_t \mid R_{<t}, \mathcal{S}, \Sigma^*)}_{\text{Reasoning Process}} \cdot \underbrace{\Pr(A \mid R, \mathcal{S}, \Sigma^*)}_{\text{Answer Generation}}. \quad (2)$$

where T denotes the length of the reasoning process and R_t represents the reasoning results of the agent at step t .

The reasoning process is implemented by adopting an LLM-based agent to search over the agentic reasoning tree G in a generative way. Specifically, the process alternates in an iteration of *plan*, *expand* and *execute* steps where it generates logical plans for the current step based on the observation of the current tree, expands new branches and execute expanded operators to update the current tree. Based on this idea, we break the step-wise reasoning process in Eq. (2) down into the following formula.

$$\Pr(R_t \mid R_{<t}, \mathcal{S}, \Sigma^*) = \underbrace{\Pr_{\text{LLM}}(\psi_t \mid \psi_{<t}, G_{t-1}, \Sigma^*)}_{\langle \text{plan} \rangle} \cdot \underbrace{\Pr_{\text{LLM}}(\mathcal{P}_t \mid G_{t-1}, \psi_t)}_{\langle \text{expand} \rangle} \cdot \underbrace{\text{Env}(G_t \mid G_{t-1}, \mathcal{P}_t)}_{\langle \text{execute} \rangle}. \quad (3)$$

where $\Pr_{\text{LLM}}$ denotes the generative distribution induced by the LLM agent, ψ_t and $\mathcal{P}_t$ denotes the logical plan and expanded operator pipeline generated by the agent in step t , and Env represents

the environment transition distribution that materializes $\mathcal{P}_t$ to get an updated agentic reasoning tree G_t .

Actions for Agentic Inference. We regulate this interaction using control tags aligned with the components in Eq. (2) and Eq. (3).

- **$\langle \text{plan} \rangle$.** This action produces a high-level logical plan for the next reasoning step. Given the current agentic reasoning tree, the target schema Σ^* , and prior plans, the agent generates a plan ψ that specifies the intended reasoning decision. The plan analyzes the current environment state to determine whether to continue expanding the tree or terminate the reasoning process. If further exploration is required, it identifies the parent node to extend and the candidate operations to apply.
- **$\langle \text{expand} \rangle$.** This action generates a candidate operator pipeline expanded from a selected parent node n_{parent} . Conditioned on the plan ψ , the agent produces an operator sequence $\mathcal{P}$ to extend node n_{parent} . To ensure consistent node selection, the parent node is specified using a prefix-matching constraint: the agent must reference the full operator sequence from the root to that node. This constraint avoids ambiguous or index-based node references and enforces alignment between the proposed expansion and the recorded transformation history.
- **$\langle \text{execute} \rangle$.** This action applies the candidate operator pipeline through the environment and performs state transitions. Specifically, given a candidate pipeline $\mathcal{P}_t$, the environment executes its operators sequentially over the materialized tables at the selected parent node. After each operator is successfully applied, a new intermediate state node is created to store the resulting tables, and a directed edge is added from the previous state node to the new one. Repeating this process for all operators in $\mathcal{P}_t$ yields a chain of new nodes, which updates G_{t-1} to G_t . If execution fails due to a runtime exception (e.g., column mismatch or type error), no new state node is created; instead, the parent node is annotated with a failure state that records the error trace.
- **$\langle \text{answer} \rangle$.** This action terminates the reasoning process and returns the final pipeline. It is triggered when a leaf node state satisfies the target schema Σ^* . The root-to-leaf operator path associated with that node is extracted as the final pipeline $\mathcal{P}^*$.

Agentic Inference Procedure. We extend the LLM vocabulary with the action tags defined above, so that our tree-based reasoning can be expressed as a structured language modeling process.

During inference, the model iteratively generates tagged outputs corresponding to the defined interaction actions. Specifically, in each iteration, the model first generates a plan in the form of $\langle \text{plan} \rangle \dots \langle \text{plan} \rangle$, which specifies the next reasoning decision. Based on this plan, the model predicts an operator expansion wrapped by $\langle \text{expand} \rangle \dots \langle \text{expand} \rangle$. The produced pipeline is then executed by the environment, and the resulting state and execution feedback are recorded through $\langle \text{execute} \rangle \dots \langle \text{execute} \rangle$. The above process iterates until a state satisfies the target schema. When this condition holds, the final pipeline is returned using $\langle \text{answer} \rangle \dots \langle \text{answer} \rangle$, which terminates the reasoning process.

Complexity Analysis. The search process is bounded by two configurable parameters: the maximum exploration turn T and the maximum operator sequence length L per expansion. Therefore, the worst-case search complexity is $O(T \times L)$.

EXAMPLE 4. Figure 4 illustrates an example reasoning trajectory for the task in Figure 2. ① **Initialization:** Reasoning starts from node n_0 , which corresponds to an initial state associated with source tables $\mathcal{S}$. During inference, the model generates a $\langle \text{plan} \rangle$ action, specifying a logical plan that cleans the movies table and select relevant columns in the ratings table. An $\langle \text{expand} \rangle$ action then produces operators Deduplicate, ValueTransform and SelectColumn, which are applied through $\langle \text{execute} \rangle$ to produce new states, i.e., n_1 , n_2 and n_3 . ② **Execution Failure:** Subsequent $\langle \text{plan} \rangle$ and $\langle \text{expand} \rangle$ actions propose integrating rating data using an ExeCode operator. However, during $\langle \text{execute} \rangle$, a missing-column runtime error is raised, and the transition is recorded as a failure at n_4 . ③ **Backtracking and Alternative Expansion:** The model preserves the failure at n_4 and append the error trace to subsequent prompts. Based on the recorded failure feedback, a new $\langle \text{plan} \rangle$ analyzes the error and tries to debug it by keeping the column values of ratings table when applying the operator SelectColumn. Specifically, it selects n_2 as the parent state for further exploration. In addition to applying the updated SelectColumn, the $\langle \text{expand} \rangle$ action also proposes an alternative operator sequence (SplitColumn, Filter, Pivot). By referencing the failure at n_4 , the model avoids repeating the same mistake when generating this new branch. These operators are applied to generate new states (n_5, n_6, n_7, n_8). ④ **Termination:** When a leaf state satisfies the target schema Σ^* , an $\langle \text{answer} \rangle$ action is issued to extract the corresponding root-to-leaf operator path as the final pipeline.

5 AGENTIC TRAINING FRAMEWORK

We propose a *Progressive Agentic Training* framework for optimization. To mitigate reward sparsity, the framework includes a *Cold-Start Curriculum* for initialization (Section 5.1), followed by a *Multi-turn RL* stage with a *Hybrid Reward* mechanism (Section 5.2). To alleviate data scarcity, we develop a *Data Synthesis* method (Section 5.3) that generates diverse ADP tasks for effective training.

5.1 Cold-Start Curriculum

To mitigate the issue where an untrained LLM fails to produce executable pipelines, we design a Cold-Start Curriculum that initializes the LLM’s reasoning capability for our structured and execution-grounded planning. Specifically, pre-trained LLMs generally lack an explicit understanding of operator syntax and the tree-based agentic reasoning procedure, which are previously described. Thus, without specialized supervision, the model cannot generate meaningful action sequences for ADP tasks. To address this, we employ a two-stage supervised fine-tuning (SFT) curriculum.

Stage 1: Operator Syntax Learning. This stage focuses on the syntax and semantics of individual operators. Given source tables and a ground-truth operator pipeline $\mathcal{P}_{\text{total}}$ as defined in Eq. (1), consisting of a sequence of operators $(o^{(1)}, \dots, o^{(N)})$. We first execute these operators sequentially to collect a sequence of generated intermediate table sets $\{\mathcal{T}_0, \dots, \mathcal{T}_N\}$. Then, we sample a contiguous sub-sequence of operators $\mathcal{P}_{\text{sub}} = (o^{(i)}, \dots, o^{(j)})$ with the corresponding input intermediate table set $\mathcal{T}_{\text{in}} = \mathcal{T}_{i-1}$ and output intermediate table set $\mathcal{T}_{\text{out}} = \mathcal{T}_j$. In this way, we construct the operator syntax learning dataset $\mathcal{D}_{\text{op}}$ consisting of training triples

$(\mathcal{P}_{\text{sub}}, \mathcal{T}_{\text{in}}, \mathcal{T}_{\text{out}})$. Let $\phi(\cdot)$ denote the serialization function. It serializes $\mathcal{P}_{\text{sub}}$ by concatenating all serialized operators and intermediate table set $\mathcal{T}$ by serializing it into markdown format. We conduct SFT to optimize the following loss $\mathcal{L}_{\text{op}}$:

$$\mathcal{L}_{\text{op}} = - \sum_{(\mathcal{P}_{\text{sub}}, \mathcal{T}_{\text{in}}, \mathcal{T}_{\text{out}}) \in \mathcal{D}_{\text{op}}} \log \text{Pr}_{\text{LLM}}(\phi(\mathcal{P}_{\text{sub}}) \mid \phi(\mathcal{T}_{\text{in}}), \phi(\mathcal{T}_{\text{out}})). \quad (4)$$

Through Eq. (4), the agent learns to recognize operator function signatures and argument formats, while isolating operator execution logic from higher-level planning decisions.

Stage 2: Reasoning Procedure Learning. This stage aligns the model with the *Tree-based Agentic Reasoning* mechanism (Section 4.2). To achieve this, we synthesize a dataset of high-quality reasoning trajectories by distilling knowledge from strong teacher models (e.g., DeepSeek-R1). Specifically, for each ADP task defined by source tables $\mathcal{S}$ and a target schema Σ^* , we aim to generate a ground-truth trajectory τ . To enhance robustness, we apply input perturbations Φ , such as randomly shuffling rows and columns in the source tables. Next, we employ In-Context Learning [8] to guide the teacher model. By utilizing elaborate prompts that exemplify the tree-based reasoning process defined in Eq. (2), we generate a set of candidate trajectories $\mathcal{U}$. To unify the training objective, we represent each trajectory as a sequence $\tau = (r_1, \dots, r_m)$. Here, the initial subsequence $(r_1, \dots, r_{m-1})$ corresponds to the iterative reasoning steps $(R_1, \dots, R_T)$ defined in Eq. (3), while the final element r_m corresponds to the answer generation A in Eq. (2). These candidates are filtered using a process reward function $R_{\text{llm}}(\tau)$ (detailed in Section 5.2), and only the top-ranked trajectories are retained to form the final dataset $\mathcal{U}^*$. Finally, the model is fine-tuned on $\mathcal{U}^*$ to minimize the masked negative log-likelihood:

$$\mathcal{L}_{\text{reason}} = - \sum_{\tau \in \mathcal{U}^*} \sum_{t=1}^m M_t \cdot \log \text{Pr}_{\text{LLM}}(\phi(r_t) \mid \phi(r_{<t}), \phi(\mathcal{S}), \Sigma^*). \quad (5)$$

Here, r_t encapsulates both the current tree state and the agent’s generated response. The serialization function $\phi(\cdot)$ transforms these structured components into a linear token sequence. Crucially, we introduce a loss mask M_t to ensure that gradients are only calculated for tokens corresponding to the agent’s outputs.

5.2 Multi-turn RL with Hybrid Reward

Motivation. Data preparation pipeline generation is a sequential decision process with execution feedback, where each action affects subsequent states and future operator choices. Optimizing such behavior cannot be reduced to single-shot generation. Instead, the policy should be trained over multi-turn interaction trajectories. In each turn, the agent produces structured actions (e.g., $\langle \text{plan} \rangle$ and $\langle \text{expand} \rangle$), the environment executes the proposed operators, and feedback is returned. Then, subsequent decisions are conditioned on these execution outcomes.

Basic Idea. The above setting motivates us to develop a multi-turn reinforcement learning method, where the policy is optimized over action sequences rather than single outputs. Specifically, under

our interaction framework, each trajectory contains both (i) agent-generated tokens (actions and their contents) and (ii) environment-generated tokens, such as serialized intermediate tables and execution traces. In particular, since environment-generated tokens are deterministic execution outputs rather than policy decisions, they are excluded from policy gradient optimization. That is, we apply a binary token mask that removes gradients on tokens inside `<execute>` blocks, and optimize the policy only over agent decision tokens within `<plan>`, `<expand>`, and `<answer>`.

We optimize the policy using Group Relative Policy Optimization (GRPO) [35], which estimates baselines from group-level reward statistics, supporting more stable optimization over multi-turn trajectories. Using only final pipeline correctness as the reward, however, results in sparse and delayed learning signals. We therefore introduce a hybrid reward design, as described below.

Hybrid Reward. We design a hybrid reward mechanism that provides denser learning signals by evaluating both outcome quality and reasoning behavior. The overall reward $R(\tau)$ for a trajectory τ is defined as a weighted sum of three components:

$$R(\tau) = \alpha \cdot R_{\text{out}}(\tau) + \beta \cdot R_{\text{part}}(\tau) + \gamma \cdot R_{\text{llm}}(\tau). \quad (6)$$

where R_{out} , R_{part} , and R_{llm} correspond to strict correctness, partial progress, and reasoning process signals, respectively, and α, β, γ are hyperparameters that balance their contributions. Now, we detail each of the three components R_{out} , R_{part} , and R_{llm} as follows.

Outcome Reward R_{out} . This reward provides a direct success signal based on final output correctness. When the `<answer>` action is issued, the generated resulting target table $\hat{T}$ is evaluated by:

$$R_{\text{out}}(\tau) = \mathbb{I}(\hat{T} \equiv T^*). \quad (7)$$

where $\mathbb{I}(\cdot)$ denotes the indicator function.

Partial Reward R_{part} . This reward provides a partial-progress signal when strict correctness is not achieved ($R_{\text{out}}(\tau) = 0$). We measure the similarity between the generated table $\hat{T}$ and ground-truth T^* , and define $R_{\text{part}}(\tau) = \text{avg}(S_{\text{sch}}, S_{\text{shp}}, S_{\text{cnt}})$, where $S_{\text{sch}}, S_{\text{shp}}, S_{\text{cnt}}$ evaluate alignment in schema, shape and content, respectively:

$$S_{\text{sch}} = \frac{|C_{\hat{T}} \cap C_{T^*}|}{|C_{\hat{T}} \cup C_{T^*}|}, \quad S_{\text{shp}} = \exp\left(-\left|\frac{|\hat{D}| - |D^*|}{|D^*|}\right|\right), \quad (8)$$

$$S_{\text{cnt}} = \frac{1}{|C_m|} \sum_{c \in C_m} \frac{\sum_i \mathbb{I}(\hat{D}[c]_i = D^*[c]_i)}{\max(|\hat{D}|, |D^*|)}.$$

Here, $C_{\hat{T}}$ and C_{T^*} denote the column-name sets, $|\hat{D}|$ and $|D^*|$ denote the row counts, and $C_m = C_{\hat{T}} \cap C_{T^*}$ is the set of matched columns used for cell-wise value comparison.

Process Reward R_{llm} . Relying solely on data similarity rewards may lead to *reward hacking*, which has been studied in other reinforcement learning settings [38, 46], and can also arise in ADP scenarios (e.g., renaming columns without performing the required data cleaning). To discourage such “shortcut” strategies, we introduce a process reward that evaluates the quality of the reasoning trajectory. We use an instruction-tuned LLM (e.g., GPT-4o) as a judge to score each trajectory based on the actions defined in Section 4.

Specifically, the judge evaluates three criteria: (1) *Plan-action consistency*, measuring whether generated pipeline (from `<expand>`)

Table 1: Statistics of datasets.

Dataset	Statistics		Pipeline Length	# Op Types
	# Train	# Test		
Synth-Spider	6,788	2,008	1~28	31
Synth-Bird	-	1,150	2~25	31
Parrot [12]	13,965	1,365	1~8	17
Buildings [36]	-	105	-	-

correctly implements the plan (from `<plan>`). (2) *Feedback responsiveness*, checking whether subsequent `<plan>` outputs explicitly respond to the prior execution feedback (e.g., reported error traces). (3) *Backtracking justification*, verifying that parent-node switches are supported by recorded failure evidence from the current branch.

5.3 Data Synthesis for Training

To train our agentic model, we require supervision in the form of executable data preparation pipelines together with their source and target tables. Since such ADP task instances are not available at scale, we construct them through synthesis by converting SQL benchmarks into ADP tasks consisting of source tables, a target table, and a corresponding transformation pipeline. Synthesizing such training tasks has two key challenges. First, the generated pipelines should correspond to meaningful analytical transformations rather than arbitrary operator chains, so that the supervision reflects realistic operator dependencies. Second, the synthesized source tables should include diverse data quality issues while preserving the intended transformation logic. Excessive noise injection may break key fields or relationships [31], making it difficult to construct any feasible pipeline that produces the target table.

ADP Task Construction from NL2SQL. To obtain realistic ADP tasks, we instantiate them from NL2SQL benchmarks [27, 44]. For each query q , we execute it to produce the target table T^* and use an LLM to generate the corresponding target schema specification. We also translate q into an operator pipeline by generating candidate pipelines with an LLM and selecting the shortest one that exactly reproduces T^* through execution. This yields a clean task instance consisting of source tables, a target schema specification, a target table, and an operator pipeline.

Reversible Noise Injection. Since NL2SQL data is typically clean, we introduce controlled noise to increase data diversity for training. We inject noise by applying inverse transformations of data-cleaning operators, so that each corruption corresponds to a valid cleaning step. Specifically, we sample an operator and use an LLM to generate its inverse transformation logic. For example, the inverse of a date-standardization operator converts a normalized date (e.g., “2023-01-01”) into heterogeneous formats (e.g., “01/01/23”). After each corruption step, we verify executability by applying the corresponding cleaning operator and checking that the previous table state is restored. Only reversible corruptions are kept. Repeating this process produces dirty source tables together with a matching cleaning pipeline. The final ground-truth pipeline is formed by concatenating the cleaning pipeline with the task pipeline.

6 EXPERIMENTS

6.1 Experimental Setup

Datasets. As shown in Table 1, we evaluate DEEPPREP on four

datasets, including one public benchmark, two datasets synthesized using our synthesis method, and one real-world dataset.

(1) **Parrot** [12] is a benchmark for translating natural language into data preparation pipelines. To make it suitable for ADP evaluation, we augment each case by generating the task specification from the target tables and operator sequences using an LLM [34].

Note that the public benchmark **Parrot** [12] mainly contains short operator pipelines and limited operator-type coverage. We therefore synthesize two datasets to provide more complex pipelines and broader operator coverage, using our method in Section 5.3.

(2) **Synth-Spider** is a dataset synthesized by our method from the Spider NL2SQL benchmark [44]. Using the ADP task synthesis pipeline described in Section 5.3, we convert NL2SQL cases into ADP tasks with executable operator pipelines.

(3) **Synth-Bird** is another synthesized dataset constructed from the Bird benchmark [27], which features larger databases and more complex analytical queries. We apply the same synthesis procedure to generate ADP tasks from the Bird validation set. As we use Synth-Bird only for evaluation, this dataset only has test split.

(4) **Buildings** [36] is a real-world dataset comprising 105 tasks collected from data processing pipelines in the smart building domain across 21 energy companies. It involves messy, real-world data with complex multi-step data preparation operations. It only provides source tables and target table schema descriptions.

Evaluation Methodology. For comprehensive evaluation, all models are trained only on the **Synth-Spider** training set and evaluated under two test settings: *in-domain* and *out-of-domain*.

(1) *In-domain Test.* We evaluate on the **Synth-Spider** test set, which is synthesized using the same procedure as the training data.

(2) *Out-of-domain Test.* **Synth-Bird** is closer to the training distribution since it is generated by the same synthesis pipeline but from a different benchmark. **Parrot** is an independent dataset with different task characteristics. **Buildings** is a real-world dataset with complex multi-step data preparation.

Baselines. We compare DEEPREP with both *prompting baselines* and *training baselines*. Prompting baselines use off-the-shelf LLMs, while training baselines involve parameter updates.

(1) *Prompting Baselines.* These methods use frozen LLMs with different planning and search strategies to generate pipelines.

- CodeGen [29, 33] prompts an LLM to iteratively generate and execute data preparation code until the target table is obtained.
- Plan-and-Solve (PaS) [39] prompts an LLM to produce a plan and then generates an operator pipeline to obtain the target table.
- ReAct [2] is a linear reasoning agent that iteratively predicts the next operator based on previous execution results.
- MCTS-OP [24] applies Monte Carlo Tree Search (MCTS) over individual operators defined in Section 2.2
- MontePrep [11] applies MCTS over actions such as schema mapping, operator discovery, code synthesis, and refinement instead of individual operators.
- DeepAnalyze [45] is an agentic LLM that generates data processing programs using supervised and reinforcement learning.
- AutoPrep [9] is a multi-agent framework for data preparation. It generates a high-level plan and then produces data preparation programs through specialized agents.

- TRAE and Claude Code (CC) are AI-powered coding agents with agentic coding capabilities. We evaluate them as representative state-of-the-art agentic coding systems².

(2) *Training Baselines.* These baselines use the same backbone models as DEEPREP but adopt different training strategies, allowing us to evaluate the effect of our agentic training framework.

- Imitation Learning (IL) [26] fine-tunes the model on the high-quality trajectories distilled from strong teacher models.
- RL-GRPO [35] applies reinforcement learning with Group Relative Policy Optimization using only outcome-level rewards.

Prompt Design and Tuning for Baseline Methods. Specifically, (1) for methods with public implementations, i.e., MontePrep [11], DeepAnalyze [45], AutoPrep [9], and MCTS-Op [24], we use their official codebases and default configurations (e.g., prompt design, parameters, etc.). When a method was designed for a different task and could not be directly applied to ADP, we adapted its prompt to the ADP setting and tuned its parameters using the same grid-search procedure adopted by DEEPREP. For example, AutoPrep was designed for TableQA rather than autonomous data preparation, so we replaced its original prompt with an ADP-oriented prompt and tuned it accordingly. (2) For methods without public implementations, including CodeGen, ReAct, and PaS, we reimplemented them following the descriptions in their papers and applied the same prompt-tuning and parameter-selection procedure used for DEEPREP. Therefore, all methods were evaluated under a comparable tuning method. Complete prompts, hyperparameters, and implementation details are provided in our technical report [1].

Evaluation Metrics. We evaluate all methods using three metrics.

(1) *Exact Match Accuracy (Acc.)* measures the percentage of test cases

where the produced table $\hat{T}$ matches the ground-truth table T^* . We follow the table matching method in [12], which is invariant to row and column permutations but requires exact cell-value equality.

(2) *Completion Rate (Comp.)* measures the percentage of test cases that produce a final output table. A case is counted as incomplete if execution exceeds the maximum interaction limit, triggers runtime errors, or produces an empty result.

(3) *Inference Cost (Cost)* measures the average monetary cost per case. For closed-source LLMs, we use official API pricing. For open-source models, we convert end-to-end runtime to cost using the hourly GPU price: $c_i = p_{\text{gpu}} t_i / 3600$, where t_i is the runtime in seconds. We report the average over all cases, using an A800-80G GPU priced at \$0.91/hour.

Backbone LLMs and DEEPREP Variants. We implement DEEPREP on both closed-source and open-source LLM backbones.

(1) *Closed-source backbones.* We implement DEEPREP with our proposed tree-based agentic reasoning directly on the strongest proprietary models, including GPT-5 (i.e., gpt-5-mini-2025-08-07) and Claude-4 (i.e., claude-sonnet-4-20250514).

(2) *Open-source backbones.* We implement DEEPREP based on open-source LLMs, i.e., Qwen2.5 [18] (with 0.5B, 1.5B, 3B, 7B and 14B parameters) and Qwen3 [40] (0.6B, 1.7B, 4B, 8B and 14B).

²We use CC v2.1.119 and TRAE from <https://github.com/bytedance/trae-agent>.

Table 2: Comparison of DEEPPREP with baselines on open-source LLMs. The best and second-best results are highlighted in bold and underlined, respectively.

(a) Results on Qwen3-14B.

Method	Synth-Spider		Synth-Bird		Parrot	
	Acc.	Comp.	Acc.	Comp.	Acc.	Comp.
<i>Prompting Baselines</i>						
PaS	7.67	20.67%	3.09	21.86%	9.56	22.36%
MCTS-OP	21.91	72.56%	7.22	78.69%	24.95	83.02%
MontePrep	9.46	19.62%	18.63	38.15%	26.94	57.87%
AutoPrep	36.75	71.61%	10.41	39.89%	28.54	74.05%
CodeGen	45.47	68.18%	29.48	49.20%	30.83	81.08%
ReAct	40.39	69.02%	16.04	46.56%	30.80	76.44%
<i>Training Methods (Only Trained on Synth-Spider)</i>						
IL	<u>61.70</u>	<u>85.26%</u>	<u>48.01</u>	<u>81.23%</u>	37.95	<u>89.39%</u>
RL-GRPO	51.22	77.68%	32.25	67.22%	35.06	82.9%
DEEPPREP (Ours)	67.18	97.21%	54.09	92.26%	40.44	96.41%

(b) Results on Qwen3-8B.

Method	Synth-Spider		Synth-Bird		Parrot	
	Acc.	Comp.	Acc.	Comp.	Acc.	Comp.
<i>Prompting Baselines</i>						
PaS	4.68	7.42%	0.65	4.33%	3.09	8.32%
MCTS-OP	12.90	15.19%	0.90	73.56%	17.33	23.71%
MontePrep	8.81	19.97%	13.70	21.96%	25.95	56.08%
AutoPrep	26.44	50.85%	8.52	42.28%	25.20	70.42%
CodeGen	6.27	12.10%	7.47	15.69%	7.27	25.50%
ReAct	29.63	52.59%	10.41	46.22%	22.11	57.97%
DeepAnalyze	35.66	82.02%	1.74	62.17%	29.01	66.67%
<i>Training Methods (Only Trained on Synth-Spider)</i>						
IL	<u>59.65</u>	<u>90.54%</u>	<u>43.30</u>	<u>80.43%</u>	<u>36.15</u>	<u>91.08%</u>
RL-GRPO	47.81	76.54%	29.95	68.69%	32.82	85.05%
DEEPPREP (Ours)	65.99	97.46%	53.39	92.78%	39.93	98.46%

6.2 Overall Comparison

Exp-1: Accuracy and Completion Rate Comparison. We compare DEEPPREP with prompting baselines and training baselines on accuracy and completion rate. Results on Qwen3-14B and Qwen3-8B are reported in Table 2, as they are strong open-source backbones at two model scales commonly used in code and data tasks. Additional backbone results are presented in later experiments.

(1) In-Domain Test. DEEPPREP achieves the highest accuracy and completion rate across both backbones on the Synth-Spider test set. On Qwen3-14B, it reaches 67.18 accuracy (Table 2a), exceeding the strongest prompting baseline CodeGen by 21.71 points and the strongest training baseline by 5.48 points, with similar gains in completion rate. These gains stem from combining tree-structured exploration with execution-grounded actions, enabling the agent to revise earlier decisions and avoid invalid operator sequences.

(2) Out-of-Domain Test. Under the single-source training method, DEEPPREP maintains a clear advantage on out-of-domain datasets. On Synth-Bird, DEEPPREP (Qwen3-14B) achieves 54.09 accuracy compared with 48.01 for IL, and the 8B model achieves 53.39 versus 43.30. On Parrot, we observe a similar ordering among methods. DeepAnalyze shows much lower performance on Synth-Bird (1.74 accuracy), indicating limited robustness when pipeline length and operator dependencies increase. These results align with our training design, where progressive curriculum and hybrid rewards

Table 3: Overall Ablation Study on Qwen3-8B. PAT represents Progressive Agentic Training and TAR represents Tree-based Agentic Reasoning.

Method	Synth-Spider		Synth-Bird		Parrot	
	Acc.	Comp.	Acc.	Comp.	Acc.	Comp.
DeepPrep	65.99	97.46%	53.39	92.78%	39.93	98.46%
w/o PAT	22.21	59.76%	8.87	41.90%	25.27	70.62%
w/o PAT, TAR	4.68	7.42%	0.65	4.33%	3.09	8.32%

expose the model to multi-step, feedback-driven reasoning trajectories rather than single-pass program generation.

Finding 1: DEEPPREP achieves the highest accuracy and completion rate among the compared methods and shows good generalization across datasets.

Exp-2: Cost-Accuracy Trade-off Comparison. We compare methods under a cost-accuracy trade-off by jointly evaluating accuracy and cost. Figure 5 plots prompting baselines and DEEPPREP variants across different LLM backbones, and shows two Pareto frontiers: *Previous Pareto*, computed from baselines only, and *Ours Pareto*, computed after adding DEEPPREP.

First, DEEPPREP lies on or near the Pareto frontier in most settings and extends the frontier across all three datasets. In Figure 5, the *Ours Pareto* curve is shifted toward the upper-left relative to the *Previous Pareto*, meaning that DEEPPREP achieves higher accuracy at the same cost, or similar accuracy at lower cost. This appears on Synth-Spider, Synth-Bird, and Parrot, indicating that the cost-accuracy advantage is consistent across datasets and backbones.

Second, DEEPPREP reaches accuracy close to strong closed-source baselines at substantially lower cost. On Synth-Spider, DEEPPREP (Qwen3-14B) achieves 67.18 accuracy at a very low cost, achieving a comparable accuracy with ReAct on Claude-4 (69.92 accuracy) and GPT-5 (67.03 accuracy) while reducing cost by about 57× and 15×, respectively. This is attributed to our design that uses structured tree-based reasoning to guide smaller or open models toward valid pipelines with fewer failed executions.

Third, compared with the agentic coding systems TRAE and Claude Code, DEEPPREP achieves comparable accuracy at much lower cost. For example, with GPT-5, DEEPPREP achieves accuracy comparable to TRAE while reducing cost by 2.7× on Synth-Spider. This further demonstrates the cost-effectiveness of DEEPPREP.

Finding 2: DEEPPREP extends the cost-accuracy Pareto frontier, achieving accuracy close to strong closed-source baselines at substantially lower inference cost.

6.3 Ablation Studies

Exp-3: Overall Ablation Studies of DEEPPREP. We evaluate the contribution of two modules (TAR and PAT) in DEEPPREP. The results are reported in Table 3. As reported, removing PAT while retaining TAR causes substantial degradation, reducing accuracy by 43.78 on Synth-Spider, 44.52 on Synth-Bird, and 14.66 on Parrot. This indicates that the model must first learn the operator syntax and reasoning protocol before tree-based reasoning can be effectively utilized. Further removing TAR causes the method to degenerate into single-turn operator generation, leading to a larger performance drop (by 61.31 on Synth-Spider). Meanwhile, the completion rate falls to near-zero levels. This demonstrates that TAR

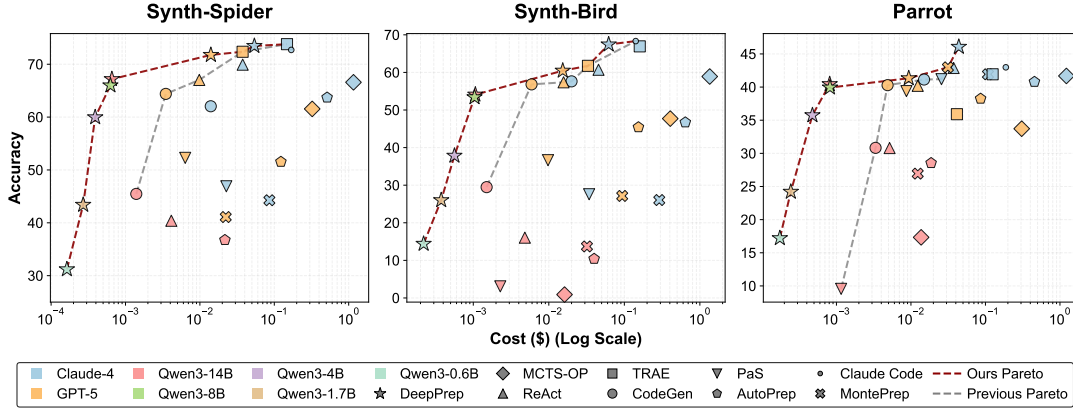


Figure 5: Cost-Accuracy trade-off comparison. Colors represent different LLM backbones, and shapes denote different methods. The red and grey dashed lines indicate the Pareto frontiers of our method and previous baselines, respectively.

Table 4: Detailed Ablation Studies on PAT (Qwen3-8B).

(a) Ablation study on Cold Start stage. OSL represent Operator syntax Learning module in Section 5.1

Method	Synth-Spider		Synth-Bird		Parrot	
	Acc.	Comp.	Acc.	Comp.	Acc.	Comp.
Cold-Start	60.66	91.63%	45.27	81.39%	36.22	92.16%
w/o OSL	59.65	90.54%	43.30	80.43%	36.15	91.08%

(b) Ablation study on the reward of Multiturn RL stage.

Method	Synth-Spider		Synth-Bird		Parrot	
	Acc.	Comp.	Acc.	Comp.	Acc.	Comp.
DEEPPREP	65.99	97.46%	53.39	92.78%	39.93	98.46%
w/o R_{llm}	64.54	96.96%	51.67	92.31%	39.12	94.21%
w/o R_{llm}, R_{part}	61.85	98.71%	47.15	95.57%	37.22	99.34%

is critical for leveraging execution feedback, and recovering from erroneous decisions through backtracking.

Exp-4: Detailed Ablation Studies on Progressive Agentic Training. We evaluate the contribution of specific components within our progressive agentic training. The results are detailed in Table 4.

Table 4a demonstrates that cold-start curriculum is useful. Removing this component causes a simultaneous decline in both accuracy and completion rate across all datasets. For instance, the completion rate on Synth-Spider drops from 91.63 to 90.54. Because this component familiarizes the model with the operator syntax and parameters. Without it, the model tends to generate non-executable actions and fail to complete the reasoning trajectory.

Table 4b validates the design of our reward function in RL. First, removing process reward (R_{llm}) results in an accuracy drop, which shows that R_{llm} prevents “reward hacking”. Without semantic verification, the policy learns to satisfy superficial schema constraints, such as renaming columns, while neglecting the actual data transformation logic. Second, relying solely on outcome rewards (w/o R_{out}, R_{part}) yields the highest completion rate but the lowest accuracy. This suggests that sparse rewards induce conservative behavior. The agent avoids complex exploration or backtracking to minimize the risk of runtime errors. Thus, it prioritizes safe but incorrect paths to ensure the episode terminates successfully.

Finding 3&4: Each key component of DEEPPREP contributes to its overall performance.

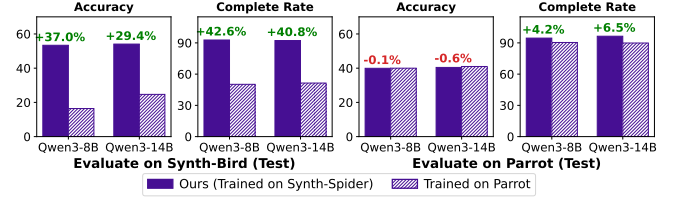


Figure 6: Evaluation results of models trained on Synth-Spider (Train) and Parrot (Train).

Table 5: Results on the Buildings dataset (real-world).

Method	Acc.	Comp.	Cost (\$)
ReAct	74.29	99.05%	0.0052
CodeGen	71.43	100%	0.0054
MontePrep	72.38	86.67%	0.043
MCTS-OP	69.52	99.05%	0.057
DEEPPREP (Ours)	82.85	100%	0.0082

6.4 In-Depth Analysis

Exp-5: Effectiveness of Synthesized Training Data. We evaluate whether models trained on our synthesized Synth-Spider dataset generalize better than those trained on the public Parrot dataset. We first evaluate these two models on Synth-Bird (Test) under the out-of-domain setting. As illustrated in the left of Figure 6, training on Synth-Spider yields much higher performance: accuracy increases by 37.0 and 29.4 points on Qwen3-8B and Qwen3-14B over Parrot. To further highlight the effectiveness of synthesized data, we adopt a more challenging setting. We evaluate models trained on Synth-Spider under a totally out-of-domain setting and models trained on Parrot under an in-domain setting on Parrot (Test), as illustrated in the right of Figure 6. Our synthesized data are still competitive under such a challenging experimental setting. Models trained on Synth-Spider achieve accuracy comparable to models trained on Parrot (40.44 vs. 40.99 on Qwen3-8B).

Finding 5: Our synthetic training data provides effective supervision for agentic training, significantly improving the performance on unseen tasks.

Exp-6: Generalization on Real-World Dataset. We have compared DEEPPREP with prompting baselines on the real-world dataset Buildings [36]. Table 5 reports Accuracy, Completion Rate and API

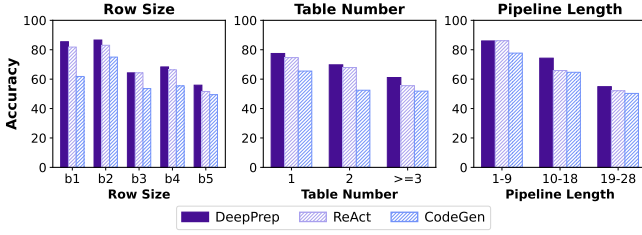


Figure 7: Scalability analysis on Synth-Spider with varying (a) row size, (b) source table number, and (c) pipeline length. Cost per case. As illustrated, DEEPPREP generalizes well on the real-world dataset. Specifically, DEEPPREP achieves the best Accuracy of 82.85 and maintains relatively low inference cost (0.008\$ per case). This enhances its usability in real-world.

Error Analysis. We further conduct an error analysis of this experiment and identify two key challenges in real-world scenarios. (1) Real cases often contain abbreviated or domain-specific column names, leading to selecting the wrong columns required to be mapped into the target columns. (2) The target specification in real cases is often under-specified. For example, whether daily or monthly granularity is expected is not specified clearly in the requirement. This increases the reasoning burden during the data preparation stage. We will address these challenges in our future work. The complete statistics and description of the error analysis can be found in our technical report [1].

Finding 6: DEEPPREP generalizes well to the real-world Buildings dataset.

Exp-7: Scalability Analysis. We evaluate the scalability of DEEPPREP on Synth-Spider from three dimensions: row size, number of source tables, and pipeline length and report the accuracy results, time usage and resource usage.

Scalability Analysis of Accuracy. As shown in Figure 7(a), DEEPPREP remains robust on small and medium-sized tables. Performance degrades only for the largest row-size bucket (b_5 : 36 to 124k+ rows), where accuracy decreases by 29.54 points compared with the smallest bucket (b_1 : 1 to 9 rows). This degradation is primarily caused by the table sampling strategy used to control token usage, where relevant information may be omitted from the sampled content. As shown in Figure 7, performance decreases as more source tables are involved, but DEEPPREP consistently achieves the highest accuracy across different table number groups. As illustrated, all methods degrade as the pipeline length increases. However, DEEPPREP shows the smallest degradation, and its advantage over baselines becomes larger on complex cases.

Scalability Analysis of Time and Resource Usage. DEEPPREP exhibits stable growth in both execution time and memory usage as workload complexity increases. For example, on Synth-Spider, the average execution time increases only from 8.91 to 11.30 seconds when moving from the smallest to the largest column-size bucket, while memory usage remains between 66 MB and 69 MB. The stable scaling behavior can be explained by the generative search framework, as defined in Section 4. The process is bounded by the maximum exploration turn T and the maximum operator sequence length L per expansion, yielding a worst-case complexity of $O(T \times L)$. Unlike MCTS-based methods, DEEPPREP generates only

one operator pipeline conditioned on the current tree state. The “pruning” is implicit in the LLM’s decision-making. Consequently, the effective branching factor depends on the LLM’s learned policy and remains bounded regardless of input size, which explains the stable growth in time and memory usage.

Complete scalability results across all datasets and detailed time and resource usage analysis are provided in our technical report [1].

Finding 7: DEEPPREP maintains robust accuracy and stable resource usage as workload complexity increases.

7 RELATED WORK

Data Preparation. Prior data preparation work mainly focuses on programming-by-example (PBE). PBE systems [3, 13, 16, 21, 37] synthesize transformations from input-output examples, which are effective for well-specified transformations but require example-level supervision for each task. Other systems target specific data preparation sub-problems [5, 9, 10, 17, 22, 28, 41]. For example, Auto-Tables [28] focuses on table reconstruction operations such as pivoting and stacking. Such systems are not designed to jointly handle value-level cleaning [4] and multi-table integration [48]. Text-to-Pipeline [12] maps natural language descriptions to pipelines, but assumes detailed step-wise specifications as input. In contrast, DEEPPREP targets end-to-end autonomous data preparation from high-level target schema descriptions and supports a much broader operator space covering cleaning, reshaping, and integration.

LLM-Powered Data Preparation. Recent work explores using LLMs for data preparation pipeline construction. Prompting-based approaches directly generate transformation code from natural language specifications [22, 29, 33]. To incorporate execution feedback, ReAct-style methods interleave reasoning with tool actions and update subsequent decisions based on observed results [2, 43]. Such approaches follow a linear interaction trajectory, where actions are appended sequentially. More recent work explores tree-structured search over reasoning or operator spaces, including Tree-of-Thoughts (ToT) [42] and MCTS-based agents [11, 24]. For example, MCTS-style approaches typically rely on rollout-based value estimates and scalar reward signals to guide node selection and backpropagation [15]. However, data preparation pipelines expose structured execution feedback, such as schema mismatches, execution errors for operators, and intermediate table states. DEEPPREP is designed to operate directly over such execution-grounded states, where each node corresponds to materialized tables and branch decisions are guided by structured execution feedback rather than scalar rollout statistics. This design supports operator-level revision and state-aware backtracking during pipeline construction.

8 CONCLUSION

We propose DEEPPREP, an LLM-powered agentic system for autonomous data preparation. DEEPPREP constructs pipelines through iterative, execution-grounded interaction, with tree-based agentic reasoning enabling structured exploration and non-local revision. To train the model, we introduce progressive agentic training with a hybrid reward and data synthesis. Experiments show that DEEPPREP achieves accuracy comparable to GPT-5 at 15× lower cost while establishing state-of-the-art among open-source baselines.

REFERENCES

- [1] 2026. *Technical Report*. <https://anonymous.4open.science/r/DeepPrep-0432/tree/main/pdf/report.pdf>
- [2] Renat Aksitov, Sobhan Miryosefi, Zonglin Li, Daliang Li, Sheila Babayan, Kavya Kopparapu, Zachary Fisher, Ruiqi Guo, Sushant Prakash, Pranesh Srinivasan, et al. 2023. Rest meets react: Self-improvement for multi-step reasoning llm agent. *arXiv preprint arXiv:2312.10003* (2023).
- [3] Daniel W Barowy, Sumit Gulwani, Ted Hart, and Benjamin Zorn. 2015. FlashRelate: extracting relational data from semi-structured spreadsheets using examples. *ACM SIGPLAN Notices* 50, 6 (2015), 218–228.
- [4] Chengliang Chai, Kaisen Jin, Nan Tang, Ju Fan, Dongjing Miao, Jiayi Wang, Yuyu Luo, Guoliang Li, Ye Yuan, and Guoren Wang. 2025. Cost-effective Missing Value Imputation for Data-effective Machine Learning. *ACM Transactions on Database Systems* 50, 3 (2025), 1–36.
- [5] Qixu Chen, Yeye He, Raymond Chi-Wing Wong, Weiwei Cui, Song Ge, Haidong Zhang, Dongmei Zhang, and Surajit Chaudhuri. 2025. Auto-Test: Learning Semantic-Domain Constraints for Unsupervised Error Detection in Tables. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–27.
- [6] Tamraparni Dasu and Theodore Johnson. 2003. *Exploratory data mining and data cleaning*. John Wiley & Sons.
- [7] Haoyu Dong and Zhiruo Wang. 2024. Large language models for tabular data: Progresses and future directions. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2997–3000.
- [8] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Baobao Chang, et al. 2024. A survey on in-context learning. In *Proceedings of the 2024 conference on empirical methods in natural language processing*. 1107–1128.
- [9] Meihao Fan, Ju Fan, Nan Tang, Lei Cao, Guoliang Li, and Xiaoyong Du. 2024. AutoPrep: Natural Language Question-Aware Data Preparation with a Multi-Agent Framework. *CoRR abs/2412.10422* (2024). <https://doi.org/10.48550/ARXIV.2412.10422> arXiv:2412.10422
- [10] Meihao Fan, Xiaoyue Han, Ju Fan, Chengliang Chai, Nan Tang, Guoliang Li, and Xiaoyong Du. 2024. Cost-effective in-context learning for entity resolution: A design space exploration. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 3696–3709.
- [11] Congcong Ge, Yachuan Liu, Yixuan Tang, Yifan Zhu, Yaofeng Tu, and Yunjun Gao. 2025. MontePrep: Monte-Carlo-Driven Automatic Data Preparation without Target Data Instances. *arXiv preprint arXiv:2509.17553* (2025).
- [12] Yuhang Ge, Yachuan Liu, Yuren Mao, and Yunjun Gao. 2025. Text-to-Pipeline: Bridging Natural Language and Data Preparation Pipelines. *arXiv preprint arXiv:2505.15874* (2025).
- [13] Sumit Gulwani, William R Harris, and Rishabh Singh. 2012. Spreadsheet data manipulation using examples. *Commun. ACM* 55, 8 (2012), 97–105.
- [14] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shiroong Ma, Peiyou Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [15] Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhitong Hu. 2023. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 8154–8173.
- [16] Yeye He, Xu Chu, Kris Ganjam, Yudian Zheng, Vivek Narasayya, and Surajit Chaudhuri. 2018. Transform-data-by-example (TDE) an extensible search engine for data transformations. *Proceedings of the VLDB Endowment* 11, 10 (2018), 1165–1177.
- [17] Zhipeng Huang and Yeye He. 2018. Auto-detect: Data-driven error detection in tables. In *Proceedings of the 2018 International Conference on Management of Data*. 1377–1392.
- [18] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
- [19] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, Vol. 2024. 54107–54157.
- [20] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516* (2025).
- [21] Zhongjun Jin, Michael R Anderson, Michael Cafarella, and HV Jagadish. 2017. Foofah: Transforming data by example. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 683–698.
- [22] Eugenie Y Lai, Yeye He, and Surajit Chaudhuri. 2025. Auto-Prep: Holistic Prediction of Data Preparation Steps for Self-Service Business Intelligence. *arXiv preprint arXiv:2504.11627* (2025).
- [23] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. 2022. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems* 35 (2022), 21314–21328.
- [24] Boyan Li, Jiayi Zhang, Ju Fan, Yanwei Xu, Chong Chen, Nan Tang, and Yuyu Luo. 2025. Alpha-sql: Zero-shot text-to-sql using monte carlo tree search. *arXiv preprint arXiv:2502.17248* (2025).
- [25] Changlun Li, Chenyu Yang, Yuyu Luo, Ju Fan, and Nan Tang. 2025. Weak-to-strong prompts with lightweight-to-powerful llms for high-accuracy, low-cost, and explainable data transformation. *Proceedings of the VLDB Endowment* 18, 8 (2025), 2371–2384.
- [26] Dacheng Li, Shiyi Cao, Tyler Griggs, Shu Liu, Xiangxi Mo, Eric Tang, Sumanth Hegde, Kourosh Hakhmaneshi, Shishir G Patil, Matei Zaharia, et al. 2025. LLMs Can Easily Learn to Reason from Demonstrations Structure, not content, is what matters! *arXiv preprint arXiv:2502.07374* (2025).
- [27] Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems* 36 (2024).
- [28] Peng Li, Yeye He, Cong Yan, Yue Wang, and Surajit Chaudhuri. 2023. Auto-Tables: Synthesizing Multi-Step Transformations to Relationalize Tables without Using Examples. *Proc. VLDB Endow.* 16, 11 (2023), 3391–3403. <https://doi.org/10.14778/3611479.3611534>
- [29] Xue Li and Till Döhmen. 2024. Towards efficient data wrangling with llms using code generation. In *Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning*. 62–66.
- [30] Weizheng Lu, Jing Zhang, Ju Fan, Zihao Fu, Yueguo Chen, and Xiaoyong Du. 2025. Large language model for table processing: A survey. *Frontiers of Computer Science* 19, 2 (2025), 192350.
- [31] Theodoros Rekatsinas, Xu Chu, Ihab F Ilyas, and Christopher Ré. 2017. Holoclean: Holistic data repairs with probabilistic inference. *arXiv preprint arXiv:1702.00820* (2017).
- [32] Yucheng Ruan, Xiang Lan, Jingying Ma, Yizhi Dong, Kai He, and Mengling Feng. 2024. Language modeling on tabular data: A survey of foundations, techniques and evolution. *arXiv preprint arXiv:2408.10548* (2024).
- [33] Ranjan Sapkota, Konstantinos I Roumeliotis, and Manoj Karkee. 2025. Vibe coding vs. CodeGen: Fundamentals and practical implications of agentic ai. *arXiv preprint arXiv:2505.19443* (2025).
- [34] ByteDance Seed, Jiaze Chen, Tiantian Fan, Xin Liu, Lingjun Liu, Zhiqi Lin, Mingxuan Wang, Chengyi Wang, Xiangpeng Wei, Wenyan Xu, et al. 2025. Seed1.5-thinking: Advancing superb reasoning models with reinforcement learning. *arXiv preprint arXiv:2504.13914* (2025).
- [35] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).
- [36] Ankita Sharma, Xuanmao Li, Hong Guan, Guoxin Sun, Liang Zhang, Lanjun Wang, Kesheng Wu, Lei Cao, Erkang Zhu, Alexander Sim, et al. 2023. Automatic data transformation using large language model—an experimental study on building energy data. In *2023 IEEE International Conference on Big Data (Big-Data)*. IEEE, 1824–1834.
- [37] Rishabh Singh. 2016. Blinkfill: Semi-supervised programming by example for syntactic string transformations. *Proceedings of the VLDB Endowment* 9, 10 (2016), 816–827.
- [38] Mia Taylor, James Chua, Jan Betley, Johannes Treutlein, and Owain Evans. 2025. School of reward hacks: Hacking harmless tasks generalizes to misaligned behavior in llms. *arXiv preprint arXiv:2508.17511* (2025).
- [39] Xuezhi Wang and Denny Zhou. 2024. Chain-of-thought reasoning without prompting. *Advances in Neural Information Processing Systems* 37 (2024), 66383–66409.
- [40] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [41] Junwen Yang, Yeye He, and Surajit Chaudhuri. 2021. Auto-pipeline: synthesizing complex data pipelines by-target using reinforcement learning and search. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2563–2575.
- [42] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
- [44] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 3911–3921.

- [45] Shaolei Zhang, Ju Fan, Meihao Fan, Guoliang Li, and Xiaoyong Du. 2025. Deep-analyze: Agentic large language models for autonomous data science. *arXiv preprint arXiv:2510.16872* (2025).
- [46] Yuxin Zhang, Meihao Fan, Ju Fan, Mingyang Yi, Yuyu Luo, Jian Tan, and Guoliang Li. 2025. Reward-sql: Boosting text-to-sql via stepwise reasoning and process-supervised rewards. *arXiv preprint arXiv:2505.04671* (2025).
- [47] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. 2025. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. *arXiv preprint arXiv:2504.03160* (2025).
- [48] Erkang Zhu, Yeye He, and Surajit Chaudhuri. 2017. Auto-join: Joining tables by leveraging transformations. *Proceedings of the VLDB Endowment* 10, 10 (2017), 1034–1045.

```

# Role Definition
You are a data expert with extensive knowledge in data preparation.

# Operation Space
{operator_definition}

# Demonstration
## Demonstration 1
## Demonstration 2
...
# Hint
You are dealing with a complex task with multiple rounds of interaction, don't try to complete it in one step.

- Output with tags below to answer the question.
- <think> your_think_here </think>: This tag is used for analyzing what action should be taken next.
- <operator> your_operator_chain_here </operator> or <solution> your_operator_chain_here </solution>
- The operator chain are concatenated by " -> ", which will be executed by an external executor.
- If previous exploration results can deduce a correct operator chain to get the target table or it is the last turn, use the <solution> tag to summarize the correct operator chain. Output your final solution with operators used in the previous exploration. The solution must end with a Terminate operator.
...
You will have {max_explore_turn} to explore. For each exploration, you should first reflect with the tag <think> based on the previous <observation>, and then use <operator> tag to explore. After exploration, you must output a solution with tag <solution>. Make sure all operators in the <solution> can be executed successfully.

# Initial Input Tables & Target Table Schema
...
# Completion
follow the Rules in "#Hint" Section.

```

Figure 8: Prompt of DEEPPREP.

A IMPLEMENTATION DETAILS OF TREE-BASED AGENTIC REASONING

In this section, we provide the implementation details of the tree-based agentic reasoning mechanism described in Section 4, including prompt construction, schema handling, table content representation, state representation, failed branch reuse, and the algorithm pseudo-code.

A.1 Prompt Construction

We present the detailed prompts of DEEPPREP. The prompt of DEEPPREP consists of an initial prompt and trajectory prompts that are incrementally updated during multi-turn interaction.

Initial Prompt Construction. As shown in Figure 8, the initial prompt of DEEPPREP consists of a system prompt and a user prompt.

The system prompt contains four components:

- **Role Definition.** The model is assigned the role of a “data expert with extensive knowledge in data preparation,” establishing the task context for autonomous data preparation.
- **Operation Space.** The full set of operator definitions O is provided, including each operator’s name, parameters, and functional description. This enables the model to generate syntactically correct operator instances. We omit the operator definition in the prompt figure due to the space limit; a complete definition can be found in our published code repository³.
- **Demonstration.** Two few-shot examples are included to illustrate the interaction protocol. Each demonstration shows how to use the `<plan>` tag for analysis, the `<expand>` tag for exploration, and the `<answer>` tag for finalizing the pipeline.

³https://anonymous.4open.science/r/DeepPrep-0432/src/physicalop/data_transform.py

- **Hint.** This section specifies the interaction protocol, including: (a) the usage of `<plan>` for analyzing the current state and planning the next step, (b) the distinction between `<expand>` for exploration and `<answer>` for the final pipeline, (c) the constraint that each turn’s operator chain must be generated from scratch rather than cached from previous turns, (d) naming conventions for intermediate tables produced by Join and Union operations, and (e) the maximum number of exploration turns.

The user prompt contains two components:

- **Initial Input Tables.** The source tables are serialized into a Markdown-formatted representation. Each table is labeled with a name (e.g., `table_1`) and includes the column headers and sampled row data. For source tables, we sample up to 20 columns and 20 rows to control token usage while preserving sufficient information for the model to understand the table structure and content.
- **Target Table Schema Description.** Instead of providing the full target table content, we provide a structured schema description that includes: (a) a natural language task description explaining the transformation intent; and (b) for each target column, its name, a semantic description, and a list of requirements (e.g., “Distinct values for each row,” “Non-negative integer values”). This schema-only representation avoids leaking the target table values while providing sufficient guidance for the model to construct the correct pipeline.

Trajectory Prompt during Interaction. As shown in Figure 9, during the multi-turn interaction process, the prompt is incrementally updated with execution feedback. Specifically, after each exploration turn where the agent generates an operator chain wrapped in `<expand>` tags, the environment executes the operators and returns an `<execute>` block containing:

- **Intermediate Table Data.** The execution results of each operator in the chain are serialized. For each operator, both the operator string and the resulting table sample are shown. For intermediate tables, we sample up to 8 columns and 5 rows to further control token usage, as intermediate tables tend to be more numerous and their detailed content is less critical for high-level planning.
- **Reminder.** A reminder is appended to the observation, indicating the remaining exploration turns and prompting the agent to either continue exploring with `<expand>` or finalize the pipeline with `<answer>`.

A.2 Schema Handling

Yes, schemas of intermediate tables are included in prompts. The schema information is conveyed through two complementary mechanisms:

Explicit Schema from Source Tables. For source tables in the initial prompt, the full schema is provided, including column names, data types, and semantic descriptions. This gives the model a complete understanding of the input data structure.

Implicit Schema from Intermediate Tables. For intermediate tables produced during tree-based exploration, the schema information is implicitly conveyed through the sampled data. For example, when a GroupBy operator produces an intermediate table, the column names and data types are evident from the sampled

Role Definition

You are a data expert with extensive knowledge in data preparation. You will be given the schema description of the target table and the input tables. You need to generate the correct operators to transform the input tables to get the target table. This is a complicated task, try to solve it in multiple turns.

Operation Space

{operator_definition}

Demonstration

Demonstration 1

Demonstration 2

...

Hint

You are dealing with a complex task with multiple rounds of interaction, don't try to complete it in one step.

- Output with tags below to answer the question.

- `<think>` your_think_here `</think>`: This tag is used for analyzing what action should be taken next.

- `<operator>` your_operator_chain_here `</operator>` or `<solution>` your_operator_chain_here `</solution>`

- The operator chain are concatenated by " --> ", which will be executed by an external executor.

- Please first use the `<operator>` tag to fully explore. During exploration, you can split the complicated task into multiple sub-tasks. Thus, you only need to generate several new operators at one turn. As shown in the demonstration, we only generate part of the operator chain at one turn.

- If previous exploration results can deduce a correct operator chain to get the target table or it is the last turn, use the `<solution>` tag to summarize the correct operator chain. Output your final solution with operators used in the previous exploration. The solution must end with a Terminate operator.

- Other Tips:

- The table in the executing results will be truncated. Thus, when filtering the record, try to adopt multi-step fuzzy filtering to ensure the correctness of the filtering.

- Carefully check the generated table with the target table schema description. DO NOT output redundant columns.

- For most operators, the result table name remains the same as the source table name. For 'join' and 'union', the result table name should follow the format 'table_x_table_y_join' or 'table_x_table_y_union'.

- ****DO NOT**** Generate the `<observation>` tag! Because the execution results will be given by an external executor.

- The capabilities you should have:

- You should have the ability to effectively explore the operator space to find the correct operator chain.

- You should have the ability to correct the previous wrong operator in the new exploration.

- You should have the ability to examine the execution results to determine whether a correct target table is generated to determine whether a correct operator chain is found.

You will have {max_explore_turn} to explore. For each exploration, you should first reflect with the tag `<think>` based on the previous `<observation>`, and then use `<operator>` tag to explore. After exploration, you must output a solution with tag `<solution>`. Make sure all operators in the `<solution>` can be executed successfully.

Initial Input Tables & Target Table Schema

...

Completion

follow the Rules in "#Hint" Section.

Figure 9: The trajectory prompt template of DEEPREP during multi-turn interaction. After each exploration turn, the execution results of the operator chain are appended as `<observation>` blocks, which contain the sampled intermediate table data and a `<reminder>` tag indicating the remaining exploration turns. The agent can then analyze the observation with `<think/>` and either continue exploring with `<operator>` or output the final pipeline with `<solution>`.

rows shown in the `<execute>` block. This design choice is motivated by two considerations: (1) Including explicit schema descriptions for every intermediate table would significantly increase the

prompt length, especially for tasks with long pipelines that produce many intermediate states. (2) The sampled data provides sufficient schema information for the model to reason about subsequent operations, as column names are directly visible in the table header and data types can be inferred from the sample values.

Algorithm 4 DEEPPREP: Tree-Based Agentic Inference

Require: Source tables S and target schema Σ^*

```
1: Initialize  $G_0 = (N, E)$  with  $N = \{n_0\}$ ,  $n_0 \leftarrow S$ 
2: while the agent does not use  $\langle \text{answer} \rangle$  do
3:    $\psi_t \sim \text{PrLLM}(\cdot \mid G_{t-1}, \Sigma^*, \psi_{<t})$   $\triangleright \langle \text{plan} \rangle$ 
4:    $\mathcal{P}_t \sim \text{PrLLM}(\cdot \mid G_{t-1}, \psi_t)$   $\triangleright \langle \text{expand} \rangle$  from chosen parent
5:    $n_{\text{curr}} \leftarrow$  parent node identified by  $\psi_t$ 
6:   for each operator  $o$  in  $\mathcal{P}_t$  do
7:      $\mathcal{T}_{\text{out}} \leftarrow o(n_{\text{curr}})$   $\triangleright \langle \text{execute} \rangle$  operator on current state
8:     if execution succeeds then
9:        $n_{\text{new}} \leftarrow$  new node with state  $\mathcal{T}_{\text{out}}$ 
10:       $N \leftarrow N \cup \{n_{\text{new}}\}$ ;  $E \leftarrow E \cup \{(n_{\text{curr}} \xrightarrow{o} n_{\text{new}})\}$ 
11:       $n_{\text{curr}} \leftarrow n_{\text{new}}$ 
12:    else
13:      Annotate  $n_{\text{curr}}$  with failure trace
14:      break
15:    end if
16:  end for
17:   $G_t \leftarrow (N, E)$ 
18: end while
19:  $n_{\text{leaf}} \leftarrow \text{select}(\cdot \mid S, \Sigma^*, G_t)$ 
20:  $D^* \leftarrow n_{\text{leaf}}$ 
21: return  $D^*$ 
```

Algorithm 5 Direct ToT Adaptation for Data Preparation

Require: Source tables S and target schema Σ^*

```
1: Initialize a textual reasoning tree  $G_0 = (N, E)$  with  $N = \{n_0\}$ 
2:  $n_0 \leftarrow \text{TextSerialize}(S, \Sigma^*)$ 
3: while the search budget is not exhausted do
4:    $n_i \leftarrow \text{Select}(G_{t-1})$ 
5:    $\{z_t^{(k)}\}_{k=1}^K \sim \text{PrLLM}(\cdot \mid n_i, G_{t-1}, S, \Sigma^*)$ 
6:    $\{v_t^{(k)}\}_{k=1}^K \sim \text{PrLLM}(\cdot \mid z_t^{(k)}, S, \Sigma^*)$ 
7:    $G_t \leftarrow \text{Update}(G_{t-1}, \{z_t^{(k)}, v_t^{(k)}\}_{k=1}^K)$ 
8: end while
9:  $z^* \leftarrow \text{SelectBest}(G_T)$ 
10:  $\mathcal{P}^* \sim \text{PrLLM}(\cdot \mid z^*, S, \Sigma^*)$   $\triangleright$  translate final thought into executable pipeline
11:  $D^* \leftarrow \text{Env}(S, \mathcal{P}^*)$ 
12: return  $D^*$ 
```

Schema Derivation from Operators. Intermediate schemas are derived from the source tables and the generated operators. For example, given a source table containing a column “birth”, DEEPPREP may generate an operator `AddNewColumn(table, name, func)` to create a new column “birth-year” using an extraction function `func`. The schema of the resulting intermediate table therefore consists of the original schema plus the newly generated column. Similarly, a `Join` operator produces a table whose schema is the union of the two input tables’ schemas (with the join key appearing once), and a `SelectColumn` operator produces a table whose schema is the subset of the input schema specified by the selected columns.

A.3 Table Content Representation and Sampling Strategy

To handle large tables within the context window constraints of LLMs, we employ a differentiated sampling strategy:

- **Source tables** (in the initial prompt): We sample up to 20 columns and 20 rows. This provides the model with a comprehensive view of the input data structure and content.
- **Intermediate tables** (in the $\langle \text{execute} \rangle$ blocks): We sample up to 8 columns and 5 rows. Since intermediate tables are produced at each exploration turn and multiple operators may be executed per turn, a more aggressive sampling strategy is necessary to prevent context overflow. The reduced sample size still preserves the essential schema and value distribution information needed for the model to evaluate the correctness of its operations.
- **Truncation indicators.** When the number of columns exceeds the sampling limit, we truncate the remaining columns and append “...” to indicate the truncation. Similarly, when the number of rows exceeds the limit, we append “.....” after the last sampled row.

Impact on Performance. As discussed in Section 6.4 (Scalability Analysis), this sampling strategy may cause performance degradation for very large tables. Specifically, for cases containing 36 to 124,842 rows, accuracy decreases by 29.54 points compared with cases containing 1 to 9 rows. This degradation is primarily caused by the table sampling strategy used to control token consumption, where relevant information may be omitted from the sampled content for very large tables. We leave more sophisticated sampling strategies for large tables as future work.

A.4 State Representation

As defined in Section 4.1, a state in the agentic reasoning tree is represented by the *current working set of tables* produced during data preparation. We provide a concrete example to illustrate how states evolve as operators are applied.

EXAMPLE 5. Consider a data preparation task with the following sequence of operations:

- (1) Initial state n_0 : The environment contains a single source table A .
- (2) An operator splits table A into tables B and C . The new state n_1 consists of $\{B, C\}$.
- (3) An operator transforms table B into table D . The new state n_2 consists of $\{D, C\}$.

In this process, newly generated tables replace their source tables in the state representation. Specifically, when an operator transforms a table X into a new table X' , the state is updated by removing X and adding X' . Tables that are not affected by the current operator remain unchanged in the state.

This state representation ensures that each node in the agentic reasoning tree captures the complete set of tables available at that point in the pipeline, enabling the agent to reason about the current data state and plan subsequent operations.

A.5 Failed Branch Reuse

During tree-based exploration, failed branches are preserved with their error traces and reused to inform future exploration. Specifically, when a branch produces an execution error, the following information is retained:

- **Erroneous operator.** The operator that caused the execution failure is recorded.

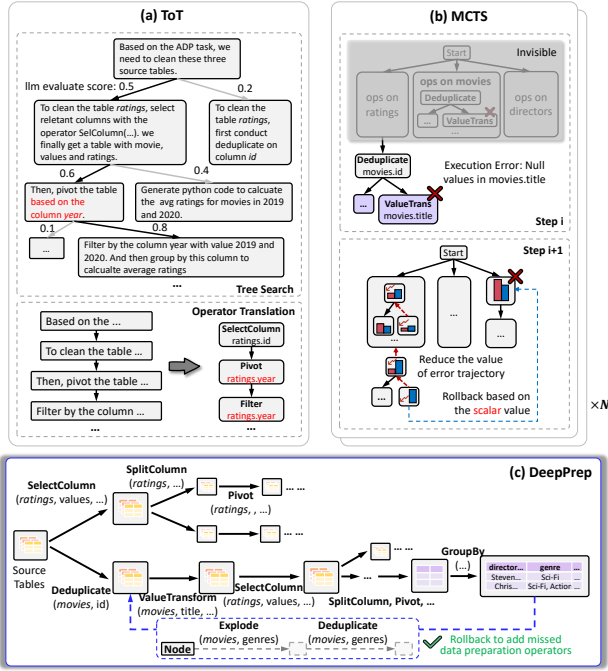


Figure 10: Comparison of ToT, MCTS and DEEPPREP.

Algorithm 6 Direct MCTS Adaptation for Data Preparation

Require: Source tables $\mathcal{S}$ and target schema Σ^*

```

1: Initialize a search tree  $G_0 = (N, E)$  with  $N = \{n_0\}$ ,  $n_0 \leftarrow \mathcal{S}$ 
2: Initialize node scores  $Q(n_0) \leftarrow 0$ 
3: while the search budget is not exhausted do
4:    $n_i \leftarrow \text{Select}(G_{t-1}, Q)$ 
5:    $\{z_t^{(k)}\}_{k=1}^K \sim \text{Pr}_{\text{LLM}}(\cdot \mid n_i, G_{t-1}, \mathcal{S}, \Sigma^*)$ 
6:    $n_j \leftarrow \text{Expand}(G_{t-1}, n_i, z_t^{(1)})$ 
7:    $Q(n_j) \leftarrow 0$ 
8:    $\{z_t^{(l)}\}_{l=1}^L \sim \text{Pr}_{\text{LLM}}(\cdot \mid n_j, \mathcal{S}, \Sigma^*)$ 
9:    $\{\mathcal{T}^{(l)}\}_{l=1}^L \leftarrow \text{Env}(n_j, \{z_t^{(l)}\}_{l=1}^L)$ 
10:   $r \leftarrow \text{Evaluate}(\{\mathcal{T}^{(l)}\}_{l=1}^L, \Sigma^*)$ 
11:  for each ancestor  $n_a$  of  $n_j$  in  $G_t$  do
12:     $Q(n_a) \leftarrow Q(n_a) + r$ 
13:  end for
14:   $G_t \leftarrow G_{t-1}$ 
15: end while
16:  $z^* \leftarrow \arg \max_{n \in N} Q(n)$ 
17:  $\mathcal{P}^* \sim \text{Pr}_{\text{LLM}}(\cdot \mid z^*, \mathcal{S}, \Sigma^*)$ 
18:  $D^* \leftarrow \text{Env}(\mathcal{S}, \mathcal{P}^*)$ 
19: return  $D^*$ 

```

- **Execution error trace.** The runtime error message (e.g., `ValueError`, `KeyError`) is captured and stored as a failure annotation on the corresponding node.

These failure annotations are serialized into the prompt for subsequent reasoning steps, enabling the agent to recognize and avoid repeating the same errors when generating new branches. This mechanism is a key benefit of the tree structure over linear reasoning: it transforms failures into informative signals that guide more effective exploration, rather than treating them as dead ends.

EXAMPLE 6. Consider the reasoning trajectory in Example 4. In Step 2, the generated ExeCode operator fails because it references a non-existent column “year” in table ratings. DEEPPREP records the corresponding error trace and appends it to the prompt. In the next exploration step, the agent analyzes the error and generates a revised operator that preserves the required column when applying SelectColumn, thereby correcting the failure. Without the failure annotation, the agent might repeat the same mistake in subsequent branches.

A.6 Algorithm Pseudo-Code

We present the pseudo-code of DEEPPREP’s tree-based agentic inference in Algorithm 4. The algorithm formalizes the iterative process of planning, expanding, and executing over the agentic reasoning tree.

Complexity Analysis. The search process is bounded by two configurable parameters: the maximum exploration turn T and the maximum operator sequence length L per expansion. Therefore, the worst-case search complexity is $O(T \times L)$. Unlike classical tree-search methods such as MCTS, which explicitly expand and evaluate many candidate branches, DEEPPREP generates only one operator pipeline conditioned on the current tree state at each step. The “pruning” is implicit in the LLM’s decision-making: the agent observes failure traces and avoids revisiting failed branches, rather than enumerating all possible expansions. Consequently, the effective branching factor depends on the LLM’s learned policy and remains bounded regardless of input size.

Comparison with ToT and MCTS. To further clarify the novelty of DEEPPREP’s search strategy, we also provide the pseudo-code of Tree-of-Thought (ToT) and MCTS adapted for data preparation in Algorithm 5 and Algorithm 6, respectively. The key differences are:

- **Execution grounding.** Unlike ToT (Algorithm 5), which operates over textual reasoning states and invokes the environment only once at the end, DEEPPREP executes every operator on real data and grounds search decisions in materialized intermediate states.
- **Failure-aware exploration.** Unlike MCTS (Algorithm 6), which propagates only scalar rewards through backpropagation, DEEPPREP preserves structured error traces and reuses them to guide subsequent exploration, enabling the agent to reason about *why* a branch failed rather than simply penalizing it.
- **Generative tree reasoning.** Unlike MCTS, which relies on rollout-based value estimation for node selection, DEEPPREP reasons over the entire search tree—including intermediate tables, execution results, and failure traces—to generate subsequent actions.

Figure 10 illustrates a concrete comparison. In Figure 10(a), ToT decides to apply a Pivot operator using the column year, although this column has not been generated yet. Because no execution feedback is available during search, the error is discovered only when the final pipeline is executed. In Figure 10(b), MCTS encounters an execution error due to null values in movies.title during ValueTransform, but the failure is propagated merely as a scalar penalty, without preserving the semantic cause. In Figure 10(c), DEEPPREP executes operators and observes the resulting intermediate tables. When execution reveals missing data preparation steps, the agent analyzes the failure trace and rolls back to a valid state, inserting the missing operators before continuing exploration.

```

# Role Definition

You are a data expert with extensive knowledge in table transformations. You will be given the
schema description of the target table and the input tables. You need to generate the correct
operators to transform the input tables to get the target table.

# Operator Space

{operator_definition}

# Demonstration
## Demonstration 1
## Demonstration 2
...
# Hint

Task Definition:

- You can use two types of TAGS to complete the task.
  - <think> your_think_here </think>: This tag is used for analyzing what action should be taken
  next based on the current status and historical information.
  - <solution> your_solution_here </solution>: Use this tag to output your operator chain as a
  solution. The solution must end with a Terminate operator to output a table as the target table.
- An **external** executor will parse your operator chain, execute each operator sequentially, and
  return the results after you generate a solution.
  - The executing results will be wrapped by the tag <observation> your_observation_here
  </observation>.
  - The table in the executing results will be truncated to 3 rows.
  - Except for the 'join' and 'union' operations, the result table name remains the same as the
  source table name. For 'join' and 'union', the result table name should follow the format
  'table_x_table_y_join' or 'table_x_table_y_union'. This information will be used to generate a
  correct operator argument.
- You will have 0 turns to reflect and refine your solution. For each turn, you should first analyze
  by using the tag <think> based on the previous <observation> and previous <reflect>, then use
  the <solution> tag to propose or revise your operator chain. You should not generate <reflect>
  tag since it is generated by another agent.

# Initial Input Tables & Target Table Schema
...

# Completion

Remember to follow the format in "#Hint" Section.

```

Figure 12: Prompt of PlanAndAction.

```

# Role Definition

You are a data expert with extensive knowledge in table transformations. Please call the function
based on input tables and target table schema description. Output one operator at one time.

# Operator Space

{operator_definition}

# Demonstration
## Demonstration 1
## Demonstration 2
...
# Hint

Task Definition:

- You can use two types of TAGS to complete the task.
  - <think> your_think_here </think>: This tag is used for analyzing what action should be taken
  next based on the current status and historical information.
  - <operator> your_operator_here </operator>: Use this tag to output ONLY **1** operator. If
  the task is completed, output the operator 'Terminate'.
- An **external** executor will execute your operator and the executing results will be wrapped
  by the tag <observation> your_observation_here </observation>.
  - The table in the executing results will be truncated in 3 rows.
  - Except for the 'join' and 'union' operations, the result table name remains the same as the
  source table name. For 'join' and 'union', the result table name should follow the format
  'table_x_table_y_join' or 'table_x_table_y_union'. This information will be used to generate a
  correct operator argument.
  - You **SHOULD NOT** generate the observations, since it will be returned by an external
  executor.
- You need to complete the task within {max_explore_turn} turns. Thus, you will have
  {max_explore_turn}-1 to explore. For each exploration, you should first analyze by using the
  tag <think> based on the previous <observation>, and then use <operator> tag to output the
  next operator.

# Initial Input Tables & Target Table Schema
...

# Completion

Complete this task WITHIN {max_explore_turn} turns. Currently, turn {cur_turn}. Generate
**One** operator at this turn. Remember to follow the Rules in "#Hint" Section.

```

Figure 13: Prompt of ReAct.

```

# Role Definition

You are a Code Agent to generate python code to transform the input tables into the target tables.

# Demonstration
## Demonstration 1
## Demonstration 2
...
# Hint

- You are allowed to use two tags to generate code to complete the task in multiple turns.
- In each turn, you can use:
  - <think> your_think_here </think>: This tag is used for analyzing what action should be taken
  next based on the current status and historical information.
  - <python> your_python_code_here </python>: This tag is used to output the python code you
  generated in the current turn. When generating the python code, you should follow the following
  rules:
    - The input tables will be provided in the dict object 'input_tables'. You can access each table
    by the key (which is the name of the table) in the dict. For example, you can access the table
    'table_1' by 'input_tables['table_1']'.
    - Your generated target table should be assigned to a pd.DataFrame object named 'target_df'.
    - <end> END </end>: If the task can be completed by the code, append this tag after </python>
    tag.
  - After generating the python code in each turn, an external executor will execute the code and
  display the target table 'target_df'.

CRITICAL REQUIREMENTS:
1. The output table MUST match EXACTLY the schema described in "Target Table Schema
Description".
2. Column names MUST be identical (case-sensitive) to the target schema.
3. Do NOT add extra columns. Do NOT omit any required columns.
4. Data types and formats MUST match the target schema.
5. Before finishing, VERIFY your result matches the target schema exactly.

# Initial Input Tables & Target Table Schema
...

# Completion

Generate your output for turn 1:

```

Figure 14: Prompt of CodeGen baseline.

```

# Role Definition

You have access to the following logical operators:
{logical_op_registry}
Your task is to generate the next logical operator to tackle the task.

# Demonstration
## Demonstration 1
## Demonstration 2
...
# Hint

- You must output the logical operator wrapped in XML tags: <logical_operator> ...
  </logical_operator>.
- Do not output Python code directly. Only output **ONE** next logical step.
- Output **one** logical operator at a time, the logical operator will be passed to the programmer
  agent to implement and implemented results will be passed back in the <observation> ...
  </observation> tags.

# Initial Input Tables & Target Table Schema
...

# Completion

Please generate the next logical operator:

```

Figure 15: Prompt of AutoPrep Planner.

```

# Role Definition

You are a Code Agent to generate python code to transform the input tables into the target tables.

# Demonstration
## Demonstration 1
## Demonstration 2
...
# Hint

- The input tables are truncated and will be provided in the object 'input_tables'.
- Carefully check the generated table with the target table schema description.
- Wrapped your generated code with ```your_code_here``` with NO OTHER TEXT.
- Your generated target table will be maintained in a pd.DataFrame object named 'target_df'.

# Initial Input Tables & Target Table Schema
...

# Completion

Generate the python code wrapped by ```. Remember to follow the format in "#Hint" Section.

```

Figure 11: Prompt of CodeGen.

```

# Role Definition
You are a data expert with extensive knowledge in table transformations. Your goal is to generate a complete operator chain from the current state to a terminal state to solve the given task.

# Operator Space
{operator_definition}

# Demonstration
## Demonstration 1
## Demonstration 2
...
# Hint
- You should generate a plausible operator chain to complete the task.
- The operator chain should start from the current state.
- The operator chain must end with a 'Terminate' operator.
- You can use two types of TAGS to complete the task.
  - <think> your_think_here </think>: This tag is used for analyzing what action should be taken next based on the current status and historical information.
  - <operator> your_operator_chain_here </operator>
  - You should generate a plausible operator chain to complete the task.
  - The operator chain should start from the current state.
  - The operator chain must end with a 'Terminate' operator.

# Initial Input Tables & Target Table Schema
...
# History Operator Chains and Outputs
...

# Completion
Based on the historical operations and the current state, generate all possible next operators.

```

Figure 18: Prompt of MCTS-OP Simulation.

```

# Initial Input Tables & Target Table Schema
...
# History Operator Chains and Outputs
...
# Final Output Observation
...

# Completion
Review the operator chain and the final output. Determine if the solution path correctly transforms the initial data to solve the problem.

- If the operator chain is logical and the final output is correct, respond with only '[Correct]'.
- If the operator chain is flawed or the final output is incorrect, respond with only '[Incorrect]'.
- Do not provide any explanations or suggestions.

Output:

```

Figure 19: Prompt of MCTS-OP Reward.

```

# Role Definition
You are a Data Cleaning Programmer. Your goal is to translate a specific data cleaning requirement into a sequence of Physical Operators.

# Operator Space
{operator_definition}

# Demonstration
## Demonstration 1
## Demonstration 2
...
# Hint
- Output the plan wrapped in XML tags: <physical_operator> ... </physical_operator>.
- Use '→' to represent the execution order if multiple operators are required.
- Do not output Python code directly unless it's inside the 'func' argument of an operator.
# Initial Input Tables & Target Table Schema
...

# Completion
Please generate the physical operators:

```

Figure 16: Prompt of AutoPrep Programmer.

```

# Role Definition
You are an expert in data science and table transformation. Your task is to expand the current step by generating all possible next-step operators based on the given context.

# Operator Space
{operator_definition}

# Demonstration
## Demonstration 1
## Demonstration 2
...
# Hint
- You should generate all possible operators for the next step.
- The operators you generate should be valid and reasonable based on the current state.
- Use the provided operation space to construct the operators.
- Output the operators in a list within the '<operators>' tag, and use [SPLIT_FLAG] to separate each operator.
- For example:
  <operators>
  Operator1(...)
  [SPLIT_FLAG]
  Operator2(...)
  </operators>
- Do not generate any other text or explanations.

# Initial Input Tables & Target Table Schema
...
# Historical operations
...

# Completion
Based on the historical operations and the current state, generate all possible next operators.

```

Figure 17: Prompt of MCTS-OP Expand.

B IMPLEMENTATION DETAILS OF BASELINES

In this section, we provide the implementation details of all baselines compared in our experiments, including their prompt templates, configuration choices, and the comparison with agentic coding systems.

B.1 Prompt Templates of Baselines

We present the prompt templates used by each baseline. All methods share the same prompt structure consisting of Role, Demonstration, and Hint sections. For operator-assisted methods, an additional Operator Space section is included. All methods use the same ADP demonstration cases for In-Context Learning. We omit the operator definition and demonstration examples in the prompt figures due to the space limit.

CodeGen. As shown in Figure 11, this method treats the data preparation task as a standard programming problem. The prompt defines the model’s role as a Code Agent, responsible for generating Python code to transform source tables into a target table. To ensure the generated code is executable and evaluable, the prompt imposes strict formatting constraints in the “Hint” section. Specifically, the model is required to wrap the code within triple backticks and ensure the final result is stored in a specific `pd.DataFrame` object named `target_df`. At each turn, the execution result of the generated Python code is appended to the prompt, serving as a signal to revise the code at the next turn. The agent iterates until exceeding the maximum turn or actively ending the task with an `(end)` tag.

Plan-and-Solve (PaS). As shown in Figure 12, this method restricts the model to a predefined set of operator types to improve controllability. The prompt defines the model as a “data expert” and provides a specific `operator_definition` space. To facilitate

reasoning and interaction with the environment, the prompt instructs the model to use specific XML tags: (1) `<think>`: Used for analyzing the current execution state and planning the next step; (2) `<solution>`: Used to output the operator sequence for the current step; (3) `<observation>`: A placeholder for the execution feedback returned by the environment. Furthermore, the prompt specifies naming conventions for intermediate tables (especially for Join and Union operations) to ensure the operator parameters remain consistent across steps.

ReAct. As illustrated in Figure 13, this method adopts a linear ReAct-style interaction loop to solve the task. The prompt limits the model to generate exactly one operator instance per turn using the `<operator>` tag, preceded by a specific analysis step using the `<think>` tag. Unlike the single-pass methods, ReAct is explicitly aware of a maximum exploration turns limit (`max_explore_turn`), encouraging it to dynamically adjust its subsequent decisions based on the execution feedback (`<observation>`) returned by the environment. Similar to Plan-and-Solve, it also enforces strict naming conventions for Join and Union operations to maintain context across the multi-turn interaction.

DeepAnalyze. Figure 14 presents the prompt for the DeepAnalyze baseline. In contrast to the standard CodeGen method, this approach allows the model to interact with the environment over multiple turns. The prompt instructs the model to use `<think>` for analyzing the current execution state and `<python>` to output data processing programs for the current turn. Crucially, the prompt includes a “CRITICAL REQUIREMENTS” section, which explicitly emphasizes strict schema alignment (e.g., case-sensitive column names, exact data types) to guide the model in self-correction before finalizing the task with the `<end>` tag.

AutoPrep. We implement AutoPrep using a hierarchical multi-agent framework, consisting of a Planner and several Programmers. (1) **AutoPrep Planner.** As shown in Figure 15, the Planner operates at a high level. It is provided with a registry of logical operators (`<logical_op_registry>`) and is tasked with generating the next abstract logical plan (e.g., “clean data”) wrapped in `<logical_operator>` tags. It does not write code or specify operator parameters. (2) **AutoPrep Programmer.** As shown in Figure 16, the Programmer receives the logical plan and translates it into a data preparation pipeline of executable operators. It has access to the full set of supported data preparation operators and uses the `->` symbol to represent the application order of multiple operators (e.g., `SplitColumn -> RenameColumn`) required to fulfill a single logical intent. Since AutoPrep was originally designed for TableQA rather than autonomous data preparation, we replace its original prompt with an ADP-oriented prompt and apply the same tuning procedure as DEEPREP.

MCTS-OP. We implement a Monte Carlo Tree Search (MCTS) baseline adapted for operator generation. This method decomposes the reasoning process into three distinct phases, each governed by a specific prompt:

(1) **Expansion:** As shown in Figure 17, this prompt is used to grow the search tree. The model is instructed to analyze the current history and generate a list of “all possible next-step operators”

that are valid and reasonable. To facilitate parsing by the search algorithm, the model must output each candidate operator in a structured format within `<op_i>` tags, where i is the candidate index.

(2) **Simulation (Rollout):** As shown in Figure 18, this prompt is used to estimate the value of a node by completing a partial pipeline. Given the current operator history, the model generates a continuation of the operator chain that leads to the target table. The prompt instructs the model to output the remaining operators within a `<completion>` tag.

(3) **Reward:** As shown in Figure 19, this prompt evaluates the quality of a completed pipeline. The model is asked to assess the generated operator chain by comparing the resulting table with the target schema description. It outputs a numerical score within a `<score>` tag, which is used as the node value for backpropagation in the MCTS algorithm.

MontePrep. MontePrep [11] applies Monte Carlo Tree Search and performs tree search over actions such as schema mapping, operator discovery, code synthesis, and refinement instead of individual operators. We use the authors’ released code with default settings.

TRAE. TRAE is an open-source agentic coding system developed by ByteDance⁴, which has shown strong performance on SWE-bench [19]. We evaluate it as a representative of state-of-the-art agentic coding systems.

Claude Code (CC). Claude Code is a commercial AI-powered IDE that integrates agentic coding capabilities with planning, code generation, execution, and iterative refinement loops. We install Claude Code v2.1.119 and evaluate it using the Claude-4 backbone, as required by its framework.

B.2 Configuration and Tuning Details

To ensure fair and reproducible comparisons, we apply a consistent configuration and tuning methodology across all baselines.

Methods with Publicly Available Implementations. For methods with publicly available implementations, including MontePrep [11], DeepAnalyze [45], AutoPrep [9], and MCTS-OP [24], we use their official codebases and default configurations (e.g., prompt design, parameters, etc.).

Methods Requiring Reimplementation. For methods without publicly available implementations, including CodeGen, ReAct, and PaS, we reimplemented them following the descriptions in their papers and applied the same prompt-tuning and parameter-selection procedure used for DEEPREP. Therefore, all methods were evaluated under a comparable tuning method.

Methods Requiring Adaptation. When a method was originally designed for a different task and could not be directly applied to ADP, we adapted its prompt to the ADP setting and tuned its parameters using the same grid-search procedure adopted by DEEPREP. For example, AutoPrep was originally designed for TableQA rather than autonomous data preparation, so we replaced its original prompt with an ADP-oriented prompt and tuned it accordingly.

Shared Configuration. All methods share the following configuration:

- **Prompt structure.** All methods share the same prompt structure consisting of Role, Demonstration, and Hint sections. For

⁴<https://github.com/bytedance/trae-agent>

Table 7: Overview of the Operator Space in DEEPPREP.

Category	Operators
Cell-level	ValueTransform, StandardizeDatetime, CastType
Row-level	Filter, Sort, TopK, Deduplicate DropNA, MissingValueImputation, ErrorDetection OutlierDetection
Column-level	SplitColumn, RenameColumn, AddNewColumn, DropColumn Concatenate, SelectColumn, Subtitle
Table-level	Pivot, Stack, WideToLong, Transpose, Explode Join, Union, Append
Aggregation	GroupBy, Count, CalculateStatistic
Escape Hatch	ExeCode

Table 6: Comparison between DEEPPREP and Agentic Coding Systems.

LLM	Method	Synth-Spider		Synth-Bird		Parrot	
		Acc.	Cost (\$)	Acc.	Cost (\$)	Acc.	Cost (\$)
GPT-5	TRAE	72.36	0.038	61.75	0.033	41.19	0.041
	DEEPPREP	71.76	0.014	60.46	0.015	41.33	0.009
Claude-4	CC	72.71	0.169	68.33	0.141	42.98	0.188
	TRAE	73.80	0.148	66.93	0.160	41.93	0.125
	DEEPPREP	73.46	0.054	67.43	0.062	46.07	0.044

operator-assisted methods, an additional Operator Space section is included.

- **Demonstration.** All methods use the same ADP demonstration cases for In-Context Learning.
- **Serialization.** All methods share the same serialization function and hyperparameters (maximum column number and row number) for table serialization.
- **Temperature.** For all prompting methods, the temperature is set to 0.01 to ensure reproducibility.
- **Maximum exploration turns.** The maximum exploration turn is set to 5 for methods that require specifying this hyperparameter, including DEEPPREP, CodeGen, and DeepAnalyze.
- **Training baselines.** Training baselines (IL, RL-GRPO) use the same backbone models and hyperparameters as DEEPPREP, differing only in the training strategy. This isolates the effect of our progressive agentic training.

B.3 Comparison with Agentic Coding Systems

Following the suggestion of Reviewer #1, we compare DEEPPREP with two strong agentic coding systems: Claude Code (CC) and TRAE. These systems represent state-of-the-art general-purpose coding agents that implement planning–coding–testing loops with environment feedback.

Table 6 reports the comparison results. DEEPPREP achieves accuracy comparable to CC and TRAE across all benchmarks while *consistently reducing cost*. With GPT-5, DEEPPREP achieves accuracy comparable to TRAE while reducing cost by 2.7× on Synth-Spider, 2.2× on Synth-Bird, and 4.6× on Parrot. With Claude-4, DEEPPREP achieves accuracy comparable to both CC and TRAE, while achieving the best accuracy on Parrot and reducing cost by 2.7×–4.3×.

Why DEEPPREP is More Cost-Effective. The cost advantage of DEEPPREP over general-purpose coding agents stems from two aspects:

- **Clearer intermediate evaluation.** Since each operator produces a well-defined intermediate table, the agent can precisely diagnose where and why a pipeline fails based on execution

feedback, avoiding unnecessary debugging iterations that free-form code agents often spend on locating bugs in long code blocks.

- **Reduced output token consumption.** When backtracking to a prior state, DEEPPREP only needs to generate the delta operator sequence from the parent node, whereas coding agents must re-generate the entire code block from scratch, including all previously correct logic, leading to redundant token consumption.

C OPERATOR SPACE DETAILS

In this section, we provide the complete definitions of all 31 operators supported by DEEPPREP (Section 2.2), along with analyses of operator pipeline length distribution and operator type distribution across datasets.

C.1 Complete Operator Definitions

As outlined in Table 7, the operators are categorized into four groups: Data Cleaning, Column Transformation, Table Transformation, and Other. We detail the functionality and parameters for each operator below.

Operators for Data Cleaning. These operators focus on improving data quality by handling errors, missing values, and inconsistencies.

(1) **StandardizeString.** This function standardizes the string format within a specific column. It accepts two arguments: column, representing the target column name, and func, a lambda function defining the transformation logic. For example, to remove whitespace and convert the City column to title case, we set column to City and func to lambda x: x.strip().title().

(2) **ErrorDetection.** This function identifies erroneous values based on specific logic. It takes column and func as arguments, where func returns a boolean indicating validity. For instance, to detect invalid email addresses, func could be a regex match function lambda x: not re.match(r"[^@]+@[^@]+\.[^@]+", x).

(3) **DropNA.** This function removes rows containing missing values. It requires subset, a list of columns to inspect for null values, and how, which specifies the condition (e.g., 'any' or 'all'). For example, to drop rows where both Revenue and Cost are missing, we set subset to ['Revenue', 'Cost'] and how to 'all'.

(4) **MVI (Missing Value Imputation).** This function fills missing values using a specified strategy. It takes column and mode as arguments. The mode can be a statistical method (e.g., 'mean', 'median') or a constant value. For example, to fill missing values in the Age column with the average age, we set column to Age and mode to 'mean'.

(5) **OutlierDetection.** This function handles statistical outliers. It accepts column, mode (e.g., 'z-score', 'iqr'), and action (e.g., 'clip', 'drop'). For instance, to cap values in the Price column that exceed 3 standard deviations, we set mode to 'z-score' and action to 'clip'.

(6) **StandardizeDatetime.** This function converts a column to a standard datetime object. It requires column and date-format. For example, to parse dates like "2023/12/25" in the Date column, we set date-format to "%Y/%m/%d".

(7) **Deduplicate.** This function removes duplicate rows. It takes subset (columns to consider for identifying duplicates) and keep (which duplicate to retain, e.g., 'first', 'last', or 'none').

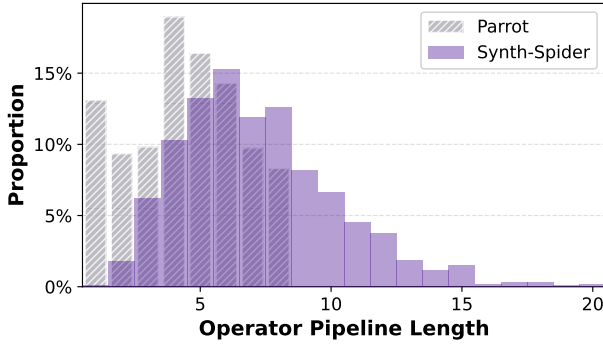


Figure 20: Operator Pipeline Length Distribution.

Operators for Column Transformation. These operators modify the schema by adding, removing, or restructuring columns.

(1) **AddNewColumn.** This function creates a new column based on row-wise calculation. It takes a single argument *func*, a lambda function accepting a row dictionary. For example, to create a Profit column, we use `lambda row: row['Revenue'] - row['Cost']`.

(2) **Cast.** This function converts the data type of a column. Its arguments are *column* and *dtype* (e.g., 'int', 'float', 'str'). For example, to convert a Year column stored as text to integers, we set *dtype* to 'int'.

(3) **Explode.** This function transforms each element of a list-like string into a row, replicating index values. It takes *column* and *split-comma* (the separator). As shown in Example ??, to split "Sci-Fi/Action" in the Genre column into separate rows, we set *split-comma* to '/'.

(4) **RenameColumn.** This function renames existing columns. It accepts *rename-map*, a dictionary mapping old names to new names. e.g., {'Yr': 'Year', 'Pop': 'Population'}.

(5) **DropColumn.** This function removes columns from the table. It takes *drop-columns*, a list of column names to delete.

(6) **SplitColumn.** This function splits a single column into multiple columns based on a pattern. It requires *src-column* and *func*. For example, to extract the numeric rating from a string "8.5/10", we set *func* to `lambda x: x.split('/')[0]`.

(7) **Concatenate.** This function combines multiple columns into one string column. It takes *columns* (list of source columns) and *func* (formatting logic). For example, to combine First and Last names, *func* could be `lambda x: f'{x['First']} {x['Last']}`.

Operators for Table Transformation. These operators restructure the table shape or aggregate data.

(1) **Join.** This function merges two tables. Arguments include *left* and *right* (table names), *on* (join keys), and *how* (type of join: 'inner', 'left', 'outer').

(2) **Union / Append.** These functions combine tables vertically. *Union* takes a list of tables and a *how* parameter (e.g., set logic), while *Append* specifically adds appended-table to the end of *src-table*.

(3) **Pivot.** This function reshapes data from long to wide format. It requires *index*, *columns* (keys to become new headers), and *values*. For example, pivoting a sales table to show months as columns.

(4) **Transpose.** This function swaps rows and columns of the table. It takes no parameters.

(5) **Stack / WideToLong.** These functions reshape data from wide to long format. *Stack* takes *id-vars* (columns to keep) and *value-vars* (columns to unpivot). *WideToLong* handles more complex patterns with suffix extraction defined by subnames, *i*, *j*, and *sep*.

(6) **Subtitle.** This function is used to handle hierarchical headers often found in spreadsheets. It takes *title* to identify the main header row and propagates it to sub-headers.

(7) **Sort.** This function sorts the table rows. It takes *by* (column names) and *ascending* (boolean).

(8) **GroupBy.** This function groups data and applies aggregation. It requires *by* (grouping columns) and *agg* (a dictionary mapping columns to functions, e.g., {'Profit': 'mean'}).

(9) **TopK.** This function retains the top-*k* rows based on a specific order. It takes the integer argument *k*.

(10) **Filter.** This function filters rows based on a condition. It takes *func*, a lambda function returning a boolean for each row. e.g., `lambda row: row['Year'] > 2000`.

(11) **SelectColumn.** This function selects a subset of columns to keep, discarding the rest. It takes *columns* as a list.

(12) **Count.** This function counts the number of records, optionally grouped by unique values. It takes *table-name*.

(13) **CalculateStatistic.** This function computes statistics across tables or complex aggregations. It takes *tables*, *var-name*, and *func*.

Other Operators. These operators handle special cases and flow control.

(1) **CodeGeneration.** This function allows the agent to generate arbitrary Python code for tasks not covered by the predefined operators. It takes *tables* and the generated code string. This ensures DEEPPREP can handle long-tail requirements.

(2) **Terminate.** This operator signals the end of the reasoning process. It takes *output-tables* to specify the final result to be returned to the user.

In total, we define **31** distinct operators, providing comprehensive coverage for diverse data cleaning and transformation tasks.

C.2 Operator Pipeline Length Distribution

Figure 20 shows the distribution of operator pipeline lengths across Synth-Spider, Synth-Bird, and Parrot. Synth-Spider and Synth-Bird share a similar pipeline length distribution, with the majority of pipelines containing 3–7 operators. In contrast, Parrot exhibits a noticeably different distribution, with pipelines tending to be shorter and less varied in length. This difference in pipeline length distribution partially explains the transfer performance gap observed in Exp-5.

C.3 Operator Type Distribution

Figures 21 and 22 show the operator type distributions on Synth-Spider and Synth-Bird, respectively. As discussed in Exp-5, the operator-type distribution shift is the primary factor explaining transfer performance, rather than pipeline length distribution. We provide a finer-grained analysis of operator coverage and compositional diversity:

Operator Coverage. Our Synth-Spider dataset provides broader operator coverage than Parrot in three aspects. First, as shown in Table 1, Synth-Spider supports 14 additional types of operators

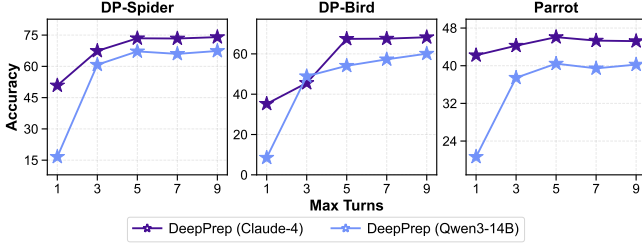


Figure 23: Accuracy of DEEPPREP when changing the maximum number of exploration turns.

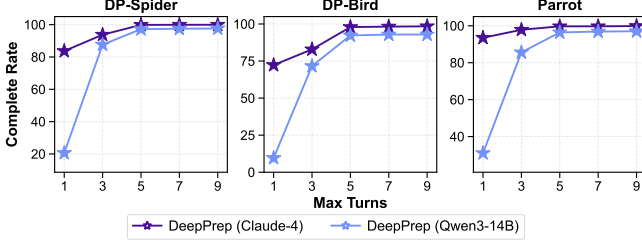


Figure 24: Completion rate of DEEPPREP when changing the maximum number of exploration turns.

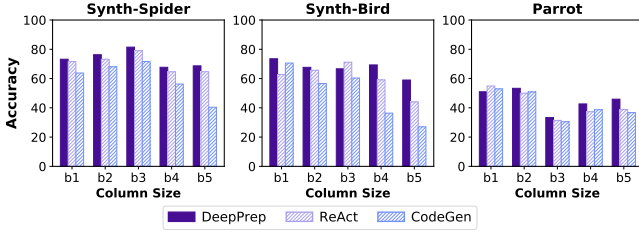


Figure 25: Accuracy comparison on tables with varying number of columns. For Synth-Spider, b1 (1-4), b2 (5-7), b3 (8-9), b4 (10-14), b5 (15-124,843). For Synth-Bird, b1 (1-11), b2 (12-27), b3 (29-74), b4 (75-1,003), b5 (1,004-20,015). For Parrot, b1 (2-4), b2 (5-7), b3 (8-11), b4 (12-16), b5 (18-587).

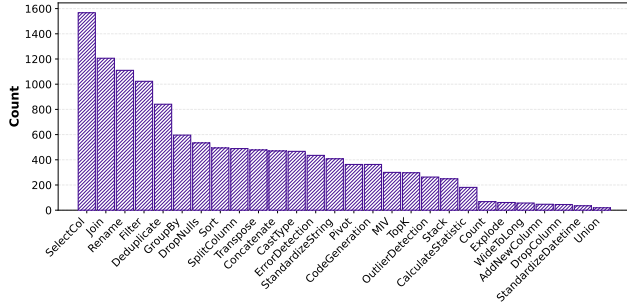


Figure 21: Operator Count Distribution on Synth-Spider.

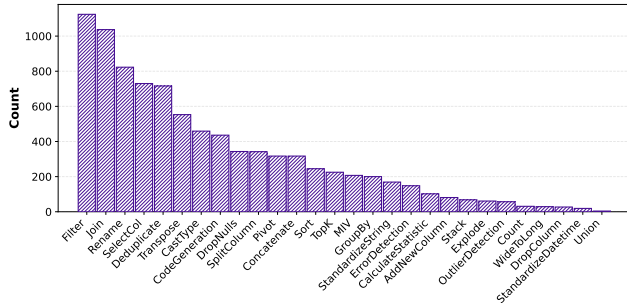


Figure 22: Operator Count Distribution on Synth-Bird.

compared with Parrot, including commonly used data cleaning operations such as Missing Value Imputation, as well as common schema editing operations such as Concatenation. Second, our operators support function arguments, enabling more customized data preparation operations. For example, AddNewColumn can take a lambda expression as an argument to define a customized mapping from source columns to the newly created column. Third, we include highly flexible data preparation operators such as ExeCode. When a data preparation requirement cannot be covered by the currently defined operator space, ExeCode allows the model to complete the operation by writing executable code.

Operator Distribution Similarity. The unigram Jensen-Shannon Divergence (JSD) between Synth-Spider and Synth-Bird is only 0.025, with a cosine similarity of 0.952, indicating nearly identical distributions. In contrast, the JSD between Synth-Bird and Parrot is 0.264 (**10.6× larger**), and the bigram and trigram JSDs reach 0.660 and 0.886 respectively, indicating nearly disjoint compositional patterns.

Compositional Diversity. Synth-Spider and Synth-Bird produce 496 and 417 unique bigrams (2,875 and 1,947 unique trigrams), while Parrot yields only 189 and 758 respectively. Synth-Spider and Synth-Bird also exhibit higher cross-category bigram ratios (0.799/0.765 vs. Parrot’s 0.708), meaning their operator combinations span more diverse functional categories. Synth-Spider’s top-5 and top-10 bigram concentration ratios are also lower than Parrot’s (0.226 and 0.326 vs. 0.248 and 0.361), indicating that its operator distribution is less dominated by a few frequent patterns.

D SUPPLEMENTARY EXPERIMENTAL RESULTS

In this section, we provide supplementary experimental results, including sensitivity analyses, time and memory usage, data size bucket details, pipeline length and table number analyses, error analysis on real-world data, model size scaling results, and training hyperparameters.

D.1 Sensitivity Analysis: Exploration Turns

Figure 23 and Figure 24 show the impact of the maximum number of exploration turns on accuracy and completion rate, respectively. Overall, the performance of DEEPPREP improves as the exploration turn increases and stabilizes at around five turns across all datasets and model backbones. For example, on Claude-4, increasing the budget from one to five turns improves accuracy by 22.63 points on Synth-Spider and 32.21 points on Synth-Bird. Beyond five turns, the additional gain is less than one point. This indicates that DEEPPREP can solve most ADP cases within five exploration turns. As illustrated in Figure 24, DEEPPREP successfully completes the ADP task at nearly 100% completion rate with five turns.

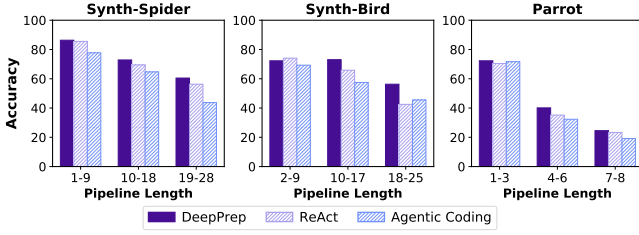


Figure 27: Comparison on Accuracy with varying Pipeline Length.

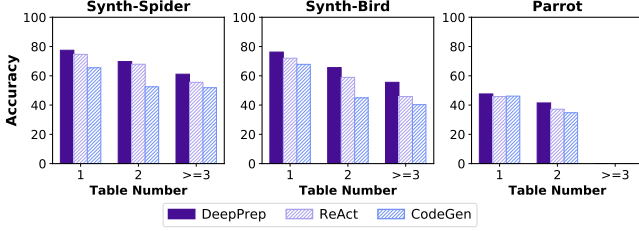


Figure 28: Comparison on Accuracy with varying number of source tables.

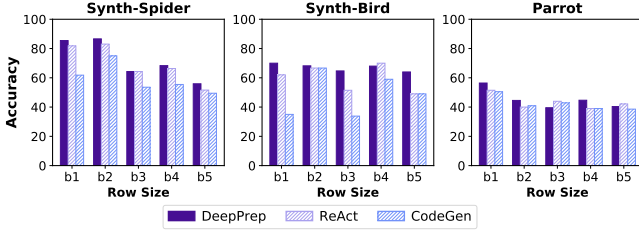


Figure 26: Accuracy comparison on tables with varying number of rows. For Synth-Spider, b1 (1-9), b2 (10-15), b3 (16-21), b4 (22-35), b5 (36-124,842). For Synth-Bird, b1 (1-110), b2 (115-1,000), b3 (1,001-1,022), b4 (1,023-2,004), b5 (2,005-360,006). For Parrot, b1 (2-13), b2 (14-30), b3 (31-100), b4 (107-2,000), b5 (2,075-200,000).

Table 8: Average Execution Time (s) and Memory Usage (MB) under Varying Data and Task Complexity.

Data Type	Group	Synth-Spider		Synth-Bird		Parrot	
		Time	Mem.	Time	Mem.	Time	Mem.
Column Size	b1	8.91	66.31	11.46	77.34	7.09	114.42
	b2	8.21	66.54	12.13	74.07	9.76	116.51
	b3	8.48	67.07	12.12	85.35	11.49	125.12
	b4	10.19	69.32	12.88	83.31	10.36	137.34
	b5	11.30	68.37	13.78	90.70	11.36	143.35
Row Size	b1	7.87	65.69	11.48	88.66	9.54	124.43
	b2	8.13	65.99	11.36	74.92	10.57	124.72
	b3	9.30	66.78	13.57	79.24	9.07	120.89
	b4	10.63	68.33	12.64	80.00	10.13	127.47
	b5	11.72	71.30	13.96	88.31	10.77	135.00
Operator Length	g1	8.54	66.74	11.26	80.08	6.85	114.01
	g2	12.75	70.30	14.58	85.02	11.06	130.69
	g3	14.62	68.53	14.01	81.92	12.22	132.31
Table Number	1	8.06	66.51	9.94	79.00	9.58	124.41
	2	10.54	68.82	13.11	83.38	10.81	128.74
	≥3	13.49	68.54	14.92	81.08	–	–

D.2 Sensitivity Analysis: Data Size

Figures 25 and 26 show the accuracy of DEEPPREP and baselines under varying column sizes and row sizes, respectively. DEEPPREP

remains robust on small and medium-sized tables. Performance degrades only for the largest buckets, where the table sampling strategy may omit relevant information from the sampled content for very large tables.

D.3 Time and Memory Usage

Table 8 reports the average execution time and memory usage across different data and task complexity groups. DEEPPREP exhibits stable growth in both execution time and memory usage as workload complexity increases. For example, on Synth-Spider, the average execution time increases only from 8.91 to 11.30 seconds when moving from the smallest to the largest column-size bucket, while memory usage remains between 66 MB and 69 MB. Similar trends are observed on Synth-Bird and Parrot. These results indicate that DEEPPREP scales gracefully with increasing data size and task complexity.

Why DEEPPREP Scales Stably. The stable scaling behavior can be explained by the generative search framework formalized in Algorithm 4. At each step, the agent generates a $\langle \text{plan} \rangle \psi_t$ (Line 3) and an $\langle \text{expand} \rangle$ action (Line 4), followed by the execution of the proposed operators (Line 7). The process is bounded by the maximum exploration turn T and the maximum operator sequence length L per expansion, yielding a worst-case complexity of $O(T \times L)$. Unlike classical tree-search methods such as MCTS, which explicitly expand and evaluate many candidate branches, DEEPPREP generates only one operator pipeline conditioned on the current tree state. The “pruning” is implicit in the LLM’s decision-making: the agent observes failure traces and avoids revisiting failed branches, rather than enumerating all possible expansions. Consequently, the effective branching factor depends on the LLM’s learned policy and remains bounded regardless of input size, which explains the stable growth in time and memory usage.

D.4 Data Size Bucket Details

For row-size and column-size analyses, we sort all cases by the corresponding metric and divide them into five equal-width buckets $b_1 - b_5$, as described below.

- **The Synth-Spider dataset.**
 - Column buckets: b_1 (1-4), b_2 (5-7), b_3 (8-9), b_4 (10-14), b_5 (15-124,843).
 - Row buckets: b_1 (1-9), b_2 (10-15), b_3 (16-21), b_4 (22-35), b_5 (36-124,842).
- **The Synth-Bird dataset.**
 - Column buckets: b_1 (1-11), b_2 (12-27), b_3 (29-74), b_4 (75-1,003), b_5 (1,004-20,015).
 - Row buckets: b_1 (1-110), b_2 (115-1,000), b_3 (1,001-1,022), b_4 (1,023-2,004), b_5 (2,005-360,006).
- **The Parrot dataset.**
 - Column buckets: b_1 (2-4), b_2 (5-7), b_3 (8-11), b_4 (12-16), b_5 (18-587).
 - Row buckets: b_1 (2-13), b_2 (14-30), b_3 (31-100), b_4 (107-2,000), b_5 (2,075-200,000).

D.5 Pipeline Length and Table Number Analysis

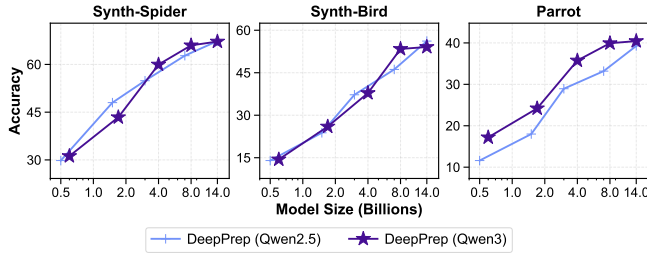


Figure 30: Performance scales with larger model size.

Table 9: Detailed experimental settings.

(a) Hyperparameters for Operator Learning.

Hyperparameter	Value
Base Model	Qwen3-8B
Global Batch Size	256
Learning Rate	1e-5
Epochs	3
Max Sequence Length	16,384
Warmup Ratio	0.05
Precision	bfloat16

(b) Hyperparameters for Tree-based Agentic Reasoning SFT.

Hyperparameter	Value
Base Model	Stage 1 Ckp
Global Batch Size	256
Learning Rate	6e-5
Epochs	15
Max Sequence Length	24,000
Warmup Ratio	0.05
Precision	bfloat16

(c) Hyperparameters for Multi-Turn RL.

Hyperparameter	Value
Algorithm	GRPO
Global Batch Size	64
Learning Rate	1e-6
Epochs	1
Group Size (G)	8
KL Coefficient (β)	0.005
Temperature / Top-p	0.6 / 0.95
Max Total Length	28,000
Max Response Length	2000
Max Exploration Turn	5
Max Prompt Length	12000

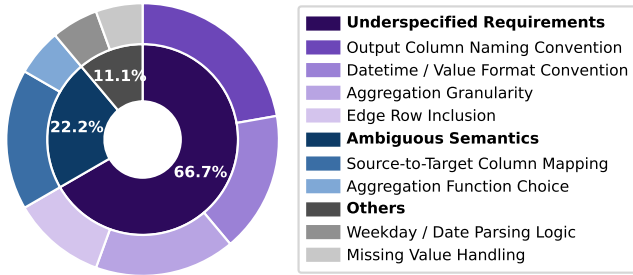


Figure 29: Error analysis on Buildings.

Pipeline Length. For pipeline length, we partition ADP cases into Simple, Medium, and Complex groups according to the number of operators in the target pipeline. As shown in Figure 27, all methods degrade as the pipeline length increases. However, DEEPREP

shows the smallest degradation, and its advantage over baselines becomes larger on Complex cases. On Synth-Spider, DEEPREP achieves 60.5 accuracy on Complex cases, outperforming CodeGen by 16.8 points.

Table Number. As shown in Figure 28, performance generally decreases as more source tables are involved, but DEEPREP consistently achieves the highest accuracy across different table number groups.

D.6 Error Analysis on Real-World Dataset

Figure 29 presents the error analysis of DEEPREP on the Buildings dataset. The analysis reveals three major sources of errors:

- **Ambiguous column semantics** (50% of failures): Real cases often contain abbreviated or domain-specific column names (e.g., hdd, cdd), which lead to selecting the wrong columns required to be mapped or aggregated into the target columns.
- **Underspecified requirements** (33% of failures): In real-world scenarios, the target specification is often described only at a high level, without explicitly defining important details such as the desired aggregation granularity (e.g., daily versus monthly). This increases the reasoning burden of the model during the data preparation stage.
- **Domain logic errors** (11% of failures): Some failures are caused by incorrect domain-specific logic, such as applying the wrong aggregation function or misinterpreting the semantic relationship between columns.

These findings not only provide additional evidence that Buildings captures realistic ADP challenges, but also highlight important directions for future work.

D.7 Model Size Scaling Results

Figure 31 shows the performance of DEEPREP with backbone models of varying sizes across the Qwen2.5 and Qwen3 families. Accuracy steadily increases with model size. On Synth-Spider, scaling Qwen3 from 0.6B to 14B improves accuracy from 31.20 to 67.18. Across comparable model sizes, the Qwen3 series also shows stronger out-of-domain performance than Qwen2.5. For example, on Synth-Bird, Qwen3-8B achieves 53.39 accuracy versus 46.17 for Qwen2.5-7B. These results suggest DEEPREP benefits from stronger backbone models and maintains positive scaling trends.

D.8 Training Hyperparameters

Table 9 summarizes the specific hyperparameters for each training stage of DEEPREP.

Stage 1: Operator Syntax Learning. As defined in Section 5.1, the goal of this stage is to focus on the syntax and semantics of individual operators. We set the learning rate to 1e-5 with a cosine scheduler and a warmup ratio of 0.05. Since this stage mainly involves generating short operator sequences, we set the maximum sequence length to 16,384. We train the model for 3 epochs with a global batch size of 256.

Stage 2: Reasoning Procedure Learning. This stage corresponds to aligning the model with the tree-based agentic reasoning mechanism in Section 5.1. We initialize the model using the checkpoint from Stage 1. The objective is to teach the model to perform global

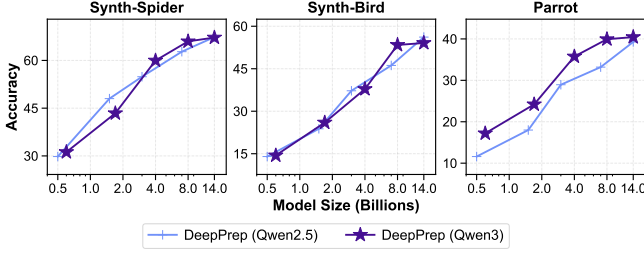


Figure 31: Performance scales with larger model size.

planning, exploration, and backtracking using the agentic reasoning tree. To accommodate the large context of the agentic reasoning tree and multi-turn interaction history, we increase the maximum sequence length to 24,000. We use a learning rate of $6e-5$ to effectively learn the complex reasoning patterns. The training runs for 15 epochs with a global batch size of 256.

Stage 3: Multi-turn RL with Hybrid Reward. In the final stage, we apply the multi-turn RL with hybrid reward described in Section 5.2. We utilize the Group Relative Policy Optimization (GRPO) algorithm to optimize the model’s decision-making strategy. During training, we sample $G = 8$ trajectories for each prompt to estimate the baseline. We use a small learning rate of $1e-6$ and set the KL divergence coefficient (β) to 0.005 to ensure training stability. To handle long-horizon tasks, we set the maximum total length to 28,000 and the maximum prompt length to 12,000. We limit the maximum exploration turns to 5 and the response length for each step to 2,000 tokens.

Training Data. Due to time and computational constraints, we utilized a curated subset of the synthesized data for training rather than the full dataset. For the initial stage, we constructed a dataset of 8,000 samples. These samples focus on basic operator usage to establish the model’s fundamental understanding of the operator space. In the SFT stage, we employed five open-source LLMs as teacher models to generate diverse reasoning trajectories. We initially distilled approximately 20,000 trajectories. To ensure high training quality, we filtered these trajectories based on the process reward function R_{lm} score (retaining scores > 0.8) and the number of interaction turns. This process yielded a final dataset of 11,198 high-quality samples. For the reinforcement learning stage, we randomly sampled 4,000 instances from the Synth-Spider dataset.

D.9 Model Size Scaling Study

Exp-8: Model Size Scaling Study. We study how DEEPPREP performs under different backbone model sizes. We train DEEPPREP with LLM backbones ranging from 0.5B to 14B parameters across the Qwen2.5 and Qwen3 families. Figure 31 shows that accuracy steadily increases with model size. On Synth-Spider, scaling Qwen3 from 0.6B to 14B improves Accuracy from 31.20 to 67.18. Across comparable model sizes, the Qwen3 series also shows stronger out-of-domain performance than Qwen2.5. For example, on Synth-Bird, Qwen3-8B achieves 53.39 Accuracy versus 46.17 for Qwen2.5-7B. These results suggest DEEPPREP benefits from stronger backbone models and maintains positive scaling trends.

Finding 8: The performance of DEEPPREP improves with larger backbone model sizes, which suggest that DEEPPREP benefits from stronger backbone models and maintains positive scaling trends.

E RUNNING CASE STUDY

In this section, we provide a complete running example of DEEPPREP to illustrate the tree-based agentic reasoning process described in Section 4.

Figure 32 shows a real case of DEEPPREP on a data preparation task. Given source tables containing movie, rating, and director information, the goal is to produce a target table with specific columns including movie title, genre, rating, and director name.

Step 1: Initial Planning. The agent first analyzes the source tables and the target schema specification using the `<plan>` action. It identifies that: (1) the *movies* table contains duplicate records that need to be removed; (2) the genre column contains composite values (e.g., “Sci-Fi/Action”) that need to be split into individual rows; and (3) information from multiple tables needs to be joined together.

Step 2: First Exploration. The agent generates an operator pipeline using the `<expand>` action: Deduplicate $\rightarrow$ Explode $\rightarrow$ Join. The environment executes these operators and returns the intermediate tables. However, the Join operator produces an incorrect result because the join key is not properly specified.

Step 3: Backtracking and Correction. Observing the execution feedback, the agent backtracks to the state after Explode and generates a revised Join operator with the correct join key. The environment executes the revised operator, and the intermediate table now correctly merges movie and director information.

Step 4: Final Pipeline. The agent continues with additional operators (SelectColumn, RenameColumn) to align the intermediate table with the target schema. After verifying the final result, the agent issues the `<answer>` action to output the pipeline.

This example illustrates three key features of DEEPPREP’s tree-based agentic reasoning: (1) *Execution grounding*: Each operator is executed on real data, providing concrete feedback for subsequent decisions. (2) *Failure-aware exploration*: When an operator fails or produces incorrect results, the agent can backtrack to a valid state and try an alternative approach. (3) *Generative tree reasoning*: The agent reasons over the entire search tree, including intermediate tables and failure traces, to generate subsequent actions.

F TRAINING FRAMEWORK DETAILS

In this section, we provide additional details on the Progressive Agentic Training framework described in Section 5, including the cold-start curriculum stages, environment token masking, and hybrid reward design.

F.1 Cold-Start Curriculum Stage 1: Operator Syntax Learning

As described in Section 5.1, Stage 1 focuses on the syntax and semantics of individual operators. We provide additional details on the data construction process.

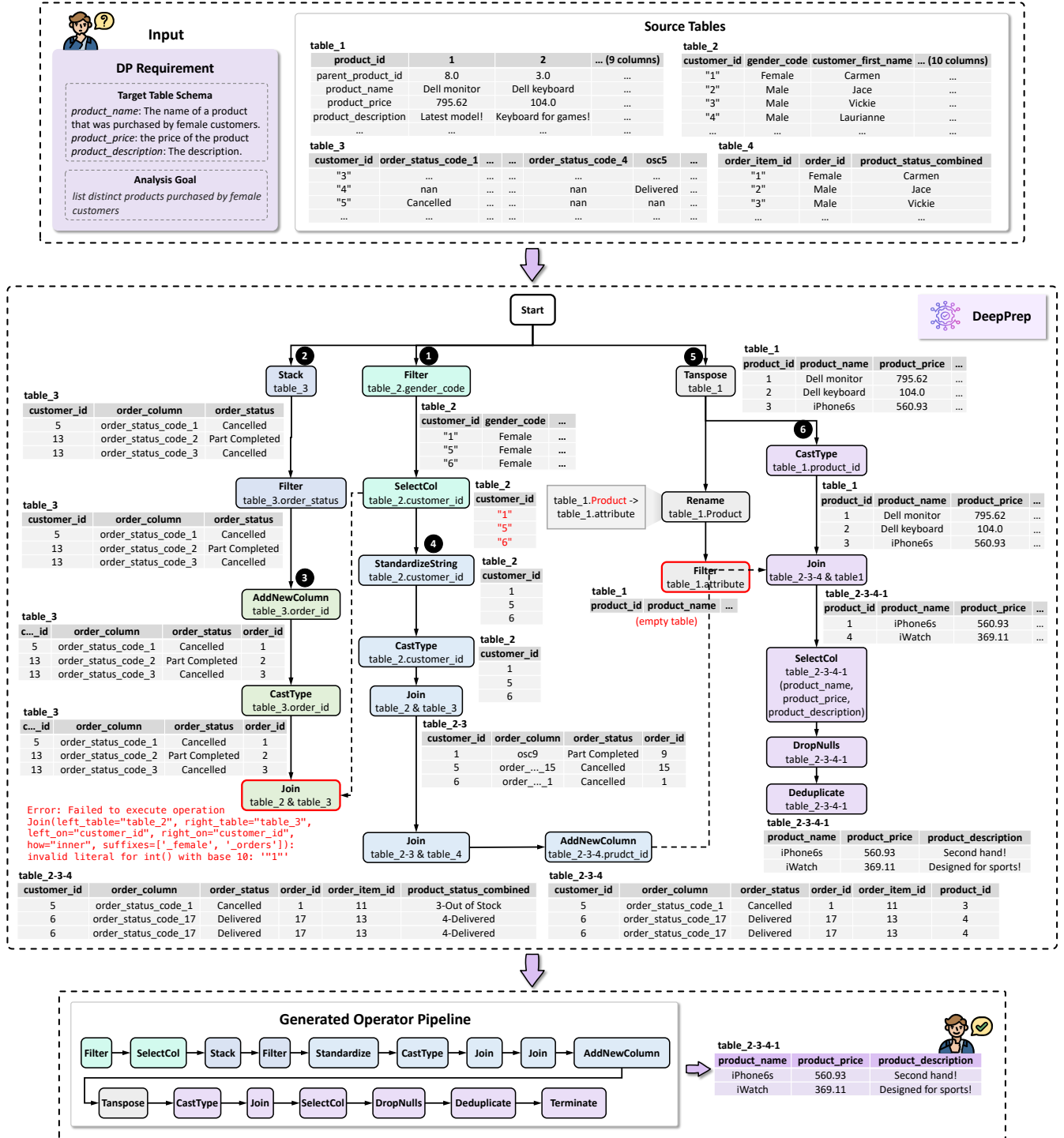


Figure 32: A real case of DEEPPREP. We only show the first 3 rows of intermediate tables due to the space limit.

Data Construction. Given source tables and a ground-truth operator pipeline $\mathcal{P}_{\text{total}} = (o^{(1)}, \dots, o^{(N)})$, we first execute these operators sequentially to collect a sequence of intermediate table sets

$\{\mathcal{T}_0, \dots, \mathcal{T}_N\}$. Then, we sample a contiguous sub-sequence of operators $\mathcal{P}_{\text{sub}} = (o^{(i)}, \dots, o^{(j)})$ with the corresponding input intermediate table set $\mathcal{T}_{\text{in}} = \mathcal{T}_{i-1}$ and output intermediate table set $\mathcal{T}_{\text{out}} = \mathcal{T}_j$.

Sub-sequence Sampling Strategy. To ensure diverse and balanced training data, we employ the following sampling strategy:

- **Variable length.** We sample sub-sequences of varying lengths (1 to 5 operators) to expose the model to both single-operator and multi-operator transformations.
- **Random starting point.** For each ground-truth pipeline, we randomly select the starting position i and then sample the ending position j based on the desired sub-sequence length.
- **Coverage.** We ensure that all operator types appear in the training data with sufficient frequency by oversampling rare operator types.

Through this process, we construct the operator syntax learning dataset $\mathcal{D}_{\text{op}}$ consisting of training triples $(\mathcal{P}_{\text{sub}}, \mathcal{T}_{\text{in}}, \mathcal{T}_{\text{out}})$, and optimize the loss $\mathcal{L}_{\text{op}}$ defined in Eq. (4).

F.2 Cold-Start Curriculum Stage 2: Reasoning Procedure Learning

Stage 2 aligns the model with the tree-based agentic reasoning mechanism. We provide additional details on the teacher model distillation process.

Teacher Model Distillation. We employ five open-source LLMs as teacher models to generate diverse reasoning trajectories. For each ADP task defined by source tables $\mathcal{S}$ and a target schema Σ^* , we use In-Context Learning with elaborate prompts that exemplify the tree-based reasoning process. The prompts include:

- The complete operator space definition.
- Two few-shot examples demonstrating the $\langle \text{plan} \rangle$ – $\langle \text{expand} \rangle$ – $\langle \text{execute} \rangle$ – $\langle \text{answer} \rangle$ interaction protocol.
- The current source tables and target schema specification.

Input Perturbation Φ . To enhance robustness, we apply input perturbations Φ to the source tables before generating trajectories. Specifically, we:

- Randomly shuffle the order of columns in each source table.
- Randomly shuffle the order of rows in each source table.
- Randomly shuffle the order of source tables when multiple tables are present.

These perturbations ensure that the model does not rely on spurious positional correlations and learns to identify the correct columns and tables based on semantic content rather than position.

Process Reward Filtering. The generated candidate trajectories are filtered using the process reward function $R_{\text{llm}}(\tau)$ (Section 5.2). We retain only trajectories with $R_{\text{llm}}(\tau) > 0.8$ to ensure high training quality. This filtering process reduces the initial 20,000 candidate trajectories to 11,198 high-quality samples.

F.3 Environment Token Masking

As described in Section 5.2, each trajectory contains both agent-generated tokens and environment-generated tokens. We provide additional details on the binary token mask implementation.

Masking Mechanism. In our interaction protocol, a trajectory $\tau = (r_1, \dots, r_m)$ interleaves agent actions and environment observations. Specifically:

- **Agent-generated tokens** (unmasked, $M_t = 1$): Tokens within $\langle \text{plan} \rangle$, $\langle \text{expand} \rangle$, and $\langle \text{answer} \rangle$ blocks. These correspond to the agent’s decision-making outputs and should be optimized by the policy gradient.
- **Environment-generated tokens** (masked, $M_t = 0$): Tokens within $\langle \text{execute} \rangle$ blocks. These are deterministic execution outputs (e.g., serialized intermediate tables, error traces) that are not under the agent’s control and should not contribute to policy gradient updates.

The binary mask M_t is applied to the loss function in Eq. (5) during SFT and to the policy gradient objective during RL. This ensures that the model is only optimized for its own decision-making, not for predicting environment outputs.

F.4 Hybrid Reward Details

We provide additional details on the hybrid reward design described in Section 5.2.

Hyperparameters. The hybrid reward weights are set to $\alpha = 1$, $\beta = 1$, and $\gamma = 1$, i.e., the total reward is the direct sum of the three components: $R(\tau) = R_{\text{out}}(\tau) + R_{\text{part}}(\tau) + R_{\text{llm}}(\tau)$. Note that R_{out} takes a value of 2.0 for exact match and 0.0 otherwise, while $R_{\text{part}} \in [0, 1]$ and $R_{\text{llm}} \in [0, 1]$. This design gives the outcome reward a dominant weight (up to 2.0) to strongly incentivize exact correctness, while the partial and process rewards provide denser learning signals for trajectories that do not achieve exact match.

LLM-as-Judge Scoring. The process reward $R_{\text{llm}}(\tau)$ is computed by an instruction-tuned LLM (e.g., GPT-4o) that evaluates each trajectory on three criteria:

- **Plan–action consistency** (weight: 1/3): Whether the generated operator pipeline (from $\langle \text{expand} \rangle$) correctly implements the plan (from $\langle \text{plan} \rangle$).
- **Feedback responsiveness** (weight: 1/3): Whether subsequent $\langle \text{plan} \rangle$ outputs explicitly respond to the prior execution feedback (e.g., reported error traces).
- **Backtracking justification** (weight: 1/3): Whether parent-node switches are supported by recorded failure evidence from the current branch.

Each criterion is scored on a scale of 0 to 1, and the final $R_{\text{llm}}(\tau)$ is the average of the three scores. The LLM judge is prompted with the full trajectory including all $\langle \text{plan} \rangle$, $\langle \text{expand} \rangle$, and $\langle \text{execute} \rangle$ blocks, along with the evaluation rubric.